P802.1DD/Draft 1.5

April 20, 2026

Draft Standard for Local and metropolitan area networks—

Resource Allocation Protocol

Unapproved draft, prepared by the

Time-Sensitive Networking (TSN) Task Group of IEEE 802.1

Sponsored by the

LAN/MAN Standards Committee
of the
IEEE Computer Society

This and the following cover pages are not part of the draft. They provide revision and other information for IEEE 802.1 Working Group members and participants in the IEEE Standards Association ballot process, and will be updated as convenient. New participants: Please read these cover pages, they contain information that should help you contribute effectively to this standards development project. Blank pages allow for the addition of cross-references to changed text without forcing renumbering of all pages in the draft. Pages are numbered from 1 (including cover pages) for the convenience of reviewers whose PDF viewers do not easily accommodate different numbering sequences. Pages will of course be renumbered prior to publication.

The text proper of this draft begins with the Title page.

Editors' Foreword

Throughout this document any notes with a Cyan background (as in this paragraph) are temporary, inserted by the Editors for a variety of purposes. They will be removed prior to SA Ballot and are not part of the normative text. These cover pages will be edited, but will be retained so that information for the stages of SA ballot can be included (including, if appropriate, cross-references to text changed in the course of SA balloting). To avoid changes to page numbering between final WG ballot and initial SA Ballot, the number of pages in this set of cover pages should remain unchanged, with pages intentionally left blank marked as such. The records of participants in the development of the standard will be added at an appropriate time.

Participation in 802.1 standards development

All participants in the standardization activities of IEEE 802.1 should be aware of the Working Group Policies and Procedures, and the fact that they have obligations under the IEEE Patent Policy, the IEEE Standards Association (SA) Copyright Policy, and the IEEE SA Participation Policy. For information on these policies see 1.ieee802.org/rules/ and the slides presented at the beginning of each of our Working Group and Task Group meeting.

As part of our IEEE 802® process, the text of the PAR (Project Authorization Request) and CSD (Criteria for Standards Development) of each project is reviewed regularly to ensure their continued validity. The PAR is summarized in these cover pages and links are provided to the full text of both PAR and CSD. A vote of "Approve" on this draft is also an affirmation that the PAR and CSD for this project are still valid.

Comments on this draft are encouraged. NOTE: All issues related to IEEE standards presentation style, formatting, spelling, etc. are routinely handled between the 802.1 Editor and the IEEE Staff Editors prior to publication, after balloting and the process of achieving agreement on the technical content of the standard is complete. Readers are urged to devote their valuable time and energy only to comments that materially affect either the technical content of the document or the clarity of that technical content. Comments should not simply state what is wrong, but also what might be done to fix the problem.

Full participation in the work of IEEE 802.1 requires attendance at IEEE 802 meetings. Information on 802.1 activities, working papers, and email distribution lists etc. can be found on the 802.1 Website:

http://ieee802.org/1/

Use of the email distribution list is not presently restricted to 802.1 members, and the working group has a policy of considering comments from all who are interested and willing to contribute to the development of the draft. Individuals not attending meetings have helped to identify sources of misunderstanding and ambiguity in past projects. The email lists exist primarily to allow the members of the working group to develop standards, and are not a general forum. All contributors to the work of 802.1 should familiarize themselves with the IEEE patent policy and anyone using the email distribution list will be assumed to have done so. Information can be found at http://standards.ieee.org/db/patents/

Comments on this draft may be sent to the 802.1 email exploder, to the Editor, or to the Chairs of the 802.1 Working Group and Time-Sensitive Networking (TSN) Task Group.

Martin Mittelberger
Editor, P802.1DD
Email:martin.mittelberger@siemens.com

Janos Farkas
Chair, 802.1 TSN Task Group

Email:Janos.Farkas@ericsson.com

Glenn Parsons
Chair, 802.1 Working Group
+1 514-379-9037
Email: glenn.parsons@ericsson.com

NOTE: Comments whose distribution is restricted in any way cannot be considered, and may not be acknowledged.

All participants in IEEE standards development have responsibilities under the IEEE patent policy and should familiarize themselves with that policy, see
http://standards.ieee.org/about/sasb/patcom/materials.html

As part of our IEEE 802 process, the text of the PAR and CSD (Criteria for Standards Development, formerly referred to as the 5 Criteria or 5C's) is reviewed on a regular basis in order to ensure their continued validity. A vote of “Approve” on this draft is also an affirmation by the balloter that the PAR is still valid.

Draft development

During the early stages of draft development, 802.1 editors have a responsibility to attempt to craft technically coherent drafts from the resolutions of ballot comments and from the other discussions that take place in the working group meetings. Preparation of drafts often exposes inconsistencies in editor’s instructions or exposes the need to make choices between approaches that were not fully apparent in the meeting. Choices and requests by the editors’ for contributions on specific issues will be found in the editors’ Introduction to the current draft and at appropriate points in the draft.

The ballot comments received on each draft, and the editors’ proposed and final disposition of comments on working group drafts, are part of the audit trail of the development of the standard and are available, along with all the revisions of the draft on the 802.1 website (for address see above).

During the early stages of draft development the proposed text can be moved around a great deal, and even minor rearrangement can lead to a lot of ‘change’, not all of which is noteworthy from the point of the reviewer, so the use of automatic change bars is not very effective. In early drafts change bars may be omitted or applied manually, with a view to drawing the readers attention to the most significant areas of change. Readers interested in viewing every change are encouraged to use Adobe Acrobat to compare the document with their selected prior draft. Note that the FrameMaker change bar feature is useless when it comes to indicating changes to Figures.

iPAR (Project Authorization Request) and CSD

Extracts from the PAR, as approved by IEEE NesCom 26th September 2024:

https://development.standards.ieee.org/myproject-web/public/view.html#pardetail/11709

and the CSD (Criteria for Standards Development):

https://www.ieee802.org/1/files/public/docs2024/dd-CSD-0724-v01.pdf

PAR Scope:

This standard specifies protocols, procedures, and managed objects for resource allocation in bridged local area networks for dynamic creation and maintenance of data streams. This standard supports control signaling through data paths and through separate control paths in support of centralized control. This standard makes provisions for backward compatibility with the Stream Reservation Protocol specified in IEEE Std 802.1Q.

PAR Need for the Project:

Successful deployment of current protocol has revealed the need to accommodate a larger number of streams and resource allocation on simultaneously available redundant paths than can be accommodated by the current in-band signaling mechanisms.

CSD Broad market potential:

The original version of the Stream Reservation Protocol (SRP) specified in IEEE Std 802.1Q has been successfully and widely accepted by the professional audio/video, consumer, and automotive markets as an essential tool to realize automatic stream setup with dynamic resource allocation. The success of SRP has expanded the requirements on that protocol beyond that capability.

Individuals affiliated with multiple vendors and users for industrial automation, professional audio/video, automotive and other systems requiring a protocol to signal the resource reservation along the end-to-end paths of streams for time-sensitive applications will participate in the development of the project.

CSD Distinct Identity:

No existing IEEE 802 standard provides dynamic resource allocation that supports the required performance and capabilities.

This standard expands bridged network capabilities in the area of resource allocation. The rate of addition of other new capabilities to existing IEEE 802.1 standards means that it is more practical to specify expanded resource allocation capabilities in a separate standard focused on that topic. The proposed project replaces the previous IEEE P802.1Qdd Draft Standard for Local and Metropolitan Area Networks — Bridges and Bridged Networks — Amendment: Resource Allocation Protocol.

CSD Technical Feasibility:

The proposed resource allocation methods are similar in principle to those of the Stream Reservation Protocol (SRP) specified in IEEE Std 802.1Q.

There is a considerable body of experience in supplying data streams with guarantees for quality of service parameters such as latency, latency variation, or bandwidth. Mechanisms needed for this project are widely used by other protocols already, e.g., IEEE 802.1Q SRP for use in bridged networks and the Resource Reservation Protocol (RSVP, IETF RFC 2205, and IETF RFC 2750) for routers and hosts that use the Internet Protocol.

CSD Economic Feasibility:

The well-established balance between infrastructure and attached stations will not be changed by the proposed standard.

All the above cost factors are well-known from the existing resource allocation protocols and registration database distribution methods.

Introduction to the current draft

This introduction is not part of the draft, and will be revised for SA ballot. A set of cover pages will be retained for use during SA ballot.

D1.5

Changes from TG ballot on D1.4.

D1.4

D1.4 comprises all architectural changes and is intended for TG-ballot.

D1.3

In D1.3 the new architecture was updated throughout clause 6. All other parts remained mainly unchanged with regard to the technical changes in architecture and will be adapted after there is consensus about clause 6.This draft is intended for further discussions before starting TG ballot.

D1.2

In D1.2 the architecture was changed to a MRP based architecture as discussed at the May 2025 Interim Session in Rennes. It is intended to discuss the new structure of the document.

D1.1

D1.0 was not used for TG ballot but as editor's draft.- D1.1 is an editor's draft as well with following changes:

Fixes some inconsistencies of D1.0

Adds backward compatibility (RAP/MSRP Gateway) skeleton structure.

Restructures Token Bucket TSpec clause.

D1.0

D1.0 is the first draft of P802.1DD and used for the first Task Group ballot.

Due to project change (P802.1DD replacing P802.1Qdd), this draft has been created based on the contents as well as the comment resolution results of P802.1Qdd/D0.9 but using a template for stand-alone IEEE 802.1 standards. In addition to the changes required to fit the new format, such as rearrangement of clauses and reworked references to other standards, the major changes from P802.1Qdd D0.9 include the following:

- Clause 5.Conformance reworked to define requirements for three types of RAP instance and ten types of RAP systems.
- Clause 6.1RAP overview added to give a brief overview of RAP.
- In Clause 6.3.1RAP architecture, added text and figures for describing the architecture of RAP/MSRP Gateway, an example of RAP Proxy systems, and RAP control of Bridging and FRER functions.
- Clause 6.3.9Traffic specification usage rules and types added to define TSpec types and usage rules.
- In all the state machines, replaced the use of primitives in conditions of state machine transitions with the use of event pointer variables as described in Clause 6.2.3Event pointer variables.
- Clause 8.YANG data model for RAP updated to reflect the changes made in Clause 7.Resource Allocation Protocol (RAP) management.

This page intentionally left blank.

This page intentionally left blank.

P802.1DD/Draft 1.5
April 20, 2026

Draft Standard for Local and metropolitan area networks—

Resource Allocation Protocol

Unapproved draft, prepared by the
Time-Sensitive Networking (TSN) Task Group of IEEE 802.1

Sponsored by the
LAN/MAN Standards Committee
of the
IEEE Computer Society

IEEE Standards Activities Department
445 Hoes Lane
Piscataway, NJ 08854, USA

Abstract: This standard specifies protocols, procedures, and managed objects for resource allocation in bridged local area networks for dynamic creation and maintenance of data streams. This standard supports control signaling through data paths and/or through separate control paths in support of centralized control. This standard makes provisions for backward compatibility with the Stream Reservation Protocol specified in IEEE Std 802.1Q.

Keywords: Bridged Network, Frame Replication and Elimination for Reliability, FRER, IEEE 802.1CB™, IEEE 802.1CS™, IEEE 802.1DD™, IEEE 802.1Q™, Link-local Registration Protocol, LRP, resource allocation, Resource Allocation Protocol, RAP, Time-Sensitive Networking, TSN, Virtual Bridged Local Area Network, VLAN Bridge

Important Notices and Disclaimers Concerning IEEE Standards Documents

IEEE Standards documents are made available for use subject to important notices and legal disclaimers. These notices and disclaimers, or a reference to this page (https://standards.ieee.org/ipr/disclaimers.html), appear in all standards and may be found under the heading “Important Notices and Disclaimers Concerning IEEE Standards Documents.”

Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

IEEE Standards documents are developed within the IEEE Societies and the Standards Coordinating Committees of the IEEE Standards Association (IEEE SA) Standards Board. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE Standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers are not necessarily members of IEEE or IEEE SA, and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

IEEE makes no warranties or representations concerning its standards, and expressly disclaims all warranties, express or implied, concerning this standard, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. In addition, IEEE does not warrant or represent that the use of the material contained in its standards is free from patent infringement. IEEE standards documents are supplied “AS IS” and “WITH ALL FAULTS.”

Use of an IEEE standard is wholly voluntary. The existence of an IEEE Standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

In publishing and making its standards available, IEEE is not suggesting or rendering professional or other services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any other person or entity to another. Any person utilizing any IEEE Standards document, should rely upon his or her own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

Translations

The IEEE consensus development process involves the review of documents in English only. In the event that an IEEE standard is translated, only the English version published by IEEE is the approved IEEE standard.

Official statements

A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations Manual shall not be considered or inferred to be the official position of IEEE or any of its committees and shall not be considered to be, nor be relied upon as, a formal position of IEEE. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall make it clear that the presenter's views should be considered the personal views of that individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working Group.

Comments on standards

Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of membership affiliation with IEEE or IEEE SA. However, **IEEE does not provide interpretations, consulting information, or advice pertaining to IEEE Standards documents**.

Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is important that any responses to comments and questions also receive the concurrence of a balance of interests. For this reason, IEEE and the members of its Societies and Standards Coordinating Committees are not able to provide an instant response to comments, or questions except in those cases where the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation requests. Any person who would like to participate in evaluating comments or in revisions to an IEEE standard is welcome to join the relevant IEEE working group. You can indicate interest in a working group using the Interests tab in the Manage Profile & Interests area of the IEEE SA myProject system.[1] An IEEE Account is needed to access the application.

Comments on standards should be submitted using the Contact Us form.[2]

Laws and regulations

Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the provisions of any IEEE Standards document does not imply compliance to any applicable regulatory requirements. Implementers of the standard are responsible for observing or referring to the applicable regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in compliance with applicable laws, and these documents may not be construed as doing so.

Data privacy

Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and data ownership in the context of assessing and using the standards in compliance with applicable laws and regulations.

1. Available at: https://development.standards.ieee.org/myproject-web/public/view.html#landing.
2. Available at: https://standards.ieee.org/content/ieee-standards/en/about/contact/index.html.

Copyrights

Photocopies

Updating of IEEE Standards documents

Users of IEEE Standards documents should be aware that these documents may be superseded at any time by the issuance of new editions or may be amended from time to time through the issuance of amendments, corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the document together with any amendments, corrigenda, or errata then in effect.

Every IEEE standard is subjected to review at least every ten years. When a document is more than ten years old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE standard.

In order to determine whether a given document is the current edition and whether it has been amended through the issuance of amendments, corrigenda, or errata, visit IEEE Xplore or contact IEEE.[3] For more information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

Errata

Errata, if any, for all IEEE standards can be accessed on the IEEE SA Website.[4] Search for standard number and year of approval to access the web page of the published standard. Errata links are located under the Additional Resources Details section. Errata are also available in IEEE Xplore. Users are encouraged to periodically check for errata.

Patents

IEEE Standards are developed in compliance with the IEEE SA Patent Policy.[5]

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the

3. Available at: https://ieeexplore.ieee.org/browse/standards/collection/ieee.
4. Available at: https://standards.ieee.org/standard/index.html.
5. Available at: https://standards.ieee.org/about/sasb/patcom/materials.html.

IEEE SA Website at http://standards.ieee.org/about/sasb/patcom/patents.html. Letters of Assurance may indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair discrimination to applicants desiring to obtain such licenses.

Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

IMPORTANT NOTICE

IEEE Standards do not guarantee or ensure safety, security, health, or environmental protection, or ensure against interference with or from other devices or networks. IEEE Standards development activities consider research and information presented to the standards development group in developing any safety recommendations. Other information about safety practices, changes in technology or technology implementation, or impact by peripheral systems also may be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, and interference protection practices and all applicable laws and regulations.

Participants

<<The following lists will be updated in the usual way prior to publication>>

At the time this standard was completed, the IEEE 802.1 working group had the following membership:

Glenn Parsons, *Chair*
Jessy Rouyer, *Vice Chair*
Janos Farkas, *Time-Sensitive Networking Task Group Chair*
Martin Mittelberger, *Editor*

The following members of the individual balloting committee voted on this standard. Balloters may have voted for approval, disapproval, or abstention.

A.N. Other

<<The above lists will be updated in the usual way prior to publication>>

When the IEEE-SA Standards Board approved this standard on <dd> <month> <year>, it had the following membership:

Chair
Vice-Chair
Past Chair
Secretary

*Member Emeritus

<<The above lists will be updated in the usual way prior to publication>>

Introduction

This introduction is not part of IEEE Std 802.1DD-20XX, IEEE Standard for Local and metropolitan area networks—Resource Allocation Protocol

This standard specifies the Resource Allocation Protocol.

This standard contains state-of-the-art material. The area covered by this standard is undergoing evolution. Revisions are anticipated within the next few years to clarify existing material, to correct possible errors, and to incorporate new related material. Information on the current revision state of this and other IEEE 802 standards may be obtained from

Secretary, IEEE-SA Standards Board
445 Hoes Lane
Piscataway, NJ 08854-4141
USA

Contents

Figures

Tables

Draft Standard for Local and Metropolitan Networks —

Resource Allocation Protocol

1. Overview

The standardization of Ethernet communication technology in IEEE Std 802.3™, specifying transmission over the physical media of individual Local Area Networks (LANs), and in IEEE Std 802.1Q™, specifying Bridges that interconnect IEEE 802® LANs,[1] has facilitated widespread deployment of networks that connect significantly more end stations, with significantly greater bandwidth, and at significantly reduced cost compared to prior technology. All these metrics have been improved by several orders of magnitude—reducing costs through the multi-vendor provision of common components (bridges, end station interfaces, integrated circuit and circuit designs, connectors, and software) for a wide range of network applications.

The use of Ethernet communication technology in networks with high-reliability and deterministic latency requirements is further supported by Time-Sensitive Networking (TSN) provisions in IEEE Std 802.1Q, IEEE Std 802.1AS, IEEE Std 802.1CB, IEEE Std 802.1CS, and the security provisions in IEEE Std 802.1AE and IEEE Std 802.1X. The provisions in these standards can be used in various ways, and include options that address different network requirements and parameters that vary by network and application scale. Network design, time to deploy, and component development, selection, validation, and configuration for a particular network can all benefit from consistent choices, across similar networks and network applications, of the provisions, parameters, and options specified in the relevant standards.

This standard specifies protocols, procedures, and managed objects to support the Resource Allocation Protocol (RAP). RAP is designed as an application of the Link-local Registration Protocol (LRP) specified by IEEE Std 802.1CS to provide automatic and dynamic stream setup with resource allocation for time-sensitive applications in need of bounded latency, zero-congestion loss and high availability and reliability.

1.1 Scope

This standard specifies protocols, procedures, and managed objects for resource allocation in bridged local area networks for dynamic creation and maintenance of data streams. This standard supports control signaling through data paths and through separate control paths in support of centralized control. This standard makes provisions for backward compatibility with the Stream Reservation Protocol specified in IEEE Std 802.1Q.

1.2 Purpose

<No purpose is provided in PAR>

[1] IEEE and IEEE 802 are registered trademarks in the U.S. Patent & Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated.

1.3 Introduction

To enable decentralized resource allocation for the streams that require using the IEEE 802.1 Time-Sensitive Networking (TSN) features, such as the QoS functions defined by IEEE Std 802.1Q and the seamless redundancy techniques defined by IEEE Std 802.1CB, and to leverage the performance and scalability of the LRP defined by IEEE Std 802.1CS, this standard specifies protocols, procedures, and managed objects for a Resource Allocation Protocol (RAP). RAP provides interoperability with the Multiple Stream Registration Protocol (MSRP) specified in IEEE Std 802.1Q. To this end, it

a) Specifies the requirements for RAP implementations declaring conformance to this standard.

b) Defines the principles of RAP operation, including the following.

    1) RAP architecture

    2) RA class and RA advertisement

    3) Talker Announcement and Listener Attachment

    4) Resource allocation constraints

    5) Traffic specification usage rules and types

    6) Stream Rank and reservation importance

    7) Resource allocation for seamless redundancy

    8) Relationship of RAP to MSRP

c) Defines LRP protocol elements for RAP.

d) Defines RAP attributes and TLV encoding

e) Specifies the RAP Endpoint Service Interface (RESI) and the operation of the RAP Endpoint.

f) Specifies the operation of the RAP MAD.

g) Specifies the operation ot the RAP MAP extensions.

h) Defines managed objects for management of RAP.

i) Defines YANG data models for RAP.

j) Provides examples of resource allocation using RAP.

k) Provides RA class template definitions.

1.4 Reference conventions

This standard makes frequent references to specific subclauses in several other IEEE 802.1 standards. To avoid repeating the full names of these referenced standards, the following conventions are used:

— A reference to “subclause x.y in IEEE Std 802.1Q-2022” is of the form: “[Q]:x.y”.

— A reference to “subclause x.y in IEEE Std 802.1CS-2020” is of the form: “[CS]:x,y”.

— A reference to “subclause x.y in IEEE Std 802.1CB-2020” is of the form: “[CB]:x,y”.

— A reference to “subclause x.y in IEEE Std 802.1AB-2016” is of the form: “[AB]:x,y”.

— A reference to “subclause x.y in IEEE Std 802.1AC-2016” is of the form: “[AC]:x,y”.

— A reference to “subclause x.y in this standard” is of the form: “x,y”.

2. Normative references

The following referenced documents are indispensable for the application of this document (i.e., they must be understood and used, so each referenced document is cited in the text and its relationship to this document is explained). For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments or corrigenda) applies.

IEEE Std 802®, IEEE Standard for Local and metropolitan area networks: Overview and Architecture.[2,3]

IEEE Std 802.1AB™, IEEE Standard for Local and metropolitan area networks—Station and Media Access Control Connectivity Discovery.

IEEE Std 802.1AC™, IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Service Definition.

IEEE Std 802.1CB™, IEEE Standard for Local and metropolitan area networks—Frame Replication and Elimination for Reliability.

IEEE Std 802.1CS™, IEEE Standard for Local and metropolitan area networks—Link-local Registration Protocol.

IEEE Std 802.1Q™, IEEE Standard for Local and metropolitan area networks—Bridges and Bridged Networks.

IEEE Std 802.3™, IEEE Standard for Ethernet.

IETF RFC 7950, The YANG 1.1 Data Modeling Language, August 2016.

IETF RFC 8343, A YANG Data Model for Interface Management, March 2018.

[2] IEEE publications are available from The Institute of Electrical and Electronics Engineers (https://standards.ieee.org/).
[3] The IEEE standards or products referred to in this clause are trademarks of The Institute of Electrical and Electronics Engineers, Inc.

3. Definitions

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause.[4]

This standard makes use of the following term defined in IEEE Std 802:[5]

- end station
- frame
- station

This standard makes use of the terms defined in IEEE Std 802.1CS:

- Portal
- record
- target port
- local target port
- LRP application
- neighbor target port

This standard makes use of the terms defined in IEEE Std 802.1CB:

- Compound Stream
- Member Stream

This standard makes use of the following terms defined in IEEE Std 802.1Q:[6]

- Bridge
- Bridge Port
- Bridged Network
- Edge Control Protocol (ECP)
- latency
- Listener
- Multicast
- Port
- Port State
- redundant trees
- Stream
- StreamID
- stream reservation (SR) class
- Talker
- time-sensitive stream
- traffic specification (TSpec)
- traffic class
- Virtual Local Area Network (VLAN)
- Virtual Local Area Network (VLAN) Bridge component
- Virtual Local Area Network (VLAN) Bridged Network

The following terms are specific to this standard:

FRER-capable RAP Bridge: A Bridge that is FRER-capable.

[4] *IEEE Standards Dictionary Online* is available at https://dictionary.ieee.org.

[5] This definition is from IEEE Std 802-2014. Notes to definitions are particular to this standard.

[6] These definitions are from IEEE Std 802.1Q-2022. Notes to definitions are particular to this standard.

FRER-capable RAP Listener: An end station acting as a Listener that is FRER-capable.

FRER-capable RAP Talker: An end station acting as a Talker that is FRER-capable.

resource allocation (RA) class: A priority of a traffic class or a set of traffic classes whose bandwidth and queue resources can be reserved for streams using the Resource Allocation Protocol (RAP).

Resource Allocation Protocol (RAP): A protocol providing resource reservation for streams.

RAP instance: An application instance of the Link-local Registration Protocol (LRP) that provides RAP functionality for an end station or Bridge.

stream: A unidirectional flow of data from one Talker to one or more Listeners that require transmission with a bounded latency.

NOTE—The term “stream”, as used in this standard, is in line with the term “time-sensitive stream” defined in [Q].

4. Abbreviations and acronyms

The following abbreviations and acronyms are used in this standard: [7]

ATS	Asynchronous Traffic Shaping
AV	audio/video
CBS	committed burst size
CFM	Connectivity Fault Management
CID	Company ID [8]
CQF	cyclic queuing and forwarding
CRC	Cyclic Redundancy Check
EISS	Enhanced Internal Sublayer Service
FDB	Filtering Database
FRER	Frame Replication and Elimination for Reliability (IEEE Std 802.1CB)
IP	Internet Protocol
IPv4	Internet Protocol version 4
IPv6	Internet Protocol version 6
ISS	Internal Sublayer Service
LAN	Local Area Network
LLC	Logical Link Control
LLDP	Link Layer Discovery Protocol (IEEE Std 802.1AB)
LLDPDU	Link Layer Discovery Protocol Data Unit
LRP	Link-local Registration Protocol (IEEE Std 802.1CS)
LRP-DS	Link-local Registration Protocol–Database Synchronization
LRP-DT	Link-local Registration Protocol–Data Transport
LRPDU	Link-local Registration Protocol Data Units
MAC	Medium Access Control
MAD	MRP Attribute Declaration
MAP	MRP Attribute Propagation
MMRP	Multiple MAC Registration Protocol
MRP	Multiple Registration Protocol
MRTs	Maximally Redundant Trees
MSRP	Multiple Stream Registration Protocol
MVRP	Multiple VLAN Registration Protocol
OUI	organizationally unique Identifier
PCP	Priority Code Point
ISIS-PCR	Intermediate System to Intermediate System with Path Control and Reservation extensions
PICS	Protocol Implementation Conformance Statement
PSFP	Per-Stream Filtering and Policing
QoS	Quality of Service
R-TAG	Redundancy tag
RA	resource allocation
RAP	Resource Allocation Protocol
RESI	RAP Endpoint Service Interface
RTID	Resource Allocation Class Template Identifier
SRP	Stream Reservation Protocol
TCP	Transmission Control Protocol

[7] The abbreviations listed include those defined in standards referenced by this standard and used in this standard.

[8] See https://standards.ieee.org/develop/regauth/tut/eui.pdf.

TLV	Type, Length, Value
TSpec	traffic specification
TSN	Time-Sensitive Networking
UNI	User Network Interface
VID	VLAN Identifier
VLAN	Virtual Local Area Network
YANG	Yet Another Next Generation [9]

[9] YANG is best viewed as a name, not an acronym.

5. Conformance

This clause specifies mandatory and optional capabilities provided by conformant implementations of this standard.

5.1 Requirements terminology

For consistency with existing IEEE and IEEE 802.1™ standards, requirements are expressed using the following terminology: [10]

a) ***Shall*** is used for mandatory requirements.
b) ***May*** is used to describe implementation or administrative choices ("may" means "is permitted to," and hence, "may" and "may not" mean precisely the same thing).
c) ***Should*** is used for recommended choices (the behaviors described by "should" and "should not" are both permissible but not equally desirable choices).

Protocol Implementation Conformance Statements (PICS) reflect the occurrences of the words "shall," "may," and "should" within the standard.

The standard avoids needless repetition and apparent duplication of its formal requirements by using ***is***, ***is not***, ***are***, and ***are not*** for definitions and the logical consequences of conformant behavior. Behavior that is permitted but is neither always required nor directly controlled by an implementer or administrator, or whose conformance requirement is detailed elsewhere, is described by ***can***. Behavior that never occurs in a conformant implementation or system of conformant implementations is described by ***cannot***. The word ***allow*** is used as a replacement for the phrase "Support the ability for," and the word ***capability*** means "can be configured to."

5.2 Protocol Conformance Statement (PCS)

A claim of conformance to this standard attests to the implementation of a system or system component specified in this and referenced standards.

The supplier of an implementation that is claimed to conform to this standard shall provide the information necessary to identify both the supplier and the implementation, and shall complete a copy of the relevant PCS proforma provided in this standard together with the Protocol Implementation Conformance Statements (PICS) for the referenced standards, as identified in the PCS.

5.3 Conformant systems, components, and functionality

This standard specifies conformance requirements for the following component:

a) RAP instance (5.4)

and for the following systems that include instances of the above component:

b) RAP Bridge (5.5)
c) FRER-capable RAP Bridge (5.6)
d) RAP end station (5.7)
e) FRER-capable RAP Talker (5.8)
f) FRER-capable RAP Listener (5.9)

[10] Originally derived from ISO/IEC style requirements, and consistent with the terminology specified in the ISO/IEC Directives Part 2:2021, Clause 7 (http://www.iec.ch/members_experts/refdocs).

5.4 RAP instance requirements

A RAP Relay instance implementation conformant to this standard shall:

a) Support the RAP MAD ingress state machine as specified in 6.6.3.
b) Support the RAP MAD egress state machine as specified in 6.6.6
c) Support the RAP instance event state machine as specified in 6.8.1.
d) Support the Application Information TLV as specified in 6.6.8.3.
e) Support the RAP attributes and their TLV encodings as specified in 6.4.
f) Support the RAP management as specified in Clause 7.

5.5 RAP Bridge requirements

A RAP Bridge implementation conformant to this standard shall:

a) Include a single VLAN Bridge component conformant to [Q]:5.4.
b) Conform to the required and optional behaviors for a relay system ([CS]:5).
c) Include a single RAP instance conformant to the requirements of 5.4.
d) Support the RAP MAP state machine as specified in 6.7.4.

5.6 FRER-capable RAP Bridge requirements

A FRER-capable RAP Bridge implementation conformant to this standard shall:

a) Conform to the required and optional behaviors for a FRER C-component ([CB]:5.15).
b) Support Active Destination MAC and VLAN Stream identification function ([CB]:6.6).

5.7 RAP end station requirements

A RAP end station conformant to this standard shall:

a) Conform to the required behaviors for a end system ([CS]:5).
b) Support the RAP Endpoint state machine as specified in 6.6.10
c) Support the operation of the Applicant state machine ([CS]:8.3) and the Registrar state machine ([CS]:8.4).
d) Include a single RAP instance conformant to the requirements of 5.4.

5.8 FRER-capable RAP Talker requirements

A FRER-capable RAP Talker implementation conformant to this standard, shall:

a) Conform to the required, recommended and optional behaviors for a Talker end system ([CB]:5.6, [CB]:5.7, [CB]:5.8).

5.9 FRER-capable RAP Listener requirements

A FRER-capable RAP Listener implementation conformant to this standard shall:

a) Conform to the required, recommended and optional behaviors for a Listener end system ([CB]:5.9, [CB]:5.10, [CB]:5.11).

6. Resource Allocation Protocol (RAP)

6.1 RAP overview

RAP defines an application of the IEEE 802.1CS Link-local Registration Protocol (LRP) that provides resource allocation in bridged networks for the dynamic creation and maintenance of data streams requiring deterministic QoS - specifically bounded latency and zero congestion loss - along with high availability and reliability.

As an LRP application, RAP utilizes the service interface provided by LRP to manage connections and handle attribute declarations and registrations between neighboring devices. RAP is responsible for processing and propagation of attributes across the network. The definitions of LRP protocol elements required for the operation of RAP are presented in 6.6.8.

Resource allocation relies on each Bridge's capability to support one or more RA classes. Each class is associated with a different priority used by the streams as the PCP value ([Q]:6.9.3) and provides a guaranteed level of QoS to each reserved stream. The definitions of RA class parameters and the RA class domain concept are presented in 6.3.2.

As a signaling protocol, RAP performs three distince signaling processes, each utilizing a dedicated RAP attribute and serving a different purpose:

— RA Advertisement (6.3.5)
— Talker Announcement (6.3.6)
— Listener Attachment (6.3.7)

The definitions of these three RAP attributes and their encoding in the TLV format are provided in 6.4.

To facilitate decentralized latency and resource computation, RAP adopts "per-hop per-class guarantees" as the fundamental principle of resource allocation. This approach enables deterministic end-to-end QoS examination to be decomposed into a hop-by-hop verification of Bridge-local constraints, as defined in 6.3.8. A per-hop adjustable Traffic Specification (TSpec), along with the original specification generated by the Talker, serves as essential input for computing latency bounds and buffer requirements at each hop. This TSpec is carried in each Talker Announcement, as described in 6.3.9.

RAP supports Stream Rank and reservation importance concepts, as described in 6.3.10, in a manner similar to MSRP's support for stream importance ([Q]:35.2.4.1). This capability allows the network to remove existing reservations associated with less important streams when resources are insufficient for reserving more important streams.

RAP also provides dynamic resource allocation for streams requiring high availability through Frame Replication and Elimination for Reliability (FRER), as specified by IEEE Std 802.1CB and described in 6.3.11. This is achieved using a specific signaling mechanism to propagate attributes across redundant trees, such as Maximally Redundant Trees (MRTs) established using ISIS-PCR ([Q]:45). Upon successful reservation, the required FRER functions are automatically configured at designated locations, in addition to resource allocation at each participating station.

RAP replaces the MAD, specified in [Q]:10.2 with an application specific RAP MAD component (6.6) and extends the MAP, specified in [Q]:10.3, with RAP-specific MAP extensions (6.7).

RAP enables MSRP devices to connect to a RAP domain for backward compatibility within a specific MSRP class.

6.2 Conventions

This subclause defines the conventions used in the specification of RAP.

6.2.1 State machine diagrams

The state machine diagrams for RAP MAD and RAP MAP use the conventions defined in [Q]:E.

6.3 Principles of RAP operation

6.3.1 RAP architecture

RAP is a signaling protocol that provides end stations with the ability to reserve network resources that will guarantee the transmission and reception of data streams across a network with the requested QoS. These end stations are referred to as Talkers (devices that produce data streams) and Listeners (devices that consume data streams).

A RAP Port instance is an LRP application that implements the RAP functionality. It contains the RAP MAD component (6.6), which provides a RAP Endpoint Service interface (6.5.2) for the RAP application and communicates with the underlying LRP via the LRP-DS Service Interface (LSI, [CS]:10).

The required extensions to the MRP attribute Propagation component (MAP, [Q]:10.3) are specified in 6.7.

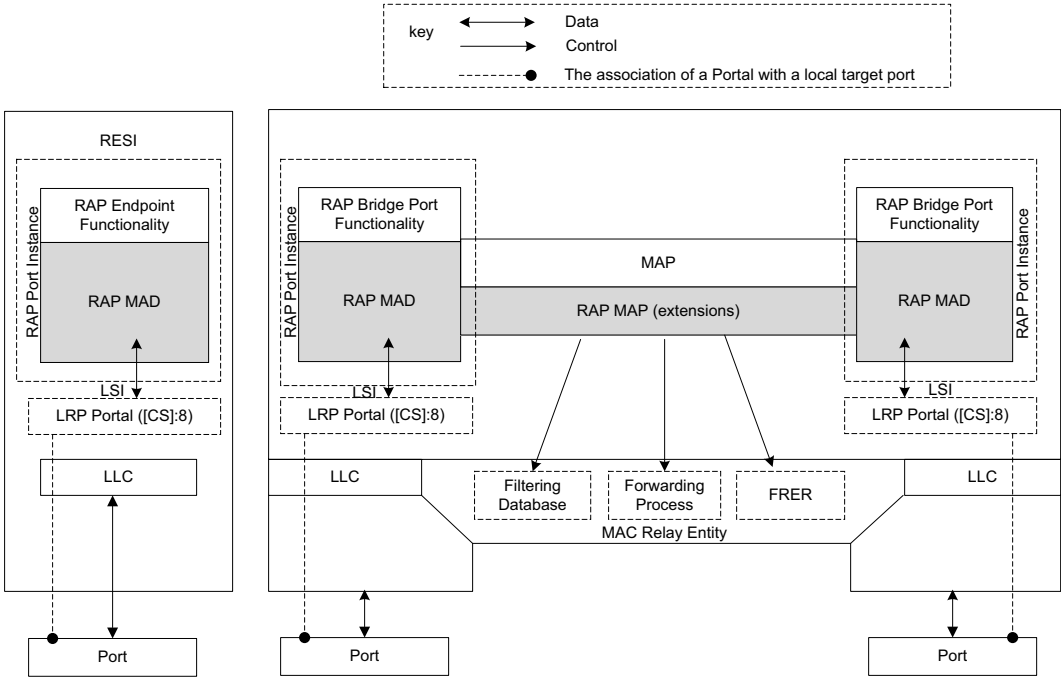

Figure 6-1—Operation of RAP

6.3.2 RA class

An RA class represents a traffic class or a set of traffic classes in an end station or Bridge that uses a given transmission selection algorithm, in conjunction with other mechanisms (e.g., PSFP specified in [Q]:8.6.5.1,

the traffic scheduling mechanisms specified in [Q]:8.6.8.4) if needed, and a resource reservation method, to provide a bounded latency and zero congestion loss for streams.

NOTE 1—It is also possible to map multiple RA classes to on traffic class. This is especially useful for bridges with limited amount of queues.

NOTE 2—An example of using more than one traffic class for a single RA class is an implementation of the cyclic queuing and forwarding shaper (CQF) using the approach as described in [Q]:T.2. In that case, two transmission queues operate in an coordinated manner and offer the same bounded latency to frames transmitted from either of the two queues.

6.3.2.1 RA class ID

The RA class ID is an unsigned integer in the range of 0 through 255 that identifies an RA class supported by an end station or Bridge.

NOTE—RA class ID is a numeric representation of the RA classes supported by a station. Among a set of RA classes on a given Bridge Port, one RA class that is assigned a numerically higher RA class ID than another RA class, is not necessarily mapped to a numerically higher traffic class than that RA class.

6.3.2.2 RA class priority

Each RA class supported by an end station or Bridge is associated with a unique priority value as defined in [Q]:6.9.3, termed RA class priority. The RA class priority indicates the received priority of the frames which are to be mapped to the traffic class(es).

NOTE—Mapping of RA class priorities to traffic classes is a matter of port local configuration and can be managed using the Traffic Class Table ([Q]:12.6.3). This standard does not specify default priority to traffic class mapping for a system that supports RA classes.

Since there are eight values available for the RA class priority, a station can support up to seven RA classes, in order to ensure that there is at least one traffic class that can support non-stream traffic that is not subject to resource allocation, such as “best effort” traffic.

6.3.2.3 RA class template and RTID

An RA class template describes a specific method for making up an RA class, including the following:

— The shaping or scheduling mechanisms employed.
— The algorithms for computing latency bounds and resources.
— The encoding of the data in the RaClassTemplateDefinedData field (6.4.2.2.5).

An RA class template is identified by an RA Class Template Identifier (RTID), which encodes a 3-octet OUI or CID value identifying the organization that defines that template, followed by a 1-octet index allocated by that organization. The RA class templates currently defined by IEEE 802.1 are listed in Table D-1.

6.3.2.4 RA class domains and priority regeneration

An RA class domain is a connected subset of end stations and Bridges that support a common RA class (i.e. the same RA class ID, 6.3.2.1) and associate the same priority value with that RA class (i.e. the same RA class priority, 6.3.2.2). Within an RA class domain, any traffic associated with the RA class priority of that domain is treated as stream traffic and subject to resource allocation by RAP. The concept of RA class domains also limits the scope of resource allocation in the sense that a stream can only be established for transmission within the domain of a given RA class.

An RA class domain boundary port is a Port of a Bridge that is part of a given RA class domain and is connected via a LAN to a neighboring station that is not part of that RA class domain. On such a Port, any traffic received with a priority identical to the RA class priority of that RA class domain is subject to priority

regeneration ([Q]:6.9.4) according to the administrative values in RA Class Domain Boundary Priority Regeneration Table (7.7) such that the traffic is forwarded by the Bridge using another priority not associated with any RA class.

The detection of RA class domains as well as determining the location of RA class domain boundary ports is controlled by RA Advertisement (6.3.5). Figure 6-2 gives an example of multiple RA Class domains established in a single network, where three domains for RA class ID 1 and three for RA class ID 2 are separately depicted on the left-hand and the right-hand side of the figure, respectively.

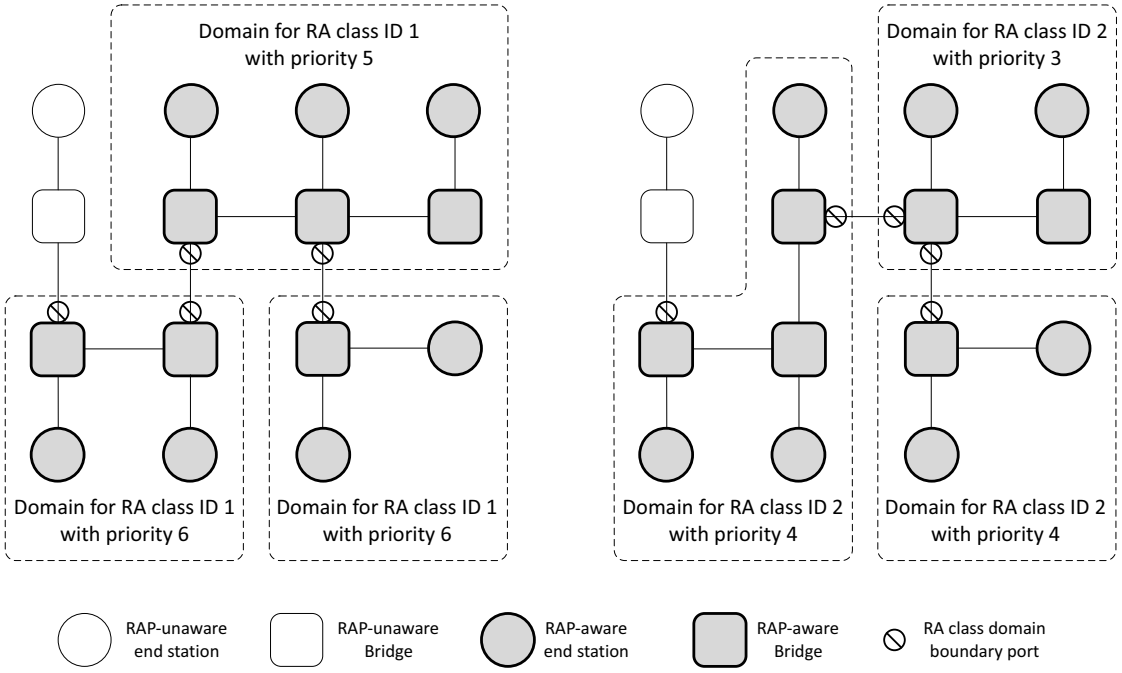

Figure 6-2—Examples of RA class domains and domain boundary ports

6.3.3 Startup state machine

.

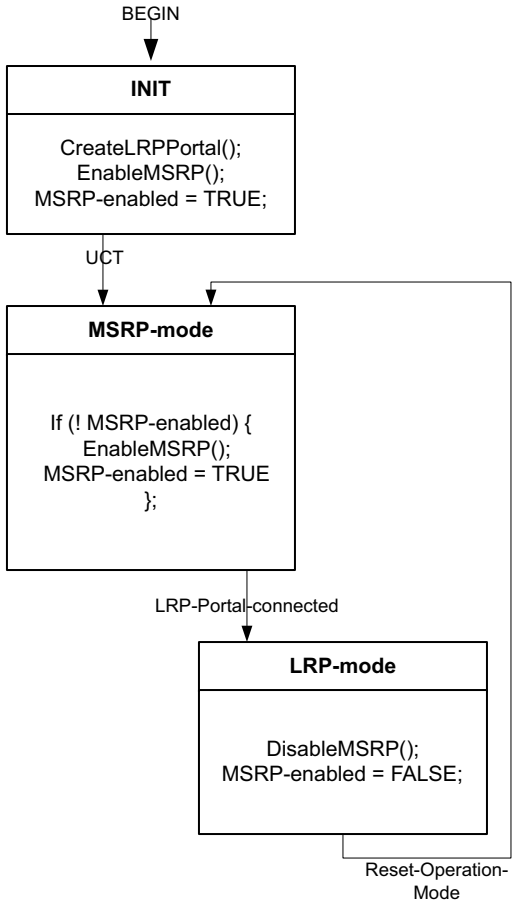

Figure 6-3—Startup state machine

The LRP-Portal-connected event is triggered by the reception of a Portal status indication(connected) (CS: 10.2.6).

The Reset-Operation-Mode event is generated by the RAP Port Instance to reenable MSRP-mode.

Editor's Note: There are other possibilities to specify the reset of the operation mode. One possibility is to specify explicitly the use of a managed object to reset the operation mode. A further possibility is to specify that a LRP Portal status indication() resets the operation mode.

6.3.4 RAP Startup state machine procedures

LRP is expected to be running at startup.

6.3.4.1 CreateLRPPortal()

This procedure adds a RAP LRP-Portal ([CS]:8) by issuing a Local Target Port request ([CS]:10.2.2) for this Bridge Port.

6.3.4.2 EnableMSRP()

This procedure enables MSRP at this Port and declares MSRP attributes ([Q]:35.2.4).

6.3.4.3 DisableMSRP()

This procedure withdraws any MSRP attributes ([Q]:35.2.4) and disables MRP at this Port.

6.3.5 RA Advertisement

RA Advertisement refers to declarations of the RA attribute (6.4.2) by a station to each neighboring station. The purposes of RA Advertisement are as follows:

— Advertisement of RA class domain configuration for detection of RA class domains and determination of the location of RA class domain boundary ports, as described in 6.3.2.4.

— End station learning of the RA class (6.3.2) and Redundancy Context (6.3.11.2) configurations in the network. To allow adaptive participation in resource allocation, an end station can support the ability to adjust its local parameters according to the information received from its neighboring Bridge and then declare its own RA attribute with the adjusted parameters.

— Advertisement of RA class template configuration. In the process of reserving a stream in a given RA class, the RA class template used by that RA class and corresponding parameter configuration in an upstream station is necessary information used by each downstream station in performing actions such as calculation of the worst-case latency of streams transmitted in that RA class from that upstream station.

The following terminology relating to RA Advertisement is used:

a) **RA declaration**: A declaration of the RA attribute on an end-station or Bridge Port.

b) **RA registration**: A registration of the RA attribute on an end-station or Bridge Port.

c) **RA deregistration**: The removal of an RA registration on an end-station or Bridge Port.

An RA registration received from a neighboring station on a Bridge Port is not propagated by the Bridge to declare that registered RA attribute to another neighboring on any of other Ports.

6.3.6 Talker Announcement

The Talker Announcement refers to the process of signaling the Talker Announce attribute (6.4.3) initially declared by the Talker of a stream across a Bridged network to potential Listeners. Each Talker Announcement is identified by a StreamId (6.4.3.1) and VID (6.4.3.5.3) as contained in the Talker Announce attribute.

The purposes of the Talker Announcement are as follows:

— Indication of a Talker’s intention of supplying a stream.

— Advertisement of the stream’s characteristics. The Talker Announce attribute contains the information needed by the network to identify the stream and determine the required resources on each Bridge along the stream path(s).

— Gathering of QoS information from the Bridges along each path such as the accumulated latency and reporting of gathered information to Listeners.

— Signaling of status information along each potential path about whether a reservation can be made, and if not the information about the cause and the location of the failure.

The following terminology relating to the Talker Announcement is used:

a) **Talker Announce declaration:** A declaration of the Talker Announce attribute on a Port.
b) **Talker Announce registration:** A registration of the Talker Announce attribute on a Port.
c) **Talker Announce deregistration:** The removal of a Talker Announce registration on a Port.
d) **Talker Announce propagation:** Propagation of a Talker Announce registration on an ingress Port of a Bridge to one or more other egress Ports of the Bridge, resulting in a Talker Announce declaration on each such egress Port.
e) **Talker Announce merge:** Merge of multiple Talker Announce registrations being propagated to a Bridge Port to result in a joint Talker Announce declaration on that Port.
f) **Originating Talker Announce registration:** A Talker Announce registration from which a given Talker Announce declaration results from is termed an originating Talker Announce registration of that Talker Announce declaration. A Talker Announce declaration can have more than one originating Talker Announce registration in the case of Talker Announce merge.

6.3.6.1 Talker Announce merge

Talker Announce merge can only occur in the Multi-Context Talker Announcement in resource allocation for seamless redundancy (6.3.11) and is associated with the FRER operation stream merging in FRER-capable Bridges or Listeners, as described in 6.3.11.5.

Talker Announce merge is performed in a FRER-capable Bridge or Listener among a set of Multi-Context Talker Announce registrations that are propagated to the same Port and contain the same StreamId (6.4.3.1) value, to result in a joint Multi-Context Talker Announce declaration on that Port for that StreamId and containing a merged set of VLAN Contexts comprising each one used in the propagation of those Talker Announce registrations.

6.3.6.2 Talker Announce status

The Talker Announce status of a Talker Announce declaration made on a given Port is associated with an ordinary stream in the case of the Single-Context Talker Announcement, or a Compound or a Member Stream in the case of the Multi-Context Talker Announcement, to be transmitted on that Port, indicating whether the stream could be reserved from the Talker to that Port along a single path or one or more paths, respectively, in the VLAN Context(s) indicted in the Talker Announce declaration, as follows:

a) **Announce Success:** The stream can be reserved on the Port, as it has not encounter any problems on a single path from the Talker to that Port, if the stream is a ordinary stream, or on at least one path from the Talker to that Port, if the stream is a Compound Stream or Member Stream. The Announce Success status is represented by the absence of a Failure Information sub-TLV (6.4.3.10) in the Value field of the Talker Announce attribute TLV (6.4.3).
b) **Announce Fail:** The stream can not be reserved on the Port, as it has encounter problems on each path from the Talker to that Port. The Announce Fail status is represented by the presence of a Failure Information sub-TLV (6.4.3.10) in the Value field Talker Announce attribute TLV (6.4.3).

6.3.6.3 Talker Announce propagation

A set of rules use in Talker Announce propagation are defined in the following subclauses. These rules refer to specific Forwarding Process functions ([Q]:8.6) in VLAN Bridges, which can be configured by use of relevant protocols or management entities specified in [Q].

6.3.7 Listener Attachment

Listener Attachment refers to the process of signaling the Listener Attach attribute (6.4.4) initially declared by one or more Listeners desiring to receive a stream across a Bridge network to the stream’s Talker. Each

Listener Attachment is identified by a StreamId (6.4.4.1) and VID (6.4.4.2) as contained in the Listener Attach attribute.

The purposes of Listener Attachment are as follows:

— Indication of the intention of one or more Listeners of receiving a stream.
— Triggering of resource allocations (or deallocation) on each Bridge along potential stream path(s).
— Signaling of reservation status hop-by-hop in an upstream direction from each participating Listener back to the Talker.

The following terminology relating to Listener Attachment is used:

a) **Listener Attach declaration**: A declaration of the Listener Attach attribute on a Port.
b) **Listener Attach registration**: A registration of the Listener Attach attribute on a Port.
c) **Listener Attach deregistration**: The removal of a Listener Attach registration on a Port.
d) **Listener Attach propagation**: Propagation of a Listener Attach registration from a Bridge Port to one or more other Bridge Ports.
e) **Listener Attach merge**: Merge of multiple Listener Attach registrations propagated to a Bridge Port to result in a joint Listener Attach declaration on that Port.
f) **Associated Talker Announce declaration**: A Talker Announce declaration with which a given Listener Attach registration is associated is termed an associated Talker Announce declaration of that Listener Attach registration.
g) **Originating Listener Attach registration**: A Listener Attach registration from which a given Listener Attach declaration results is termed an originating Listener Attach registration of that Listener Attach declaration. A Listener Attach declaration can have more than one originating Listener Attach registration in the case of Listener Attach merge.
h) **Associated Talker Announce registration**: A Talker Announce registration with which a given Listener Attach declaration is associated is termed the associated Talker Announce registration of that Listener Attach declaration.

6.3.7.1 Association between Talker Announcement and Listener Attachment

A Listener Attachment is associated with a Talker Announcement that carries the same StreamId as that of the Listener Attachment. Depending on the type of the associated Taker Announcement, there are two types of the Listener Attachment, as follows:

a) **Single-Context Listener Attachment:** A Listener Attachment that is associated with a Single-Context Talker Announcement [item a) in 6.7.2.1].
b) **Multi-Context Listener Attachment:** A Listener Attachment that is associated with a Multi-Context Talker Announcement [item b) in 6.7.2.1].

A Single-Context Listener Attach registration is said to be associated with a Single-Context Talker Announce declaration, if all of the following conditions are met:

1) The Listener Attach registration is on the same Port as the Talker Announce declaration.
2) The StreamId (6.4.4.1) of the Listener Attach registration is identical to the StreamId (6.4.3.1) of the Talker Announce declaration.
3) The VID (6.4.4.2) of the Listener Attach registration is identical to the VID (6.4.3.5.3) of the Talker Announce declaration.

A Multi-Context Listener Attach registration is said to be associated with a Multi-Context Talker Announce declaration, if all of the following conditions are met:

4) The Listener Attach registration is on the same Port as the Talker Announce declaration.

5) The StreamId (6.4.4.1) of the Listener Attach registration is identical to the StreamId (6.4.3.1) of the Talker Announce declaration.
6) The set of VLAN Context(s) (6.4.4.4) indicated by the Listener Attach registration is identical to the set of VLAN Context(s) indicated by the Talker Announce declaration (6.4.3.9.2).

A Listener Attach declaration is said to be associated with a Talker Announce registration, if all of the following conditions are met:

7) The Listener Attach declaration is on the same Port as the Talker Announce registration.
8) The StreamId (6.4.4.1) of the Listener Attach declaration is identical to the StreamId (6.4.3.1) of the Talker Announce registration.
9) The Listener Attach declaration wholly or partially results from a Listener Attach registration whose associated Talker Announce declaration originates wholly or partially from the Talker Announce registration.

A Listener Attach registration with an associated Talker Announce declaration on a Bridge Port indicates a reservation request for transmission of a stream or a Compound or Member Stream through that Port and, upon successful resource allocation, is associated with a dedicated reservation in the Bridge Port. A reservation maintained in a Bridge Port is identified by <StreamId, VID> with the corresponding values contained in the associated Listener Attach registration.

6.3.7.2 Listener Attach split

A Listener Attach split occurs when a single Listener Attach is propagated in the reverse direction to more than one port from which associated Talker Announces originate. This condition can arise only on Bridges at which a Talker Announce merge operation has occurred (6.3.6.1). A Listener Attach split identifies a potential location for stream merging in FRER-capable RAP Bridges (6.3.11.5).

6.3.7.3 Listener Attach merge

Listener Attach merge is applied, when more than one Listener Attach registration whose associated Talker Announce declaration has a common originating Talker Announce registration is propagated to the same Port, to result in a joint Listener Attach declaration on that Port.

In a Single-Context or Multi-Context Listener Attachment for a stream, Listener Attach merge can occur when multiple Listener Attachment processes originating from different Listeners of that stream converge on a Bridge Port, indicating a potential place for multicast forwarding the stream. Additionally, in a Multi-Context Listener Attachment for a Compound Stream, Listener Attach merge can also occur when multiple Listener Attachment processes running in different VLAN Contexts converge on a Bridge or Talker end station Port, indicating a potential place for applying stream splitting (6.3.11.4) to the stream.

6.3.7.4 Listener Attach status

The Listener Attach status of a Listener Attach declaration on a Port is associated with one or more paths, depending on the VLAN Context(s) in which the Listener Attach declaration is made, from the Port to each Listener whose participation in the Listener Attachment has been perceived on that Port, and indicates one of the following:

a) **Attach Ready:** In the case of the Single-Context Listener Attachment, indicating a reservation has been successfully made on each path to each Listener. In the case of the Multi-Context Listener Attachment, indicating a reservation has been successfully made on at least one path to one Listener, while the reservation status of each path is indicated by the Path status (6.3.7.6).
b) **Attach Fail:** A reservation has failed on each path to each Listener;

c) **Attach Partial Fail:** A reservation has been successfully made on the path to at least one of these Listeners, but has failed on the path to other Listeners. This status is only applicable to the Single-Context Listener Attachment.

NOTE—The Attach Partial Fail status is used only in the Single Context Listener Attachment for the same purpose for which the Listener Ready Failed status is used in MSRP as defined in [Q]:35.1.2.2. In the case of the Multi-Context Listener Attachment, there is no distinction made in the Listener Attach status between a full success and a partial success, while the latter case could arise, for example, when reservation has failed on one among all paths to a single Listener, or when reservation on all the paths has succeeded for some Listeners but failed for the others.

6.3.7.5 Listener Attach propagation

A Listener Attachment follows the path(s) traversed by its associated Talker Announcement in reversed direction.

A Listener Attach registration on a Bridge Port with an associated Talker Announce declaration is propagated to each other Port that contains an originating Talker Announce registration of an associated Talker Announce declaration. A Listener Attach registration on a Bridge Port without an associated Talker Announce declaration is not propagated by the Bridge.

6.3.7.6 Path status

A Multi-Context Listener Attachment also signals the Path status on a per VLAN Context basis. The Path status of a Listener Attach declaration on a Port for a given VLAN Context is associated with a single path within that VLAN Context from the Port to each Listener whose participation in the Listener Attachment has been perceived on that Port, and indicates one of the following:

a) **Faultless Path**: Resource allocation has succeeded without encountering any problems on the path.
b) **Faulty Path**: Resource allocation has encountered one or more problems on the path and failed wholly or partially on the path.
c) **Unresponsive Path**: No Listener Attachment with respect to the path has been received.

NOTE—In the Multi-Context Listener Attachment, resource allocation can succeed partially on a path as a result of the fact that the path goes through one or more places of stream merging in the network, as described in 6.3.11.5.

6.3.8 Resource allocation constraints

Resource allocation is constrained by limited capacity of end stations and Bridges to ensure bounded latency and zero congestion loss for reserved streams. The subsequent subclauses (6.3.8.1 through 6.3.8.3) describe three constraints taken into account in resource allocation.

6.3.8.1 Bandwidth constraint

The bandwidth constraint of a Bridge is specified in the per-RA class, per-transmission Port maxBandwidth parameter, as defined in item c) of 6.7.5.8. One such parameter configured in a Bridge places an upper bound on the amount of bandwidth that can be reserved for use by an RA class on a transmission Port.

The bandwidth constraint also applies to end stations. As in many circumstances the bandwidth availability for streams in an end station is controlled by the applications, no explicit bandwidth constraint parameters are specified by this standard for end stations, and it is a matter of each end station’s local decision.

A stream reservation request is said to meet the bandwidth constraint of a Bridge or end station, if the total bandwidth needed to transmit that stream and all reserved streams in the same RA class and on the same Port would not exceed the associated upper bound. A Talker Announcement failed due to not meeting the bandwidth constraint is reported with the RAP Failure Code *BandwidthExceeded* defined in Table 6-7.

6.3.8.2 Resource constraint

The resource constraint is associated with the availability of configurable resources for streams in a Bridge or end station's local functions, such as filtering, buffering, shaping and FRER. A stream reservation request is said to meet the resource constraint of a station, if the station has sufficient resources available for all the functions required for handling that stream. A Talker Announcement failed due to not meeting the resource constraint is reported with the RAP Failure Code *ResourceExceeded* defined in Table 6-7.

A measure of resource constraint and its determination method for streams are usually dependent on a particular implementation. No explicit parameters are specified by this standard for resource constraint.

6.3.8.3 Latency constraint

The latency constraint for a Bridge is specified in the per-RA class, per reception/transmission Port pair maxHopLatency parameter, as defined in item d) of 6.7.5.9. Such a parameter configured in a Bridge is used by the Bridge in resource allocation as a guaranteed latency upper bound provided to each stream reserved in a given RA class when transmitted through the hop from a neighboring station on a given reception Port to a given transmission Port.

Depending on the shaper being used, an RA class can specify which reference points are used for latency (Figure 6-4). In case of fully-received to fully-received latency, the reference point is if a frame is fully received by a bridge. For shaper to shaper latency, the reference point is when the frame becomes eligible for transmission.

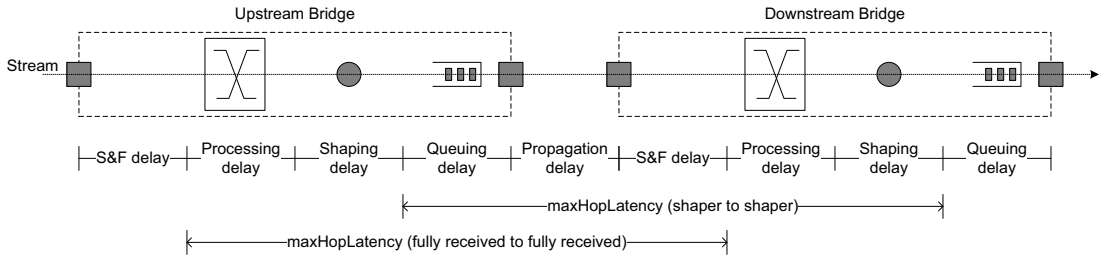

Figure 6-4—Latency reference points

A stream reservation request is said to meet the latency constraint of a Bridge or an end station (as a Listener), if allowing transmission of this stream would not cause any reserved stream to suffer from a latency exceeding the associated upper bound. A Talker Announcement failed due to not meeting the latency constraint is reported with the RAP Failure Code *LatencyExceeded* defined in Table 6-7.

The latency constraint configured on each hop along each path in the network, in conjunction with an accumulation of per-hop latency bounds in AccuMaxLatency (6.4.3.3), builds up the basis for bounded end-to-end latencies in resource allocation.

NOTE—The concept of per-hop latency bound requires the ability of each station participating in resource allocation to determine the worst-case maximum latency while checking the latency constraint. The method used for this purpose is specific to each RA class template (See Annex D).

6.3.9 Traffic specification usage rules and types

In resource allocation, the traffic specification (TSpec) carried in the Talker Announcement contains the information about the characteristics of the stream's data traffic.

6.3.9.1 Talker TSpec

A TSpec received via RESI (6.5.2) from higher layer entities in Talkers and specifying the amount of the stream data transmitted by the Talker into the network. The Talker TSpec remains unchanged during the propagation of the Talker Announcement across the network.

NOTE 1—Keeping a Talker TSpec unchanged during its propagation is to provide Bridges and Listener end stations with the original information about the stream traffic generated by Talkers. This information can be used, for example, by a specific intermediate Bridge in traffic shaping the stream to restore its original traffic characteristics.

There are also two types of TSpec supported by resource allocation, each using a different parameter set to describe the stream traffic characteristics:

a) **Talker Token Bucket TSpec**, a TSpec comprising the following set of parameters excluding any MAC framing and media-dependent overheads (e.g., preamble, 802.3 header, MAC header, VLAN tag, R-TAG, CRC, interframe gap):
   1) **MaxPayloadLength**: The maximum frame length of the traffic, in octets, associated with on any transmission Port.
   2) **MinPayloadLength**: The minimum frame length of the traffic, in octets, associated with on any transmission Port.
   3) **CommittedInformationRate**: the committed information rate, in bits per second.
   4) **CommittedBurstSize**: The committed burst size, in bits sent from the application.

b) **Interval-based TSpec**, a TSpec comprising the following set of parameters:
   1) **Interval**: The interval of time, in nanoseconds, taken as a fixed-length observation window to measure the maximum amount of the traffic.
   2) **MaximumFramePerInterval**: The maximum number of maximum sized frames can be observed from the traffic within one Interval [item 1) above].
   3) **MaximumFrameSize**: The maximum frame size of the traffic, in octets, excluding any MAC framing and media-dependent overheads (e.g., preamble, 802.3 header, MAC header, VLAN tag, R-TAG, CRC, interframe gap) associated with on any transmission Port.

NOTE 3—The Interval-based TSpec type is compatible with the TrafficSpecification defined in [Q]:46.2.3.5, except the TransmissionSelection parameter ([Q]:46.2.3.5.4).

6.3.9.2 Network TSpec

A Token Bucket TSpec that is generated when a Talker TSpec is received via RESI (6.6.2) from higher layer entities in Talkers (6.3.9.3).

Token Bucket TSpec, a TSpec comprising the following set of parameters including the media-dependent overhead ([Q]:12.4.2.2) :

1) **MaxTransmittedFrameLength**: The maximum frame length of the traffic, in octets,associated with a transmission Port for the traffic.
2) **MinTransmittedFrameLength**: The minimum frame length of the traffic, in octets, associated with a transmission Port for the traffic.
3) **CommittedInformationRate**: the committed information rate, in bits per second.
4) **CommittedBurstSize**: The committed burst size, in bits.

6.3.9.3 Translation of Talker TSpec to Network TSpec

When initiating a Talker Announcement at a Talker, higher layer entities can supply either a Talker Token Bucket TSpec or an Interval-based TSpec as the Talker TSpec. The Network TSpec field in the Talker Announce attribute always contains a Token Bucket TSpec derived from the supplied Talker TSpec. When an Interval-based TSpec is supplied as the Talker TSpec, it is translated by the Taker to a Token Bucket

TSpec as the Network TSpec as specified in Table 6-1. As the Talker Announcement propagates in the network, each Bridge on the path can adjust the Token Bucket TSpec values contained in the Network TSpec as necessary.

Table 6-1—Translation of an Interval-based TSpec to a Token Bucket TSpec

Interval-based TSpec	Token Bucket TSpec
MaximumFrameSize + ALL_OVERHEAD[a]	MaxTransmittedFrameLength
64 + MD_OVERHEAD[b]	MinTransmittedFrameLength
(MaximumFrameSize + ALL_OVERHEAD[a]) * MaximumFramePerInterval * 8 * 1e9 / Interval	CommittedInformationRate
(MaximumFrameSize + ALL_OVERHEAD[a]) * MaximumFramePerInterval * 8	CommittedBurstSize

[a] Including all MAC framing and media-dependent overheads (e.g., preamble, 802.3 header, MAC header, VLAN tag, R-TAG, CRC, interframe gap) associated with a transmission Port for the traffic.

[b] The media-dependent overhead [Q]:12.4.2.2 associated with a transmission Port for the traffic.

6.3.10 Stream Rank and reservation importance

RAP supports the concept of the Rank as specified in [Q]:46.2.3.2.1 and gives streams of Rank zero higher precedence than streams of Rank one in a manner that allows Bridges to make a reservation for a stream of Rank zero in case of lacking resources by removing existing reservations made for one or more streams of Rank one. Each reservation removed under such circumstances is termed “the reservation preempted by Rank zero” and uses the RAP Failure Code *ReservationPreempted* as defined in Table 6-7.

NOTE—RAP allows reservations of Rank one streams to be preempted only by reservations of Rank zero streams. Reservations of streams with the same Rank, regardless Rank zero or one, are handled on a first-come first-served basis.

The order used by a Bridge in the selection of a reservation, in preference to others, to be preempted by Rank zero is determined based on the reservation importance, i.e., a least important reservation is to be removed first. Among a set of reservations on a Bridge Port, each identified by <StreamId, VID> and with a reservationAge value [item d) in 6.7.5.3], one reservation is considered more important than another, according to the following rules in the shown order:

a) A Reservation with a numerically larger reservationAge value is more important.
b) If reservationAge is the same, the one with a numerically smaller StreamId is more important.
c) If StreamId is the same, the one with a numerically smaller VID is more important.

6.3.11 Resource allocation for seamless redundancy

RAP supports resource allocation for transmitting streams via redundant paths using the seamless redundancy techniques specified by IEEE Std 802.1CB. The subsequent subclauses describe additional capabilities and operations required by resource allocation for seamless redundancy, while the definitive specification is contained in 6.4, 6.5 and 6.7. For illustrative purposes, some resource allocation examples for seamless redundancy are provided in Annex B.

6.3.11.1 FRER-capable stations

Resource allocation for seamless redundancy operates in a network where a set of FRER-capable stations, either or both FRER-capable end stations and FRER-capable Bridges, are placed at specific locations to perform FRER operations on Compound and Member Streams with corresponding FRER functions, as shown in Table 6-2.

Table 6-2—FRER functions and operations in FRER-capable stations

<table>
<tr><th colspan="2" rowspan="2"></th><th colspan="3">FRER-capable stations</th></tr>
<tr><th>FRER-capable
RAP Talker</th><th>FRER-capable
RAP Bridge</th><th>FRER-capable
RAP Listener</th></tr>
<tr><td rowspan="5">FRER operations</td><td>redundancy tagging (6.3.11.3)</td><td colspan="2">sequence generation function ([CB]:7.4.1),
Stream encode/decode function ([CB]:7.6),
Redundancy tag ([CB]:7.8)</td><td rowspan="3">(none)</td></tr>
<tr><td rowspan="2">stream splitting (6.3.11.4)</td><td>Stream splitting function ([CB]:7.7)</td><td>(Multicast) [a]</td></tr>
<tr><td colspan="2">Active Destination MAC and VLAN Stream identification function ([CB]:6.6) [b]</td></tr>
<tr><td>stream merging (6.3.11.5)</td><td rowspan="2">(none)</td><td colspan="2">Active Destination MAC and VLAN Stream identification function ([CB]:6.6) [b],
sequence recovery function ([CB]:7.5),</td></tr>
<tr><td>redundancy untagging (6.3.11.3)</td><td colspan="2">Stream encode/decode function ([CB]:7.6),
Redundancy tag ([CB]:7.8)</td></tr>
</table>

[a] The stream splitting operation in a Bridge relies on the multicast ability of the Forwarding Process to replicate streams, without the need to use Stream splitting function.

[b] An end station can, but is not required to explicitly use this function; it is an implementation choice.

RAP makes use of Multi-Context Talker Announcement and Multi-Context Listener Attachment in resource allocation for seamless redundancy. The potential locations in the network (including end stations) where each stream requesting seamless redundancy is subject to FRER operations along its paths are dependent on the placement of FRER-capable stations and the VLAN topologies configured for use with seamless redundancy, and are automatically determined on a per-stream basis during the stream's Multi-Context Talker Announcement. Resource allocation, including configuration of the FRER functions needed for each stream, is executed during the stream's Multi-Context Listener Attachment.

6.3.11.2 Redundancy Context

Resource allocation for seamless redundancy operates in a network in which one or more instances of redundant trees are established and maintained by other protocols such as ISIS-PCR ([Q]:45). Each Multi-Context Talker Announcement for a Compound Stream operates within a particular instance of redundant trees. The group of VLAN topologies associated with an instance of redundant trees comprising two or more trees forms the set of correlated VLAN Contexts, termed "the Redundancy Context". A Redundancy Context is identified by the numerically smallest VLAN Context ID in the set, termed "the Redundancy Context ID". Multiple Redundancy Contexts, each with distinct set of VLAN Context IDs, can exist in a network due to the existence of multiple instances of redundant trees.

NOTE—For example, there can be one instance of redundant trees for each Bridge located at the edge of the network and rooted at that Bridge, with the intention that each Compound Stream using the redundant trees rooted at a Bridge where it enters the network be established for transmission along the multiple paths that are disjoint from each other.

The per-Bridge parameter redundancyContext (6.7.5.10), configurable as a managed object defined in Table 7-8 and containing the configuration of each existing Redundancy Context in the network, provides the information needed by operations such as Talker Announce merge as described in 6.3.6.1. Proper operation of resource allocation for seamless redundancy requires a consistent Redundancy Context configuration among all the Bridges involved in resource allocation for seamless redundancy. An inconsistent Redundancy Context configuration, e.g. resulting from misconfiguration, can cause incorrect operation. To facilitate

inconsistency detection, each Bridge advertises its redundancyContext value in the RA Advertisement (6.3.5).

A Redundancy Context is considered inconsistent if the adjacent Bridge does not announce exactly the same Redundancy Context as the local Redundancy Context.

6.3.11.3 Redundancy tagging and redundancy untagging

Redundancy tagging refers to a FRER operation in which a FRER-capable RAP Talker or a FRER-capable RAP Bridge uses the particular FRER functions, as shown in Table 6-2, to transform an ordinary stream into a Compound Stream by inserting a Redundancy tag (R-TAG) into the data frames of the stream. Redundancy untagging refers to a FRER operation in which a FRER-capable Bridge or a FRER-capable Listener uses the particular FRER function, as shown in Table 6-2, to transform a Compound Stream back into an ordinary stream by removing the R-TAG carried in the data frames of the Compound Stream.

The potential locations in the network (including end stations) where a stream requesting seamless redundancy is subject to redundancy tagging and redundancy untagging are determined automatically during the stream's Multi-Context Talker Announcement in accordance with the following rules:

a) Redundancy tagging is to be performed by the stream's Talker, if that Talker is FRER-capable, and by a FRER-capable Bridge where the stream enters the network otherwise. The Boolean flag RTagStatus (6.4.3.9.1) contained in the Talker Announcement is then set by the station responsible for redundancy tagging. If neither of these conditions for redundancy tagging is fulfilled, the Talker Announcement is failed by the Bridge where the stream enters the network.

b) Redundancy untagging is to be performed by a FRER-capable Bridge on a transmission Port where all of the Member Streams previously split from the original stream are merged back into a single Compound Stream. In case that no such Bridge Port for stream merging (6.3.11.5) exists on the path towards a given Listener, that Listener is responsible for redundancy untagging, if it is FRER-capable, and fails the Talker Announcement otherwise. The Boolean flag RTagStatus (6.4.3.9.1) contained in the Talker Announcement is cleared by the station responsible for redundancy untagging.

The RAP Failure Code *RTagFailed* defined in Table 6-7 is reported in case of failure to meet the conditions of redundancy tagging or untagging as described above.

6.3.11.4 Stream splitting

Stream splitting refers to a FRER operation in which a FRER-capable Talker or a FRER-capable Bridge uses the particular FRER functions, as shown in Table 6-2, to split a Compound Stream or a Member Stream into multiple Member Streams.

The potential locations in a network (including end stations) where a stream requesting seamless redundancy is subject to stream splitting are determined automatically during the stream's Multi-Context Talker Announcement in accordance with the following rules:

a) If the stream's Talker is FRER-capable, it is responsible for stream splitting by making several separate Talker Announce declarations, each with a different VLAN Context from the Redundancy Context chosen by the stream. Otherwise, it makes a Talker Announce declaration that include all the VLAN Contexts in the Redundancy Context, which indicates that the stream is expected to be transformed into a Compound Stream (i.e., redundancy tagging) and then split into Member Streams by a FRER-capable Bridge proxying for this Talker.

b) Each FRER-capable Bridge participated in the stream's Talker Announcement is responsible for applying stream splitting to a stream indicated by a Talker Announce registration in the Bridge, if following condition is met:

The Talker Announce registration contains more than one VLAN Context and is propagated by the Bridge to at least two other Ports. And at least one of these Ports is not in all of the VLAN Contexts as contained in the Talker Announce registration. This also indicates the fact that the VLAN topologies in which the Talker Announce registration is propagated are disjoint somehow on these Ports.

NOTE—Stream splitting is distinguished from the normal multicast, as the former transforms replicated streams into separate Member Streams with different VIDs. If a FRER-capable Bridge propagates a Multi-Context Talker Announce registration to several Ports each with the same set of VLAN Contexts, it will apply just multicast instead of stream splitting to that Compound Stream.

Meeting the conditions described in item b) above requires a proper placement of FRER-capable stations and VLAN configuration in the network. To prevent incorrect stream splitting operation, a Multi-Context Talker Announcement is failed by a Bridge with the RAP Failure Code *StreamSplitFailed* defined in Table 6-7, in either of the following three circumstances:

c) A Talker Announce registration in a Bridge that is not FRER-capable meets b).

d) If the frames cannot be forwarded to the port of the given Talker Announce declaration element.

e) If the frames cannot be sent with the given VLAN ID configuration at the port of the given Talker Announce declaration element.

6.3.11.5 Stream merging

Stream merging refers to a FRER operation in which a FRER-capable RAP Bridge or a FRER-capable RAP Listener performs the particular FRER function, as shown in Table 6-2, to combine more than one Member Stream into a Compound Stream.

The potential locations in a network (including end stations) where a stream requesting seamless redundancy is subject to stream merging are determined automatically during the stream's Multi-Context Talker Announcement in accordance with the following rules:

a) Each FRER-capable RAP Bridge participated in propagation of the stream's Talker Announcement is responsible for stream merging, if the Bridge propagates more than one Talker Announce registration of the stream on different Ports to one or more common destination Ports. Such a common destination Port, also termed stream merging Port, is in fact located at a converging point of all or some of the VLAN topologies in the Redundancy Context used by the stream.

b) A FRER-capable RAP Listener of the stream is responsible for stream merging, if it receives more than one Talker Announce registration of the stream, each containing only a single VLAN Context or partial ones of the Redundancy Context used by the stream.

Note that a Bridge Port or a Listener where all of the Member Streams split from the original stream are subject to stream merging into a Compound Stream is also responsible for applying redundancy untagging (6.3.11.3) to the Compound Stream following the stream merging operation.

6.3.12 Relationship of RAP to MSRP

RAP has conceptual and functional similarities to MSRP. Table 6-3 summarizes the similarities and differences between RAP and MSRP.

Table 6-3—RAP/MSRP differences

	MSRP ([Q]:35)	RAP
Attribute exchange framework	MRP ([Q]:10)	LRP (IEEE Std 802.1CS)
Operational modes	peer-to-peer (MSRPv0) MRP External Control (MSRPv1)	Native, Controlled and Proxy system (see IEEE Std 802.1CS, LRP)
Attribute types	[Q]:Table 35-1	Table 6-6
Traffic classes for streams	SR class	RA class
Traffic specification	[Q]:35.2.2.8.4 (MSRPv0) [Q]:35.2.2.10.6 (MSRPv1)	6.4.3.6 6.4.3.7
Queuing and transmission functions for streams	FQTSS in [Q]:34	RA class template based
Support for multiple-path streams	only in MSRPv1 with CNC	6.3.11

RAP enables MSRP devices to connect to a RAP domain for backward compatibility within a specific MSRP class.

This is limited to MSRPv0, and Streams shall use RA class template 00-80-C2-03 (D.5).

Note that in this configuration the expected performance is limited by MSRP.

6.3.12.1 Startup

If both MSRP and LRP are enabled for a Bridge Port, MSRP domain attributes are sent and an LRP-Portal is started. Either MSRP or RAP Talkers and Listener are accepted at one Bridge Port. If an LRP-Portal is established for a Bridge Port, MSRP support is disabled for this Bridge Port.

Dependent on the connected devices to a Bridge Port, either MSRP or LRP is activated on that Bridge Port (6.3.3).

If MSRP support is enabled, the Bridge shall support the MSRP class (D.5).

6.3.12.2 Attributes and Failure Code translation

Table 6-4 shows the relationship between MSRP and RAP attributes.

Table 6-4—MSRP and RAP attribute relationship

MSRPv0	RAP
StreamID ([Q]: 35.2.2.8.2)	StreamID (6.4.3.1)
PriorityAndRank, Rank ([Q]: 35.2.2.8.5)	StreamRank (6.4.3.2)
PriorityAndRank, Data Frame Priority ([Q]: 35.2.2.8.5)	DataFrameParameters, Priority (6.4.3.5.2)
DataFrameParameters, destination_address ([Q]: 35.2.2.8.3)	DataFrameParameters, DestinationMacAddress (6.4.3.5.1)
DataFrameParameters, vlan_identifier ([Q]: 35.2.2.8.3)	DataFrameParameters, VID (Single-Context) (6.4.3.5.3)
TSpec, MaxFrameSize ([Q]: 35.2.2.8.4)	Interval-based TSpec, MaximumFrameSize (6.4.3.7.3)
TSpec, MaxIntervalFrames ([Q]: 35.2.2.8.4)	Interval-based TSpec, MaximumFramesPerInterval (6.4.3.7.2)
Accumulated Latency ([Q]: 35.2.2.8.6)	AccuMaxLatency (6.4.3.3)
FailureInformation, system identifier ([Q]: 35.2.2.8.7)	Failure Information, SystemId (6.4.3.10.1)
FailureInformation, Failure Code ([Q]: 35.2.2.8.7)	Failure Information, FailureCode (6.4.3.10.2)
SRClassID ([Q]: 35.2.2.9.2)	RaClassId (6.4.2.2.1)
SRClassPriority ([Q]: 35.2.2.9.3)	RaClassPriority (6.4.2.2.2)
SRclassVID ([Q]: 35.2.2.9.4)	RaClassTemplateDefinedData (6.4.2.2.5)

Table 6-5 shows the relationship between MSRP and RAP failure codes.

Table 6-5—MSRP and RAP failure code relationship

MSRPv0 Failure Codes ([Q]: Table 46-15)		RAP Failure Codes (Table 6-7)	
Failure Code	**Description**	**Failure Code**	**Description**
1	Insufficient bandwidth	0x02	Bandwidth constraint not met
2	Insufficient Bridge resources	0x05	Resource allocation failed
3	Insufficient bandwidth for traffic class	0x02	Bandwidth constraint not met
4	StreamID in use by another Talker	0x05	Resource allocation failed
5	Stream destination_address already in use	0x05	Resource allocation failed
6	Stream preempted by higher rank	0x06	Reservation preempted by Rank zero streams
7	Reported latency has changed	0x05	Resource allocation failed
8	Egress Port is not AVB capable	0x05	Resource allocation failed
9	Use a different destination_address (i.e., MAC DA hash table full)	0x05	Resource allocation failed

Table 6-5—MSRP and RAP failure code relationship

MSRPv0 Failure Codes ([Q]: Table 46-15)		RAP Failure Codes (Table 6-7)	
10	Out of MSRP resources	0x03	Resource constraint not met
11	Out of MMRP resources	0x03	Resource constraint not met
12	Cannot store destination_address (i.e., Bridge is out of MAC DA resources)	0x03	Resource constraint not met
13	Requested priority is not an SR Class priority	0x05	Resource allocation failed
14	MaxFrameSize is too large for media	0x05	Resource allocation failed
15	msrpMaxFanInPorts limit has been reached	0x03	Resource constraint not met
16	Changes in FirstValue, other than AccumulatedLatency, for a registered StreamID	0x05	Resource allocation failed
17	VLAN is blocked or filtered on this egress Port	0x05	Resource allocation failed
18	VLAN tagging is disabled on this egress Port (untagged set)	0x05	Resource allocation failed
19	SR class Priority mismatch	0x04	Talker Announcement across RA Class domain boundary

6.4 RAP attribute and TLV encoding definitions

This subclause defines the RAP attributes and associated TLV encoding.

6.4.1 General TLV definition

The information of RAP attributes is encoded in a Type-Length-Value (TLV) format, which consists of a 1-octet Type field that indicates the type of that TLV, a 2-octet Length field that indicates the number of octets

in the Value field, and then a Value field. The Value field of a TLV encodes zero or more fixed-size parameters, followed by zero or more TLVs of specific types. Figure 6-5 illustrates the general TLV format.

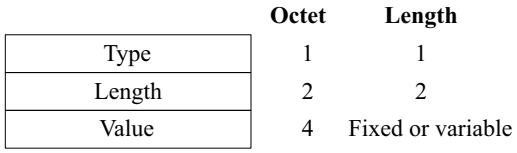

Figure 6-5—General TLV format

There are two categories of TLVs defined for RAP, attribute TLVs and sub-TLVs. An attribute TLV encodes a RAP attribute and can include one or more sub-TLVs in the Value field. An sub-TLV can further include one or more sub-TLVs. Attributes TLVs, as top-level TLVs, are never included in a sub-TLV or another attribute TLV. Table 6-6 lists the set of RAP attribute TLVs and sub-TLVs, and the values of the Type and Length field for each of them.

Table 6-6—RAP TLV and sub-TLV Type field and Length field values

TLV name	Type field (1 octet)	Length field (2 octets)	Reference
RA attribute TLV	0x00	variable	6.4.2
Talker Announce attribute TLV	0x01	variable	6.4.3
Listener Attach attribute TLV	0x02	variable	6.4.4
RA Class Descriptor sub-TLV	0x20	variable	6.4.2.2
Redundancy Context sub-TLV	0x21	variable	6.4.2.3
Data Frame Parameters sub-TLV	0x22	8	6.4.3.5
Talker Token Bucket TSpec sub-TLV	0x23	16	6.4.3.6
Interval-based TSpec sub-TLV	0x24	8	6.4.3.7
Token Bucket TSpec sub-TLV	0x25	16	6.4.3.8
Redundancy Control sub-TLV	0x26	variable	6.4.3.9
VLAN Context Information sub-TLV	0x27	variable	6.4.3.9.2
Failure Information sub-TLV	0x28	9	6.4.3.10
Organizationally Defined sub-TLV	0x29	variable	6.4.3.11
VLAN Context Status sub-TLV	0x2A	variable	6.4.4.4
Reserved for future standardization	0x03-0x1F, 0x2B-0xFF	-	-

The subsequent subclauses specify the encoding of the Value field for each attribute TLV and sub-TLV, by using the “big-endian” convention, in which higher significance bytes and bits within each byte appear to the left of and above bytes and bits of lower significance. Any fields that are labeled as “Reserved” are transmitted as zero and ignored on receipt. In addition, the term “the stream” used in the specification of Talker Announce attribute TLV and the sub-TLVs thereof refers to a stream for which the Talker Announce attribute is declared.

6.4.2 RA attribute and TLV encoding

The RA attribute is used in RA Advertisements (6.3.5). Figure 6-6 shows the encoding of the Value field of the RA attribute TLV.

	Octet	Length
MaxInterferingFrameSize	1	2
1 or more RA Class Desriptor sub-TLVs	3	variable
zero or more Redundancy Context sub-TLVs	variable	variable

Figure 6-6—Value of RA attribute TLV

6.4.2.1 MaxInterferingFrameSize

A 2-octet unsigned integer, indicating the maximum interfering frame size, in bytes, including media-dependent overhead ([Q]:12.4.2.2), that is allowed to be transmitted through the Port declaring the RA class attribute. The value of this parameter is determined by the operation of the underlying MAC Service on the Port.

6.4.2.2 RA Class Descriptor sub-TLV

One or more RA Class Descriptor sub-TLVs are included in the RA attribute TLV, each encoding in the Value field a set of parameters associated with a distinct RA class (6.3.2) as illustrated in Figure 6-7.

	Octet	Length
RaClassId	1	1
RaClassPriority	2	1
RTID	3	4
TrafficClass	7	1
RaClassTemplateDefinedData	8	variable

Figure 6-7—Value of RA Class Descriptor sub-TLV

6.4.2.2.1 RaClassId

A 1-octet unsigned integer containing the RA class ID (6.3.2.1).

6.4.2.2.2 RaClassPriority

A 1-octet unsigned integer containing the RA class priority (6.3.2.2).

6.4.2.2.3 RTID

A 4-octet RTID (6.3.2.3) identifying the RA class template used by the RA class.

6.4.2.2.4 TrafficClass

A 1-octet unsigned integer, indicating the traffic class ([Q]:8.6.6) to which the RA class priority is mapped on the Port where the RA attribute is declared.

6.4.2.2.5 RaClassTemplateDefinedData

The encoding of this field and the semantics associated with its values if any, is specific to the RA class template identified by the value contained in 6.4.2.2.3.

6.4.2.3 Redundancy Context sub-TLV

One or more Redundancy Context sub-TLVs are included in the RA attribute TLV, each describing a distinct Redundancy Context (6.3.11.2). The Value field contains one or more VLAN Context tuples, each encoding in the VlanContextId field a 12-bit VLAN Context ID, followed by a 4-bit Reserved field, as illustrated in Figure 6-8.

		Octet	Length
VLAN Context 1	VlanContextId	1	12 bits
	Reserved	2	4 bits
	...		
VLAN Context n	VlanContextId	2n-1	12 bits
	Reserved	2n	4 bits

Figure 6-8—Value of Redundancy Context sub-TLV

6.4.3 Talker Announce attribute and TLV encoding

The Talker Announce attribute is used in the Talker Announcement (6.3.6). Figure 6-9 shows the encoding of the Value Field of the Talker Announce attribute TLV.

	Octet	Length
StreamId	1	8
StreamRank	9	1
AccuMaxLatency	10	4
AccuMinLatency	14	4
Data Frame Parameters sub-TLV	18	11
Talker Token Bucket TSpec sub-TLV or Interval-based TSpec sub-TLV (Talker TSpec)	29	19 or 11
Token Bucket TSpec sub-TLV (Network TSpec)	variable	19
Redundancy Control sub-TLV *(only for Multi-Context Talker Announcement)*	variable	variable
0 or 1 Failure Information sub-TLV	variable	variable
0 or 1 RT class specific data sub-TLV	variable	variable

Figure 6-9—Value of Talker Announce attribute TLV

For support of two types of the Talker Announcement, the encoding of the Talker Announce attribute follows the following rules:

a) The Talker Announce attribute used in the Single-Context Talker Announcement [item a) in 6.7.2.1] contain no Redundancy Control sub-TLV (6.4.3.9).

b) The Talker Announce attribute used in the Multi-Context Talker Announcement [item b) in 6.7.2.1] always contains a Redundancy Control sub-TLV (6.4.3.9).

6.4.3.1 StreamId

An 8-octet field encoding the StreamID element as specified in [Q]:46.2.3.1.

6.4.3.2 StreamRank

A 1-octet field encoding a Rank value as specified in [Q]:46.2.3.2.1.

6.4.3.3 AccuMaxLatency

A 4-octet field encoding the AccumulatedLatency element as specified in [Q]:46.2.5.2.

6.4.3.4 AccuMinLatency

A 4-octet unsigned integer, indicating the minimum latency, in nanoseconds, that a single frame of the stream can encounter when transmitted from the Talker along a given path to the Port declaring this Talker Announce attribute.

6.4.3.5 Data Frame Parameters sub-TLV

This sub-TLV contains a set of parameters whose values are carried in the data frames of the stream when transmitted through the declaring Port of this attribute. Figure 6-10 shows the encoding of the Value field of the Data Frame Parameters sub-TLV.

	Octet	Length
DestinationMacAddress	1	6
Priority	7	3 bits
Reserved	7	1 bit
VID	7	12 bits

Figure 6-10—Value of Data Frame Parameters sub-TLV

6.4.3.5.1 DestinationMacAddress

A 6-octet destination MAC address of the data frames of the stream.

6.4.3.5.2 Priority

A 3-bit unsigned integer, indicating the priority to be encoded in the PCP field of the VLAN tag ([Q]:6.9.3), with which the data frames of the stream are tagged. This priority value is used by each receiving Bridge to associate the stream to a local RA class of the same priority.

6.4.3.5.3 VID

A 12-bit VID. The semantics of this field is dependent on the type of the Talker Announcement in which the Talker Announce attribute is used, as follows:

a) In the case of the Single-Context Talker Announcement, this field indicates a VID to be encoded in the VLAN tag with which the data frames of the stream are tagged, and also a single VLAN Context used by the Talker Announce attribute.

b) In the case of the Multi-Context Talker Announcement, this field is set to the numerically smallest VlanContextId value contained in a VLAN Context Information sub-TLV (6.4.3.9.2).

6.4.3.6 Talker Token Bucket TSpec sub-TLV

This sub-TLV contains a Talker Token Bucket TSpec as defined in item a) of 6.3.9. Figure 6-11 shows the encoding of the Value field of the Talker Token Bucket TSpec sub-TLV.

	Octet	Length
MaxPayloadLength	1	2
MinPayloadLength	3	2
CommittedInformationRate	5	8
CommittedBurstSize	13	4

Figure 6-11—Value of Talker Token Bucket TSpec sub-TLV

6.4.3.6.1 MaxPayloadLength

A 2-octet unsigned integer, encoding the MaxPayloadLength parameter [item a).1) of 6.3.9.1].

6.4.3.6.2 MinPayloadLength

A 2-octet unsigned integer, encoding the MinPayloadLength parameter [item a).2) of 6.3.9.1].

6.4.3.6.3 CommittedInformationRate

A 8-octet unsigned integer, encoding the CommittedInformationRate parameter [item a).3) of 6.3.9.1].

6.4.3.6.4 CommittedBurstSize

A 4-octet unsigned integer, encoding the CommittedBurstSize parameter [item a).4) of 6.3.9.1].

6.4.3.7 Interval-based TSpec sub-TLV

This sub-TLV contains an Interval-based TSpec as defined in item b) of 6.3.9.1. Figure 6-12 shows the encoding of the Value field of the Interval-based TSpec sub-TLV.

	Octet	Length
Interval	1	4
MaximumFramesPerInterval	5	2
MaximumFrameSize	7	2

Figure 6-12—Value of Interval-based TSpec sub-TLV

6.4.3.7.1 Interval

A 4-octet unsigned integer, encoding the Interval parameter [item b).1) of 6.3.9.1].

6.4.3.7.2 MaxFramesPerInterval

A 2-octet unsigned integer, encoding the MaximumFramesPerInterval parameter [item b).2) of 6.3.9.1].

6.4.3.7.3 MaxFrameSize

A 2-octet unsigned integer, encoding the MaximumFrameSize parameter [item b).3) of 6.3.9.1].

6.4.3.8 Token Bucket TSpec sub-TLV

This sub-TLV contains a Token Bucket TSpec as defined in 6.3.9.2. Figure 6-13 shows the encoding of the Value field of the Token Bucket TSpec sub-TLV.

	Octet	Length
MaxTransmittedFrameLength	1	2
MinTransmittedFrameLength	3	2
CommittedInformationRate	5	8
CommittedBurstSize	13	4

Figure 6-13—Value of Token Bucket TSpec sub-TLV

6.4.3.8.1 MaxTransmittedFrameLength

A 2-octet unsigned integer, encoding the MaxTransmittedFramelength parameter [item 1) of 6.3.9.2].

6.4.3.8.2 MinTransmittedFrameLength

A 2-octet unsigned integer, encoding the MinTransmittedFramelength parameter [item 2) of 6.3.9.2].

6.4.3.8.3 CommittedInformationRate

A 8-octet unsigned integer, encoding the CommittedInformationRate parameter [item 3) of 6.3.9.2].

6.4.3.8.4 CommittedBurstSize

A 4-octet unsigned integer, encoding the CommittedBurstSize parameter [item 4) of 6.3.9.2].

6.4.3.9 Redundancy Control sub-TLV

The Redundancy Control sub-TLV contains the information exclusively used by the Multi-Context Talker Announcement. Figure 6-14 shows the encoding of the Value field of the Redundancy Control sub-TLV.

	Octet	Length
RTagStatus	1	1 bit
Reserved	1	7 bits
1 or more VLAN Context Information sub-TLVs	2	variable

Figure 6-14—Value of Redundancy Control sub-TLV

6.4.3.9.1 RTagStatus

A Boolean value indicating whether the data frames of the stream are (TRUE) or are not (FALSE) to contain a Redundancy tag (R-TAG, [CB]:7.8) when transmitted through the Port that declares this sub-TLV.

6.4.3.9.2 VLAN Context Information sub-TLV

One or more VLAN Context Information sub-TLVs are included in the Redundancy Control sub-TLV, each describing a distinct VLAN Context (6.7.2.1). The Value field encodes in the VlanContextId field a 12-bit VLAN Context ID, followed by a 4-bit Reserved field and then zero or one Failure Information sub-TLV (6.4.3.10), as illustrated in Figure 6-15.

	Octet	Length
VlanContextId	1	12 bits
Reserved	2	4 bits
0 or 1 Failure Information sub-TLV	3	variable

Figure 6-15—Value of VLAN Context Information sub-TLV

6.4.3.10 Failure Information sub-TLV

The Failure Information sub-TLV contains the information about the location and the type of a failure encountered in the Talker Announcement. Figure 6-16 shows the encoding of the Value field of the Failure Information sub-TLV.

	Octet	Length
SystemId	1	8
FailureCode	9	1

Figure 6-16—Value of Failure Information sub-TLV

6.4.3.10.1 SystemId

An 8-octet unsigned integer, indicating the identifier of a Bridge or end station at which the failure occurs. The SystemId value for an end station shall be the 6-octet MAC Address of the end station’s port extended to 8 octets by prepending 2 octets of zero. The SystemId value for a Bridge shall be the 6-octet Bridge Identifier ([Q]:13.26.2) of the Bridge extended to 8 octets by prepending 2 octets of zero.

6.4.3.10.2 FailureCode

A 1-octet unsigned integer, containing the value of a RAP Failure Code defined in Table 6-7

Note: Error code values 0xFx are MSRP error codes that are not indicated by RAP.

.

Table 6-7—RAP Failure Codes

Name	Value	Description	Reference
LatencyExceeded	0x01	Latency constraint not met	6.3.8.1
BandwidthExceeded	0x02	Bandwidth constraint not met	6.3.8.1
ResourceExceeded	0x03	Resource constraint not met	6.3.8.2
CrossingDomainBoundary	0x04	Talker Announcement across RA Class domain boundary	6.7.2.4
ResourceAllocationFailed	0x05	Resource allocation failed	6.8.2.11, 6.8.2.12
ReservationPreempted	0x06	Reservation preempted by Rank zero streams	6.3.10
RTagFailed	0x07	Requirements for redundancy (un)tagging not fullfilled	6.3.11.3
StreamSplitFailed	0x08	Requirements for Stream splitting not fulfilled	6.3.11.4
MissingTalkerAnnounce	0x09	Missing Talker Announce at a Stream merging point	6.8.2.6
ListenerLackingFrer	0x0A	Listener lacking FRER capability	6.6.11.4
InconsistentRedundancyContext	0x0B	Inconsistent configuration of Redundancy Context	6.3.11.3
IngressBlocked	0x0C	Talker Annouce registration is ingress blocked	6.7.2.3
StreamIdAlreadyInUse	0x0E	Stream Id is already used by a different Talker	6.6.9.2
StreamDestinationAlreadyInUse	0x0F	Stream Destination is already used by a different Stream Id	6.6.5.3
RequestedPrioIsNotRAClassPrio	0x10	Requested priority is not an RA class priority	6.6.5.3
MaxFrameSizeTooLarge	0x11	Max frame size is too large	6.8.2.5
VlanTaggingDisabled	0x12	VLAN tagging is disabled on this Port	6.8.2.5
MaxFanInPortsExceeded	0x13	Max fan in ports limit has been reached	
ReportedLatencyHasChanged	0xFE		
EgressPortNotAVBCompatible	0xFD		
OUtOfMSRPResources	0xFC		
OutOfMMRPResources	0xFB		
MSRPCode16	0xFA		
VlanBlocked	0xF9		

6.4.3.11 RA class template specific sub-TLV

The RA class template specific sub-TLV contains information defined by an RA class template RTID. Figure 6-17 shows the encoding of the Value field of the RAclass template specific sub-TLV.

	Octet	Length
RaClassTemplateSpecificData	1	variable

Figure 6-17—Value of RA class template specific sub-TLV

6.4.3.11.1 OUI/CID

A 3-octet OUI or an CID.

6.4.3.11.2 OrganizationallyDefinedData

The encoding of this field and the semantics associated with its values if any, is specific to and defined by the organization that owns the OUI/CID contained in 6.4.3.11.1.

6.4.4 Listener Attach attribute and TLV encoding

The Listener Attach attribute is used in the Listener Attachment (6.3.7). Figure 6-18 shows the encoding of the Value field of the Listener Attach attribute TLV.

	Octet	Length
StreamId	1	8
VID	9	12 bits
ListenerAttachStatus	10	4 bits
VLAN Context Status sub-TLV *(only for Multi-Context Listener Attachment)*	11	variable

Figure 6-18—Value of Listener Attach attribute TLV

For support of two types of the Listener Attachment, the encoding of the Listener Attach attribute follows the following rules:

a) The Listener Attach attribute used in the Single-Context Listener Attachment [item a) in 6.3.7.1] contains no VLAN Context Status sub-TLV (6.4.4.4).

b) The Listener Attach attribute used in a Multi-Context Listener Attachment [item b) in 6.3.7.1] contains one VLAN Context Status sub-TLV (6.4.4.4).

6.4.4.1 StreamId

An 8-octet field encoding the StreamID element as specified in [Q]:46.2.3.1.

6.4.4.2 VID

A 12-bit integer, indicating a VID to be encoded in the VLAN tag with which the data frames of the stream are tagged.

6.4.4.3 ListenerAttachStatus

A 4-bit field, taking one of the following enumerated values to indicate the Listener Attach status (6.3.7.4):

a) **1: Attach Ready** [item a) in 6.3.7.4];
b) **2: Attach Fail** [item b) in 6.3.7.4];
c) **3: Attach Partial Fail** [item c) in 6.3.7.4].

6.4.4.4 VLAN Context Status sub-TLV

The VLAN Context Status sub-TLV contains the information exclusively used by the Multi-Context Listener Attachment. The Value field contains one or more VLAN Context tuples, each encoding in VlanContextId field a 12-bit VLAN Context ID, followed by a 4-bit PathStatus field, as illustrated in Figure 6-19.

		Octet	Length
VLAN Context tuple 1	VlanContextId	1	12 bits
	PathStatus	2	4 bits
	...		
VLAN Context tuple n	VlanContextId	2n-1	12 bits
	PathStatus	2n	4 bits

Figure 6-19—Value of VLAN Context Status sub-TLV

6.4.4.4.1 PathStatus

A 4-bit field, taking one of the following enumerated values to indicate the Path status (6.3.7.6):

a) **0: Faultless Path** [item a) in 6.3.7.6];
b) **1: Faulty Path** [item b) in 6.3.7.6];
c) **2: Unresponsive Path** [item c) in 6.3.7.6].

6.5 RAP Endpoint

6.5.1 RAP Endpoint overview

A RAP Endpoint associated with an end station is responsible for the following:

— Provision of service interface to higher layer entities (termed “RAP Endpoint user”) for controlling of resource allocation by end stations.
— Handling of resource allocation for an end station.
— Generation of attributes for declarations and processing of received attribute registrations.

The operation of a RAP Endpoint is specified by the following:

— RAP Endpoint Service Interface (RESI) in 6.5.2.
— RAP Endpoint relevant MAD functionality in 6.6.

6.5.2 RAP Endpoint Service Interface (RESI)

The RESI provided to RAP Endpoint user comprises a set of primitives and associated parameters, which are divided into three groups, as summarized in Table 6-8.

Table 6-8—RAP Endpoint Service Interface primitives

	primitive	reference
RA primitives (6.5.2.1)	DECLARE_RA.request	6.5.2.1.1
	REGISTER_RA.indication	6.5.2.1.2
TA primitives (6.5.2.2)	ANNOUNCE_STREAM.request	6.5.2.2.1
	ANNOUNCE_STREAM.indication	6.5.2.2.2
	TERMINATE_STREAM.request	6.5.2.2.3
	TERMINATE_STREAM.indication	6.5.2.2.4
LA primitives (6.5.2.3)	ATTACH_STREAM.request	6.5.2.3.1
	ATTACH_STREAM.indication	6.5.2.3.2
	DETACH_STREAM.request	6.5.2.3.3
	DETACH_STREAM.indication	6.5.2.3.4

6.5.2.1 RA primitives

The RA primitives specified in 6.5.2.1.1 and 6.5.2.1.2 are used for controlling and monitoring RA Advertisement (6.3.5) in an end station.

The method of using the RA primitives is specific to each end station. For example, if the RAP Endpoint user in an end station is made aware of the RA class and Redundancy Context configurations in the network through a higher layer protocol, it can make an RA declaration without having to wait for an RA registration to be received from a neighboring station. Otherwise, the end station relies on the information received in an RA registration from a neighboring station to make an RA declaration.

6.5.2.1.1 DECLARE_RA.request()

This primitive is used by the RAP Endpoint user to request the RAP Endpoint to make an RA declaration. It has the following parameters:

a) **MaxInterferingFrameSize** (6.4.2.1)
b) One or more sets of the following parameters, each associated with a distinct RA class (6.4.2.2):
    1) **RAClassId** (6.4.2.2.1)
    2) **RaClassPriority** (6.4.2.2.2)
    3) **RTID** (6.4.2.2.3)
    4) **TrafficClass** (6.4.2.2.4)
    5) **RaClassTemplateDefinedData** (6.4.2.2.5)
c) Zero or more sets of **VLAN Context ID**s, each associated with an Redundancy Context (6.4.2.3).

6.5.2.1.2 REGISTER_RA.indication()

The REGISTER_RA.indication primitive is used by the RAP Endpoint to inform the RAP Endpoint user about an RA registration. This primitive has the same set of parameters as that defined in 6.5.2.1.1.

6.5.2.2 TA primitives

The TA primitives specified in the subsequent clauses (6.5.2.2.1 through 6.5.2.2.4) are used for controlling and monitoring Talker Announcement (6.3.6).

6.5.2.2.1 ANNOUNCE_STREAM.request()

This primitive is used by the RAP Endpoint user in a Talker to request the RAP Endpoint to initiate a Talker Announcement. It has the following parameters:

a) **StreamId** (6.4.3.1)
b) **StreamRank** (6.4.3.2)
c) **AccuMaxLatency** (6.4.3.3)
d) **AccuMinLatency** (6.4.3.4)
e) A 3-tuple of {**DestinationMacAddress** (6.4.3.5.1), **Priority** (6.4.3.5.2) and **VID** (6.4.3.5.3)}
f) Either of the following as TalkerTSpec (6.4.3.6 or 6.4.3.7):
    1) A 4-tuple of {**MaxPayloadLength** (6.4.3.6.1), **MinPayloadLength** (6.4.3.6.2), **CommittedInformationRate** (6.4.3.6.3) and **CommittedBurstSize** (6.4.3.6.4)}, indicating a Talker Token Bucket TSpec (6.4.3.6)
    2) A 3-tuple of {**Interval** (6.4.3.7.1), **MaxFramesPerInterval** (6.4.3.7.2) and **MaxFrameSize** (6.4.3.7.3)}, indicating an Interval-based TSpec (6.4.3.6)
g) **Redundancy Context ID** (6.3.11.2), if supplied, indicating a Multi-Context Talker Announcement and otherwise, indicating a Single-Context Talker Announcement
h) A 2-tuple of **{OUI/CID** (6.4.3.11.1) and **OrganizationallyDefinedData** (6.4.3.11.2)}, if supplied, indicating the use of the Organizationally Defined sub-TLV (6.4.3.11)
i) **Failure Information**, set to either zero or a non-zero RAP Failure Code defined in Table 6-7.

The RAP Endpoint shall not accept an ANNOUNCE_STREAM.request with a StreamId value for which it has initiated a Talker Announcement since a previous ANNOUNCE_STREAM.request.

6.5.2.2.2 ANNOUNCE_STREAM.indication()

This primitive is used by the RAP Endpoint to inform the RAP Endpoint user in a Listener about the result of processing a Talker Announcement received. In addition to all the parameters defined in 6.5.2.2.1, this primitive has the following additional parameter for use only in the case of a Multi-Context Talker Announcement:

a) A set of 3-tuples, each with {**VlanContextId**, **SystemId, FailureCode**} and corresponding to a VLAN Context Information sub-TLV (6.4.3.9.2), where SystemId and FailureCode are provided only when that VlanContextId is reported with a RAP Failure Code in the Talker Announcement.

NOTE—As a response to a received ANNOUNCE_STREAM.indication, the RAP Endpoint user of a Listener can, if it is interested in receiving that stream, initiates a Listener Attachment by issuing an ATTACH_STREAM.request (6.5.2.3.1) with the StreamId contained in the received primitive.

6.5.2.2.3 TERMINATE_STREAM.request()

This primitive is used by the RAP Endpoint user in a Talker to request the RAP Endpoint to terminate a Talker Announcement. It has a single parameter StreamId (6.4.3.1).

If a Talker Announcement to be terminated is associated with a stream being transmitted by the Talker, it is required that the Talker stops transmitting the stream before issuing a TERMINATE_STREAM.request.

NOTE—This requirement can be viewed as part of the fundamental principle of resource allocation, i.e., reservation prior to stream transmission.

6.5.2.2.4 TERMINATE_STREAM.indication()

This primitive is used by the RAP Endpoint to inform the RAP Endpoint user in a Listener that a Talker Announcement previously received has been removed. It has a single parameter StreamId (6.4.3.1).

NOTE—As a response to a received TERMINATE_STREAM.indication, the RAP Endpoint user of a Listener can, if it has initiated a corresponding Listener Attachment, terminate that Listener Attachment by issuing a DETACH_STREAM.request (6.5.2.3.3) with the StreamId contained in the received primitive.

6.5.2.3 LA primitives

The Listener primitives specified in the following subclauses (6.5.2.3.1 through 6.5.2.3.4) are used for controlling and monitoring Listener Attachment (6.3.7).

6.5.2.3.1 ATTACH_STREAM.request()

This primitive is used by the RAP Endpoint user in a Listener to request the RAP Endpoint to initiate a Listener Attachment. It has a single parameter StreamId (6.4.4.1).

NOTE—An ATTACH_STREAM.request issued with a given StreamId won’t actually cause initiation of a Listener Attachment, if a Talker Announce registration with that StreamId is not present in the RAP Endpoint.

6.5.2.3.2 ATTACH_STREAM.indication()

This primitive is used by the RAP Endpoint to inform the RAP Endpoint user in a Talker about the result of processing a Listener Attachment received. It has the following parameters:

a) **StreamId** (6.4.4.1)
b) **VID** (6.4.4.2)
c) **ListenerAttachStatus** (6.4.4.3)

d) A set of 2-tuples, each with {**VlanContextId**, **PathStatus**} (6.4.4.4)

NOTE—The ListenerAttachStatus value and in the case of a Multi-Context Listener Attachment the set of PathStatus values contained in a received ATTACH_STREAM.indication can be used by the RAP Endpoint user to instruct the Talker whether and when to start transmission of the stream concerned, according to the semantics associated with these status parameters.

6.5.2.3.3 DETACH_STREAM.request()

This primitive is used by the RAP Endpoint user in a Listener to request the RAP Endpoint to terminate a Listener Attachment previously initiated. It has a single parameter StreamId (6.4.4.1).

NOTE—The RAP Endpoint user of a Listener can issue a DETACH_STREAM.request to terminate a specific Listener Attachment it has initiated previously at any time as desired, including but not limited to the case when receiving a TERMINATE_STREAM.indication (6.5.2.2.4) that indicates the associated Talker Announcement has been terminated.

6.5.2.3.4 DETACH_STREAM.indication()

This primitive is used by the RAP Endpoint to inform the RAP Endpoint user in a Talker that a Listener Attachment previously received for a Talker Announcement initiated by the Talker. It has a single parameter StreamId (6.4.3.1).

NOTE—A notification of a removed Listener Attachment occurs only when the RAP Endpoint of a Talker has removed any Listener Attach registration previously received as belonging to that Listener Attachment. In this case, it is up to the RAP Endpoint user to determine whether to retain the Talker Announcement concerned.

6.6 RAP MAD

6.6.1 RAP MAD Overview

Based on the Q-Standard ([Q]:10.2) this chapter specifies the operation of the MAD function for the RAP application. RAP MAD is an application specific MAD replacement for RAP that is used instead of the MAD that is described in [Q].

6.6.2 LRP Data Record to Attribute translation

An LRP Portal provides events for LRP records which contain RAP attributes in the form of TLVs.

The translation from attributes to records and vice versa follows following rule(s):

a) An attribute shall stay within the same record throughout its lifetime.

6.6.3 RAP MAD ingress state machine for Bridges

The operation of the RAP MAD ingress state machine is specified by the state machine diagram in Figure 6-20 .

For each Port, an instance of the RAP MAD ingress state machine is required.

The LRP-DS service primitive RECORD_WRITTEN.indication (Table 6-10) triggers the RAP MAD ingress state machine depending on the attributes that are received within the data record:

a) If a new RA class attribute (6.4.2) is received, the RA_REG condition is signaled to the state machine.
b) If a previously received RA class attribute (6.4.2) is no longer received, the RA_DEREG condition is signaled to the state machine.

c) If a new Talker Announce attribute (6.4.3) is received, the TA_REG condition is signaled to the state machine.

d) If a previously received Talker Announce attribute (6.4.3) is no longer received, the TA_DEREG condition is signaled to the state machine.

e) If a new Listener Attach attribute (6.4.4) is received, the LA_REG condition is signaled to the state machine.

f) If a previously received Listener Attach attribute (6.4.4) is no longer received, the LA_DEREG condition is signaled to the state machine.

The actions executed on entry to each state of the state machine are specified in clause 6.6.5.

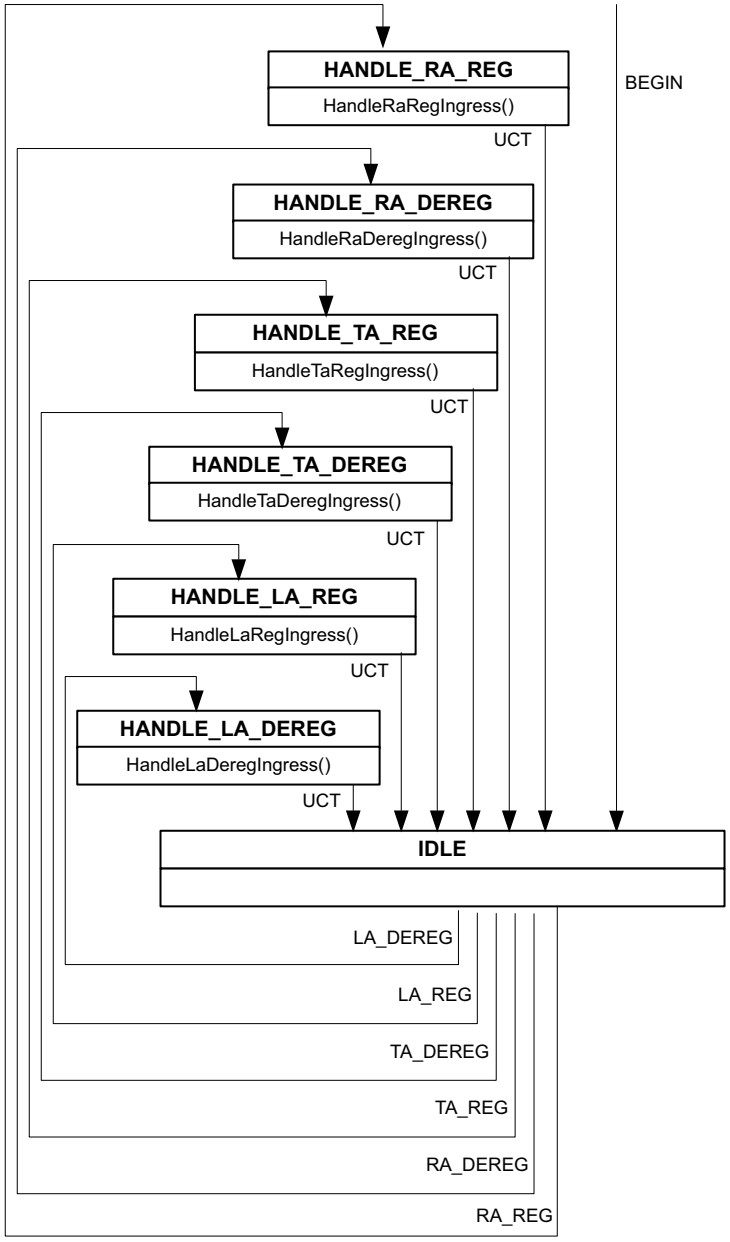

Figure 6-20—RAP MAD ingress state machine

6.6.4 RAP MAD Variables

6.6.4.1 rapPortEnabled

This variable is TRUE if RAP MAD is enabled for this port.

6.6.5 RAP MAD ingress procedures

RAP MAD ingress functions handle the events, received from an LRP-Portal.

6.6.5.1 HandleRaRegIngress()

This procedure handles the event of receiving a registration of an RA attribute from an LRP-Portal as follows:

Update neighbor's RAClass (6.7.5.6) information on the associated Port.

Check for each local RAClass at receiving Port if this Port is a boundary port and set the domainBoundaryPort [item c) in 6.7.5.8] accordingly.

Note: a Port is a boundary port if there is no neighbor with the same ClassID and the same priority (PCP)

For all registered TA for the affected RA class on this port call HandleTaRegIngress() (6.6.5.3).

For all registered LA for the affected RA class on this port call HandleLaRegIngress() (6.6.5.5).

6.6.5.2 HandleRaDeregIngress()

This procedure handles the event of receiving a deregistration of an RA attribute from an LRP-Portal as follows:

Update neighbor's RAClass (6.7.5.6) information on the associated Port.

Check for each local RAClass at receiving Port if this Port is a boundary port and set the domainBoundaryPort [item c) in 6.7.5.8] accordingly.

Note: a Port is a boundary port if there is no neighbor with the same ClassID and the same priority (PCP)

For all registered TA for the affected RA class on this port call HandleTaRegIngress() (6.6.5.3).

For all registered LA for the affected RA class on this port call HandleLaRegIngress() (6.6.5.5).

6.6.5.3 HandleTaRegIngress()

This procedure handles the event of receiving a registration of a TA attribute from a LRP-Portal.

If TA is valid (isTaValid() (6.6.9.2) == TRUE):

a) If a TA with the same StreamID and VID was already received at the same port:
   1) update Stream TA registration (6.7.5.1).
b) If the priority indicated in the RA registration is not found in any RAclass, set the ingressStatus to TA_INGRESS_FAIL and the ingressFailureCode to *RequestedPrioIsNotRAClassPrio*.
c) Set the ingressStatus of the TA registration attribute [item e) in 6.7.5.1] according to the boundary state of the associated RA class.
d) If Ingress Blocking prohibits propagation of the TA registration (6.7.2.3), set the ingressStatus to TA_INGRESS_FAIL and the ingressFailureCode to *IngressBlocked.*
e) If the TA is a Multi-Context Talker Announcement and the redundancy context this Talker is associated with is not consistent, set the ingressStatus to TA_INGRESS_FAIL and the ingressFailureCode to *InconsistentRedundancyContext.*
f) If FRER is required but the bridge is not FRER-capable, set ingressStatus to TA_INGRESS_FAIL and set ingressFailureCode to *RTagFailed*.

g) If there is already a TA registration for the same VID and DestinationMacAddress, , set the ingressStatus to TA_INGRESS_FAIL and the ingressFailureCode to *StreamDestinationAlreadyInUse*
h) Generate MAD_Join.indication(TA_REG) ([Q]:10.2).

otherwise

i) Stop TA propagation by generating MAD_Leave.indication(TA_REG) ([Q]:10.2).

6.6.5.4 HandleTaDeregIngress()

This procedure handles the event of receiving a deregistration of a TA attribute from an LRP-Portal.

Stop TA propagation by generating MAD_Leave.indication(TA_REG) ([Q]:10.2)..

6.6.5.5 HandleLaRegIngress()

This procedure handles the event of receiving a registration of an LA attribute from an LRP-Portal.

If all of the following conditions are met:

— there is a corresponding TA declaration,
— the LA attribute indicates either **Attach Ready** or **Attach Partial Fail**
— the corresponding TA declaration indicates **Announce Success**

a) Store or update local LA registration information.
b) Call updateLaReservations() (6.6.9.5).
c) Generate MAD_Join.indication(LA_REG) ([Q]:10.2).

otherwise if there is no corresponding TA declaration:

a) Set LA to **Attach Fail**
b) Call updateLaReservations() (6.6.9.5).
c) If this LA was already received on this port, generate MAD_Leave.indication(LA_REG) ([Q]:10.2).

otherwise:

a) Set LA to **Attach Fail**
b) Call updateLaReservations() (6.6.9.5).
c) Generate MAD_Join.indication(LA_REG) ([Q]:10.2).

6.6.5.6 HandleLaDeregIngress()

This procedure handles the event of receiving a deregistration of an LA attribute from an LRP-Portal.

If this LA registration exists on this port:

a) If this LA is part of an active reservation:
    1) Call updateLaReservations() (6.6.9.5).
b) Generate MAD_Leave.indication(LA_REG) ([Q]:10.2).

6.6.6 RAP MAD egress state machine for Bridges

The operation of the RAP MAD egress state machine is specified by the state machine diagram in Figure 6-21 .

For each Port, an instance of the RAP MAD egress state machine is required.

The RAP MAD egress state machine is triggered by the following events:

— MAD_Leave and MAD_Join events are issued by MAP.
— The LRP-Portal-connected event is triggered by the reception of a Portal status indication(connected) (CS: 10.2.6).
— The Reset-Operation-Mode event is generated by the RAP Port Instance to reenable MSRP-mode.

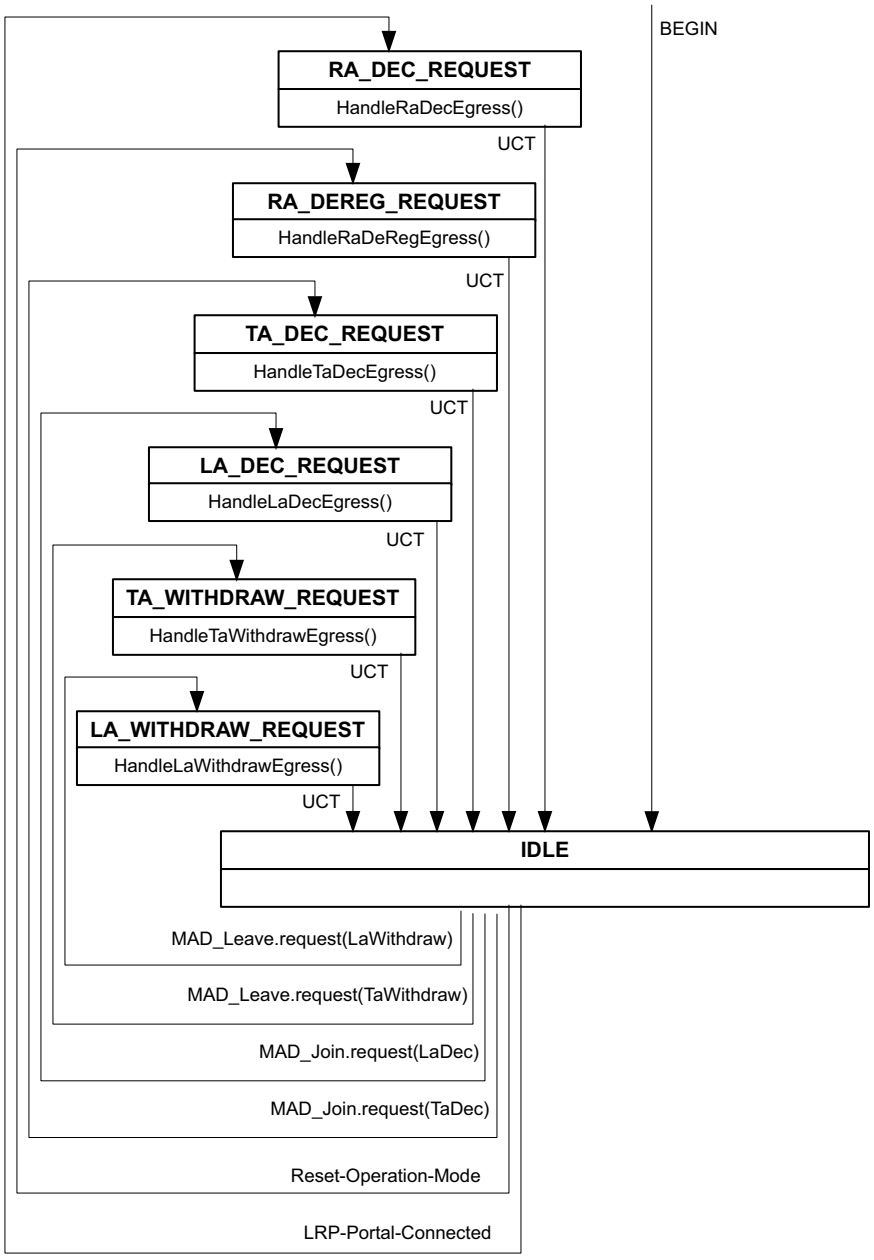

Figure 6-21—RAP MAD egress state machine

6.6.7 RAP MAD egress procedures

RAP MAD egress functions handle events after they are processed by MAP.

6.6.7.1 HandleRaDecEgress()

Declare all RA declarations at LRP Portal.

6.6.7.2 HandleRaDeDecEgress()

Remove all RA declarations from LRP Portal.

6.6.7.3 HandleTaDecEgress()

If the TA is a Multi-Context Talker Announcement and the redundancy context this Talker is associated with is not consistent, set the RAP Failure Code *InconsistentRedundancyContext.*

If no listener corresponding to the TA declaration exists, or if no reservation has yet been established:

a) Evaluate the stream reservation feasibility using the checkReservationConstraints() function (6.6.9.6) and set the appropriate error code in the TA declaration attribute.

If an associated LA at the portal was already received before:

b) call HandleLaRegIngress() (6.6.5.5).

Declare TA at LRP Portal.

6.6.7.4 HandleTaWithdrawEgress()

This function handles the egress processing of a TA deregistration issued by MAP to remove TA Declarations.

Withdraw TA declaration at LRP Portal.

If there is an associated LA registration:

a) Call HandleLaRegIngress() (6.6.5.5)

6.6.7.5 HandleLaDecEgress()

If there is an associated TA registration:

a) Call processLaDec() (6.8.2.8).
b) Declare LA at LRP Portal.

6.6.7.6 HandleLaWithdrawEgress()

This function handles the egress processing of a LA deregistration issued by MAP.

Withdraw LA declaration at LRP Portal.

6.6.8 Definition of LRP protocol elements

This subclause specifies the LRP protocol elements that are specific to the operation of RAP.

6.6.8.1 Value of AppId

The general format of AppIds is defined in [CS]:9.2. The AppId that identifies the RAP specified in this standard shall take the values shown in Table 6-9.

Table 6-9—The AppId value for RAP

Field	Value	Length (octets)	Offset (octets)
OUI or CID	00-80-C2	3	0
Application Sub-ID	1	1	3

6.6.8.2 Use of LLDP

When the IEEE Std 802.1AB Link Layer Discovery Protocol (LLDP) is deployed to advertise and discover target ports, the destination MAC address of the LLDP agent that is selected to carry the AppId (6.6.8.1) in an LRP ECP Discovery TLV ([CS]:C.2.1) and/or an LRP TCP Discovery TLV ([CS]:C.2.2), shall be the Nearest Bridge group address (01-80-C2-00-00-0E) as specified in [Q]:Table 8-1, [Q]:Table 8-2, and [Q]:Table 8-3.

6.6.8.3 Application Information TLV

An Application Information TLV, in the format specified in [CS]:9.4.2.10 and exchanged via LRP Hello LRPDUs ([CS]:9.4.2), is used by each RAP MAD instance to advertise the information about its capabilities and other operational parameters to its neighbor(s). Figure 6-22 shows the encoding of the Value field of the Application Information TLV:

	Octet	Length
ProtocolVersion	1	1

Figure 6-22—Value of Application Information TLV

a) **ProtocolVersion**: The ProtocolVersion for the version of RAP defined in this standard takes the hexadecimal value 0x00.

6.6.8.4 Use of LRP-DS service interface

For the purposes of the operation of a RAP MAD, a set of primitives and associated parameters chosen from the LRP-DS service interface defined in [CS]:10 are summarized in Table 6-10.

Table 6-10—LRP-DS service primitives used by RAP

Primitive	Subclause in IEEE Std 802.1CS-2020	Parameter	Item in IEEE Std 802.1CS-2020
FIRST_HELLO.indication [a]	10.2.5	PortalId	10.2.5.1 a)
		HelloData	10.2.5.1 b)
WRITE_RECORD.request	10.3.1.1	PortalId	10.3.1.1.1 a)
		RecordNumber	10.3.1.1.1 b)
		RecordData	10.3.1.1.1 c)
DELETE_RECORD.request	10.3.2.1	PortalId	10.3.2.1.1 a)
		RecordNumber	10.3.2.1.1 b)
RECORD_WRITTEN.indication	10.3.2.3	PortalId	10.3.2.3.1 a)
		RecordNumber	10.3.2.3.1 b)
		RecordData	10.3.2.3.1 c)

[a] It is assumed that a FIRST_HELLO.indication primitive invoked due to receiving a Hello message on a given Port by LRP be passed to a RAP MAD on that Port.

6.6.9 RAP MAD procedures

6.6.9.1 HandlePortOperStatusChange()

This procedure handles the event of receiving a change notification about a port's operational state.

If the port gets operational:

a) For each TA declaration, update TA at the LRP Portal.
b) For each existing associated LA declaration update LA at LRP-Portal.
c) For each enabled RA-class on this port call HandleRaRegIngress() (6.6.5.1).
d) For each TA registration, call HandleTaRegIngress() (6.6.5.3).
e) For each LA registration, call HandleLaRegIngress() (6.6.5.5).

otherwise:

f) For each enabled RA-class on this port call HandleRaDeregIngress() (6.6.5.2).
g) For each LA registration, call HandleLaDeregIngress() (6.6.5.6).
h) For each TA registration, call HandleTaDeregIngress() (6.6.5.4).

6.6.9.2 isTaValid()

This procedure determines whether a Talker Announce registration is valid (TRUE) or not (FALSE). It checks if there is already one or more existing Talker Announce registrations with the same StreamId and compares the new TA registration with the previous TA registrations. The procedure returns FALSE and sets the ingressFailureCode accordingly if one or more of the following conditions are met:

a) the result of the procedure isSingleContext() (6.6.9.4) for the previous and the new TA registration is different
b) the StreamRank of the previous and the new TA registration is different
c) the DestinationMacAddress of the previous and the new TA registration is different
d) the Priority of the previous and the new TA registration is different
e) the result of the procedure isSingleContext() for the new TA registration is TRUE and the new TA registration has as different VID for the same port
f) the result of the procedure isSingleContext() for the new TA registration is TRUE and the new TA registration has as different port for the same VID
g) the result of the procedure isSingleContext() for the new TA registration is FALSE and the set of VLAN Context IDs contained in the Redundancy Control sub-TLV in the new TA registration is neither the same nor a subset of any Redundancy Context configured in redundancyContext (6.7.5.10) (*InconsistentRedundancyContext*)
h) if a TA with the same StreamId already exists and is not associated with the new TA, set the ingressFailureCode to *StreamIdAlreadyInUse*.

Otherwise, the procedure returns TRUE.

6.6.9.3 isTaIngressBlocked()

This procedure returns TRUE if a Talker Announce registration element meets all the conditions for Ingress Blocking (6.7.2.3) and FALSE otherwise.

6.6.9.4 isSingleContext()

This procedure returns TRUE if the Talker Announce attribute indicates a Single-Context Talker Announcement according to [item a) in 6.4.3], and returns FALSE if it indicates a Multi-Context Talker Announcement according to the [item b) in 6.4.3]

6.6.9.5 updateLaReservations()

If this LA or TA is failed or removed:

a) Stop forwarding to the port of the LA registration.
b) Deallocate resources for this LA registration.

Otherwise:

a) If the TA declaration is a rank 0 stream, drop other reservations for non rank 0 streams if otherwise the reservation for this LA registration would not succeed. Drop rank 1 streams in order of the defined priority, lowest first.
b) Do a stream reservation, using allocateResourcesNonMerged() (6.8.2.11) for single context streams or allocateResourcesMerged() (6.8.2.12) for multi context streams.
c) Update the status of the propagated LA registration and the outgoing TA declaration based on the success of the reservation.

Update the allocated bandwidth.

If any resource allocation changed:

a) Send a ResourceChanged event to the RAP Bridge event state machine

6.6.9.6 checkReservationConstraints()

This procedure performs checks to determine whether a stream as indicated by a Talker Announce declaration meets the resource allocation constraints (6.3.8). This subclause specifies only the input and output parameters, along with the rules of handling the stream Rank, but not the detailed algorithms that are specific to the RA class template used by the stream specific or dependent on the particular Bridge implementation.

In computing the current available resources, the existing reservations in this Bridge are treated according to the StreamRank value of the TA declaration, as follows:

a) If StreamRank contains the value zero, only those existing reservations being made for the streams of Rank zero are treated as “reserved”, and all other existing reservations for the streams of Rank one are treated as “not reserved” as they can be preempted as described in 6.3.10.
b) If StreamRank contains the value one, all of the existing reservations regardless of the stream Rank are treated as “reserved”.

The procedure checks each reservation constraint defined in 6.3.8, for which there is no requirement for the order in which one constraint is examined after another. In case that one or more of these constraints are not met, the procedure returns the value of the RAP Failure Code associated with one constraint not met, as described in 6.3.8. It returns the value zero, if all of the constraints are met.

6.6.10 RAP Endpoint state machine

The operation of the RAP Endpoint state machine is specified by the state machine diagram in Figure 6-23.

There is one instance of the RAP Endpoint state machine per RAP Endpoint.

The LRP-DS service primitive RECORD_WRITTEN.indication (Table 6-10) triggers the RAP Endpoint state machine depending on the attributes that are received within the data record:

a) If a new RA class attribute (6.4.2) is received, the RA_REG condition is signaled to the state machine.
b) If a previously received RA class attribute (6.4.2) is no longer received, the RA_DEREG condition is signaled to the state machine.
c) If a new Talker Announce attribute (6.4.3) is received, the TA_REG condition is signaled to the state machine.
d) If a previously received Talker Announce attribute (6.4.3) is no longer received, the TA_DEREG condition is signaled to the state machine.
e) If a new Listener Attach attribute (6.4.4) is received, the LA_REG condition is signaled to the state machine.
f) If a previously received Listener Attach attribute (6.4.4) is no longer received, the LA_DEREG condition is signaled to the state machine.

The actions executed on entry to each state of the state machine are specified in clause 6.6.11.

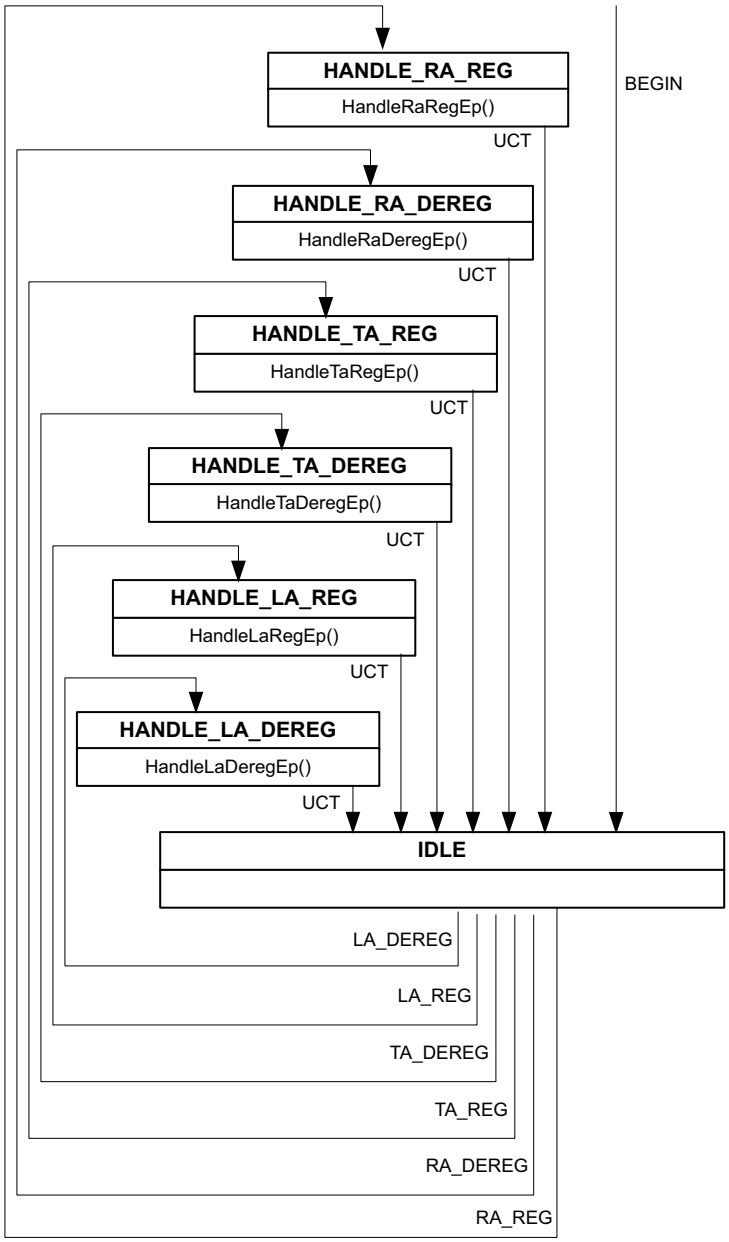

Figure 6-23—RAP Endpoint state machine

6.6.11 RAP Endpoint procedures

RAP Endpoint procedures handle events, received from an LRP-Portal in an Endpoint

6.6.11.1 HandleRaRegEp()

This procedure issues at RESI a REGISTER_RA.indication (6.5.2.1.2) with the parameter values abstracted from the RA registration supplied in the primitive.

6.6.11.2 HandleRaDeregEp()

This procedure issues at RESI a REGISTER_RA.indication (6.5.2.1.2) with the parameter values abstracted from the RA registration supplied in the primitive.

6.6.11.3 HandleTaRegEp()

This procedure performs the following actions:

a) If the TA attribute indicates a Multi-Context Talker Announcement and the RedundancyControl value in the TA attribute contains only part of the VLAN Contexts of a Redundancy Context declared by this end station, call **handleTaMerge**() (6.6.11.4) and terminate.
b) Copy the TA attribute to a new TA attribute.
c) If the TA attribute indicates **Announce Success**, perform checks of resource allocation constraints (6.3.8) and then the following: If all the resource allocation constraints are met, update both AccuMaxLatency and AccuMinLatency in the new TA attribute.
    2) Otherwise, fail TA attribute with the RAP Failure Code associated with a failure encountered during the constraint checking.
d) Issue at RESI an ANNOUCE_STREAM.indication (6.5.2.2.2) with its parameters set to the corresponding values contained in the new TA attribute.

6.6.11.4 handleTaMerge()

This procedure is invoked by the **handleTaReg**() procedure (6.6.11.3) to process a Multi-Context Talker Announcement identified by a StreamId and subject to Talker Announce merge (6.3.6.1) in the end station. It performs the following actions:

a) Search for each existing Talker Announce registration associated with the StreamId. If no such TA registration is found, issue a TERMINATE_STREAM.indication (6.5.2.2.4) with the StreamId and then terminate. Otherwise, proceed.
b) Construct a new Talker Announce attribute and fill each of the following fields with the common value contained in the corresponding field of each TA registration found in a): StreamId, StreamRank, DestinationMacAddress and Priority.
c) If one of the following conditions is met, fail the new TA attribute with the indicated RAP Failure Code and then proceed to g) below:
    1) *ListenerLackingFrer*, if this end station is not FRER-capable.
    2) *RTagFailed,* if there is at least one TA attribute whose RTagStatus contains FALSE.
d) Copy each VLAN Context Information sub-TLV (6.4.3.9.2) contained in each TA attribute found in a) to the RedundancyControl value in the new TA attribute, and set the VID in the new TA attribute to the numerically smallest VLAN Context ID copied.
e) If each TA registration found in a) indicates **Announce Fail**, fail the new TA attribute with the numerically smallest RAP Failure Code contained in RedundancyControl value in the new TA attribute and then proceed to g).
f) If the new TA attribute indicates **Announce Success**, perform checks of resource allocation constraints (6.3.8) on the new TA attribute and then the following:
    1) If all the resource allocation constraints are met, update both AccuMaxLatency and AccuMinLatency in the new TA attribute.

2) Otherwise, fail the mew TA attribute with the RAP Failure Code associated with a failure encountered during the constraint checking.

g) Issue at RESI an ANNOUCE_STREAM.indication (6.5.2.2.2) with its parameters set to the corresponding values contained in new TA attribute.

6.6.11.5 HandleTaDeregEp()

This procedure performs the following actions:

a) Remove the Talker Announce registration from the internal TA list.

b) If there is no remaining Talker Announce registration associated with the associated StreamId, issue at RESI a TERMINATE_STREAM.indication (6.5.2.2.4) with the StreamId.

c) Otherwise, call **handleTaMerge**() for the StreamId.

6.6.11.6 HandleLaRegEp()

This procedure performs the following actions:

a) Search for a Talker Announce declaration with which the LA attribute is associated, in accordance with 6.3.7.1. If no such TA declaration is found, terminate; otherwise, proceed.

b) If the LA attribute indicates either **Attach Ready** or **Attach Partial Fail**, the TA attribute indicates **Announce Success**, and there is no reservation associated with the LA attribute, then make a reservation for the LA attribute.

c) Else if the LA registration indicates **Attach Fail** and there is a reservation associated with the LA registration, then remove the reservation.

d) If there is more than one Talker Announce declaration whose StreamId is the same as the LA registration's StreamId, call **handleLaMerge**() with this StreamId; otherwise, issue at RESI an ATTACH_STREAM.indication (6.5.2.3.2) with its parameters set to the corresponding values contained in the LA registration.

6.6.11.7 handleLaMerge()

This procedure is invoked by the **handleLaReg**() procedure (6.6.11.6) to process a Multi-Context Listener Attachment identified by a StreamId and subject to Listener Attach merge (6.3.7.3) in the end station. It performs the following actions:

a) Search for each Listener Attach registration associated with the StreamId.

b) Construct a Listener Attach attribute as follows:

1) Set the StreamId in the LA attribute to the StreamId.

2) Set the VID in the attribute to the Redundancy Context Id used by this Listener Attachment.

3) Set the ListenerAttachStatus in the LA attribute to **Attach Ready**, if at least one La attribute found in a) indicates **Attach Ready**, and **Attach Fail**, otherwise.

4) Construct a VLAN Context Status sub-TLV (6.4.4.4) with each VlanContextId and PathStatus pair contained in each the LA attribute found in a), and append it to new LA attribute.

c) Issue at RESI an ATTACH_STREAM.indication (6.5.2.3.2) with its parameters set to the corresponding values contained in the new LA attribute.

6.6.11.8 HandleLaDeRegEp()

This procedure performs the following actions:

a) Remove the reservation associated with the LA attribute, if any, and then remove the Listener Attach registration in the LA attribute.
b) If there is no remaining Listener Attach registration associated with the StreamId, issue at RESI a DETACH_STREAM.indication (6.5.2.3.4) with the StreamId.
c) Otherwise, call **handleLaMerge**() for the StreamId.

6.6.12 RAP Endpoint Service Interface procedures

The subsequent subclauses specify a set of procedures for the RAP Endpoint to process request primitives received at RESI (6.5.2).

The procedures specified in these subclauses assume that the information about each attribute being declared and each attribute being registered is maintained in a database and accessible for the RAP Endpoint. A description about the data structure and operation methods of such a database is not provided; it is an implementation choice.

6.6.12.1 handleDeclareRaRequest()

This procedure is invoked when the RAP Endpoint receives at RESI a DECLARE_RA.request (6.5.2.1.1). It constructs the RA attribute (6.4.2) with the parameter values supplied and declares the RA at the LRP Portal.

6.6.12.2 handleAnnounceStreamRequest()

This procedure is invoked when the RAP Endpoint receives at RESI an ANNOUNCE_STREAM.request (6.5.2.2.1). It performs the following actions:

a) If there exist one or more Talker Announce declarations associated with the same StreamId (6.4.3.1) as indicated in the primitive, terminate the procedure.
b) Otherwise, construct a Talker Announce attribute TLV (6.4.3) with the values supplied in items a), b), c), d), e), f) and h) of 6.5.2.2.1.
c) If a non-zero RAP Failure Code value is supplied in i) of 6.5.2.2.1, construct a Failure Information sub-TLV (6.4.3.10) with that RAP Failure Code and the SystemId value of this end station, and then append it to the TA attribute TLV.
d) Construct in the TA attribute's NetworkTSpec a Token Bucket TSpec sub-TLV (6.4.3.6) with the values either copied from those supplied in item f).1) of 6.5.2.2.1 for a Token Bucket TSpec, or translated from those supplied in item f).2) of 6.5.2.2.1 for an Interval-based TSpec according to Table 6-1.
e) If no value is supplied in g) of 6.5.2.2.1, declare the TA attribute at the LRP Portal and then terminate. Otherwise, set RedundancyContextId to the value supplied.
f) If the end station is not FRER-capable, perform the following:
    1) Construct a Redundancy Control sub-TLV (6.4.3.9) with RTagStatus (6.4.3.9.1) set FALSE and with a set of VLAN Context Information sub-TLVs (6.4.3.9.2) constructed for each VLAN Context ID belonging to the Redundancy Context identified by RedundancyContextId (see item e), and then append it to TA attribute.
    2) Set VID in the TA attribute to RedundancyContextId (see item e).
    3) Declare the TA attribute at the LRP Portal.
g) Otherwise, i.e., the end station is FRER-capable, for each VLAN Context ID belonging to the Redundancy Context identified by RedundancyContextId (see item e), perform the following:
    1) Copy the TA attribute to a new TA attribute.

2) Construct a Redundancy Control sub-TLV (6.4.3.9) with RTagStatus (6.4.3.9.1) set TRUE and with a single VLAN Context Information sub-TLV (6.4.3.9.2) constructed for the VLAN Context ID, and then append it the new TA attribute.
3) Set the VID in the new TA attribute to the VLAN Context ID.
4) Declare the new TA attribute at the LRP Portal.

6.6.12.3 handleTerminateStreamRequest()

This procedure is invoked when the RAP Endpoint receives a TERMINATE_STREAM.request (6.5.2.2.3) at RESI with a StreamId value. It performs the following actions:

a) If there is no Talker Announce declaration associated with StreamId, terminate.
b) Otherwise, for each Talker Announce declaration associated with StreamId, withdraw the declaration with the corresponding Talker Announce attribute at the LRP Portal and then remove the Talker Announce declaration.

6.6.12.4 handleAttachStreamRequest()

This procedure is invoked when the RAP Endpoint receives at RESI an ATTACH_STREAM.request (6.5.2.3.1) with a StreamId value. It performs the following actions:

a) Search for each Talker Announce registration associated with the StreamId. If there is no corresponding TA, terminate; otherwise proceed.
b) If there is no reservation for the corresponding TA, make a reservation for the StreamId.
c) For each corresponding TA attribute found in a), declare the Listener Attach attribute at the LRP Portal that is constructed as follows:
    1) Set the StreamId value in the LA attribute to the StreamId.
    2) Set the VID value in the LA attribute to the VID in the TA attribute.
    3) Set the ListenerAttachStatus in the LA attribute to **Attach Ready**, if there is a reservation associated with the StreamId, and **Attach Fail**, otherwise.
    4) If the TA attribute indicates a Multi-Context Talker Announcement, construct a VLAN Context Status sub-TLV (6.4.4.4) with each VLAN Context Id contained in the TA attribute and the associated PathStatus set to **Faultless Path** if the LA attribute indicates **Attach Ready**, and **Faulty Path** if the LA attribute indicates **Attach Fail**, and then append it to the LA attribute.

6.6.12.5 handleDetachStreamRequest()

This procedure is invoked when the RAP Endpoint receives at RESI a DETACH_STREAM.request (6.5.2.3.3) with a StreamId value. It performs the following actions:

a) If there is no Listener Attach declaration associated with the StreamId, terminate.
b) If there is a reservation associated with the StreamId, remove the reservation.
c) For each Listener Attach declaration associated with the StreamId, withdraw the corresponding Listener Attach attribute at the LRP Portal and then remove the Listener Attach declaration.

6.7 MAP Extensions

6.7.1 MAP Extensions overview

Based on the Q-Standard ([Q]:10.3) this chapter specifies the operation of the MAP function for the RAP application. As the RAP application is not based on MRP but LRP, the MRP specific definitions in [Q]:10.3 do not apply to the interaction of RAP enabled MAD/MAP components.

For redundant streams RAP extends the propagation process beyond the MAP context defined in [Q]:10.3.

6.7.2 Talker Announce propagation

A set of rules used in Talker Announce propagation are defined in the following subclauses. These rules refer to specific Forwarding Process functions ([Q]:8.6) in VLAN Bridges, which can be configured by use of relevant protocols or management entities specified in [Q].

6.7.2.1 VLAN Context

The VLAN Context is what is described in Q also as MAP Context ([Q]:10.3.1).

Each Talker Announcement indicates one or more VLAN Contexts for use in Talker Announce propagation by Bridges. A VLAN Context in a Bridge is identified by a VID, termed VLAN Context ID, and corresponds to a VLAN topology formed by the set of Ports that includes each Bridge Port for which the following are true:

1) The Port is in the Forwarding state and part of the active topology that supports that VID.
2) The Port is a member of the member set ([Q]:8.8.10) for that VID.

Depending on the number of VLAN Contexts used, there are two types of the Talker Announcement, as follows:

a) **Single-Context Talker Announcement:** The Talker Announcement propagated in a single VLAN Context and used in resource allocation for transmission of a stream along a single path from the Talker to each Listener.
b) **Multi-Context Talker Announcement:** The Talker Announcement propagated in more than one VLAN Context and used in resource allocation for transmission of a stream with seamless redundancy along multiple paths from the Talker to each Listener.

One or more VLAN Contexts contained in a Talker Announce registration are used by a Bridge to determine a set of potential egress Ports in propagating a Talker Announce registration on an ingress Port. From a network-wide perspective, a specific VLAN Context used by a Talker Announcement constrains the extent of the Talker Announcement to a corresponding VLAN topology of the network.

NOTE—An appropriate VLAN configuration in the network for the set of VLAN Contexts used in resource allocation is required to control the reach of Talker Announcement. As one way to configure VLAN membership in Bridges for VLAN Contexts, each Listener desiring to receive a given Talker Announcement can use MVRP ([Q]:11.2) to issue an MVRP VLAN membership declaration for each VLAN Context ID to be used by that Talker Announcement. In such cases, the knowledge of the VLAN Context(s) concerned needs to be made available in advance to the Listener, e.g., by use of a specific higher layer protocol.

6.7.2.2 Stream DA Pruning

In addition to VLAN Context, Stream DA Pruning applies an additional constraint to Talker Announce propagation by reference to the status of MAC Address Registrations Entries ([Q]:8.8.4) in the FDBs of the Bridges. Stream DA Pruning can be enabled or disabled on a per-Bridge Port basis.

When Stream DA Pruning is enabled on a Port, a Talker Announce registration is not propagated to that Port, if the DestinationMacAddress (6.4.3.5.1) value contained in the Talker Announce attribute is not found in the MAC Address registration Entries for that Port.

NOTE—To propagate a Talker Announcement along a path containing one or more Bridges with Stream DA Pruning enabled on the egress port, a registration of the Stream Destination MAC Address used by the Talker Announcement on each such Port is required. As one way to register MAC addresses, a Listener on that path can use MMRP ([Q]:10) to issue an MMRP declaration for that MAC address. In such cases, the knowledge of the MAC address concerned needs to be made available in advance to the Listener, e.g., by use of a specific higher layer protocol.

6.7.2.3 Ingress Blocking

Ingress Blocking prohibits propagation of a Talker Announce registration on an ingress Port of a Bridge to any other egress Ports, if all of the following conditions are met:

a) Ingress Filtering ([Q]:8.6.2) is enabled on the Port.
b) The Port is not in the member set ([Q]:8.8.10) associated with the VID (6.4.3.5.3) of the Talker Announce registration.

NOTE—In addition to the need for VLAN Context on egress Ports, there is also a need for an appropriate VLAN configuration on each potential Talker Announce registration Port in the network based on the required reach of Talker Announcement. For example, in case that it is not desired to block a Talker Announcement on an ingress Port of any Bridge with Ingress Filtering enabled, the Talker can use MVRP ([Q]:11.2) to issue an MVRP VLAN membership declaration for the VID concerned. B.4 shows another example of VLAN configuration in a case where ingress blocking a Talker Announcement on specific Bridge Ports is intended.

6.7.2.4 Talker Announcement across RA class domains

For the purpose of providing diagnosis RAP allows Talker Announcement propagation across boundaries of RA class domains (6.3.2.4), but fails the Talker Announcement once it is propagated to a Bridge located outside the RA class domain from which it originates, with the RAP Failure Code *CrossingDomainBoundary* defined in Table 6-7. Talker Announce status.

The Talker Announce status of a Talker Announce declaration made on a given Port is associated with an ordinary stream in the case of the Single-Context Talker Announcement, or a Compound or a Member Stream in the case of the Multi-Context Talker Announcement, to be transmitted on that Port, indicating whether the stream could be reserved from the Talker to that Port along a single path or one or more paths, respectively, in the VLAN Context(s) indicted in the Talker Announce declaration, as follows:

a) **Announce Success:** The stream can be reserved on the Port, as it has not encounter any problems on a single path from the Talker to that Port, if the stream is an ordinary stream, or on at least one path from the Talker to that Port, if the stream is a Compound Stream or Member Stream. The Announce Success status is represented by the absence of a Failure Information sub-TLV (6.4.3.10) in the Value field of the Talker Announce attribute TLV (6.4.3).
b) **Announce Fail:** The stream can not be reserved on the Port, as it has encounter problems on each path from the Talker to that Port. The Announce Fail status is represented by the presence of a Failure Information sub-TLV (6.4.3.10) in the Value field Talker Announce attribute TLV (6.4.3).

6.7.3 Listener Attachment

Listener Attachment refers to the process of signaling the Listener Attach attribute (6.4.4) initially declared by one or more Listeners desiring to receive a stream across a Bridge network to the stream's Talker. Each Listener Attachment is identified by a StreamId (6.4.4.1) and VID (6.4.4.2) as contained in the Listener Attach attribute.

The purposes of Listener Attachment are as follows:

— Indication of the intention of one or more Listeners of receiving a stream.

— Triggering of resource allocations (or deallocation) on each Bridge along potential stream path(s).
— Signaling of reservation status hop-by-hop in an upstream direction from each participating Listener back to the Talker.

The following terminology relating to Listener Attachment is used:

a) **Listener Attach declaration**: A declaration of the Listener Attach attribute on a Port.
b) **Listener Attach registration**: A registration of the Listener Attach attribute on a Port.
c) **Listener Attach deregistration**: The removal of a Listener Attach registration on a Port.
d) **Listener Attach propagation**: Propagation of a Listener Attach registration from a Bridge Port to one or more other Bridge Ports.
e) **Listener Attach merge**: Merge of more multiple Listener Attach registrations propagated to a Bridge Port to result in a joint Listener Attach declaration on that Port.
f) **Associated Talker Announce declaration**: A Talker Announce declaration with which a given Listener Attach registration is associated is termed the associated Talker Announce declaration of that Listener Attach registration.
g) **Originating Listener Attach registration**: A Listener Attach registration from which a given Listener Attach declaration results is termed an originating Listener Attach registration of that Listener Attach declaration. A Listener Attach declaration can have more than one originating Listener Attach registration in the case of Listener Attach merge.
h) **Associated Talker Announce registration**: A Talker Announce registration with which a given Listener Attach declaration is associated is termed the associated Talker Announce registration of that Listener Attach declaration.

6.7.4 RAP MAP state machine

The events that control RAP MAP extensions are specified in the state machine diagram in Figure 6-24. The state changes are triggered by MAP or by the bridge component.

MAP uses the MAD_Join.indication() and MAD_Leave.indication() primitives to trigger the RAP MAP state machine. These indications are equivalent to the indications specified in [Q]:10.2, but are substituted by RAP specific indications with different parameters.

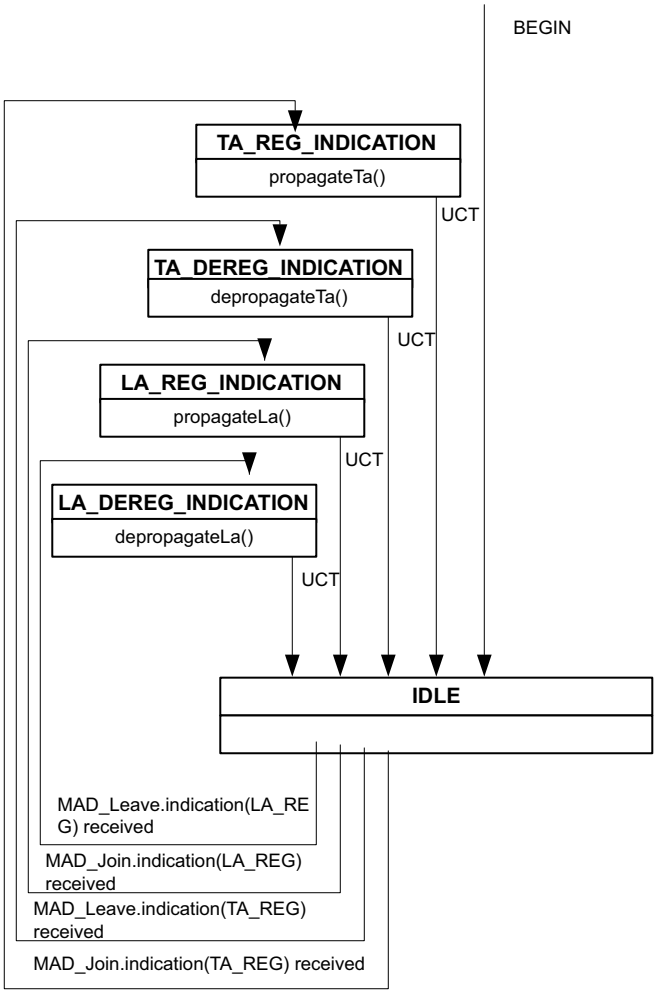

Figure 6-24—RAP MAP state machine

6.7.5 RAP MAP Variables

6.7.5.1 taReg

There is one taReg variable per bridge port that is an array in which each element represents a Talker Announce registration maintained RAP MAD and consists of the following member variables:

a) **attr**: The Talker Announce attribute (6.4.3) of the Talker Announce registration.
b) **invalid**: A Boolean indicating whether the Talker Announce registration is invalid (TRUE) or not (FALSE), as determined by the isTaInvalid() procedure (6.6.9.2).
c) **ingressBlocked**: A Boolean indicating whether the Talker Announce registration is ingress blocked (TRUE) or not (FALSE), as determined by (6.7.2.3).
d) **ingressStatus**: The ingress status of the Talker Announce registration, taking one of the following enumerated values:
    1) **TA_RECV_FAIL (0)**: The Talker Announce registration is failed by an upstream station.
    2) **TA_INGRESS_SUCCESS (1)**: The Talker Announce registration is neither failed by an upstream station nor failed on ingress of this Bridge.
    3) **TA_INGRESS_FAIL (2)**: The Talker Announce registration is failed on ingress of this Bridge with a RAP Failure Code indicated in ingressFailureCode [item e), below].
e) **ingressFailureCode**: A value of zero, indicating the Talker Announce registration is not failed on ingress of this Bridge, or the value of a RAP Failure Code defined in Table 6-7.
f) **rTagging**: A Boolean indicating whether the stream of this Talker Announce registration is subject to redundancy tagging (6.3.11.3) by this Bridge (TRUE) or not (FALSE). This variable is used only by a FRER-capable Bridge in processing a Multi-Context Talker Announcement.

6.7.5.2 taDec

There is one taDec variable per bridge port that is an array in which each element represents a Talker Announce declaration being made by RAP MAD and consists of the following member variables:

a) **attr**: The Talker Announce attribute (6.4.3) of the Talker Announce declaration.
b) **rUntagging**: A Boolean indicating whether the stream of this Talker Announce declaration is subject to redundancy untagging (6.3.11.3) on the Port as indicated in portRef (TRUE) or not (FALSE). This variable is used only by a FRER-capable Bridge in handling a Multi-Context Talker Announce declaration that is subject to Talker Announce merge (6.3.6.1).

6.7.5.3 laReg

There is one laReg variable per bridge port that is an array in which each element represents a Listener Attach registration maintained by RAP MAD and consists of the following member variables:

a) **attr**: The Listener Attach attribute (6.4.4) of the Listener Attach registration.
b) **assoTaDec**: A reference to an element in taDec (6.7.5.2) that is the associated Talker Announce declaration of the Listener Attach registration, or NULL indicating no associated Talker Announce declaration exists.
c) **isReserved**: A Boolean value indicating whether the Listener Attach registration is associated with a reservation in the Bridge (TRUE) or not (FALSE).
d) **reservationAge**: A 32-bit unsigned integer, indicating the time, in seconds, since a reservation associated with the Listener Attach registration was successfully made, and set to zero when the reservation is removed.
e) **ingressStatus**: The ingress status of the Listener Attach registration, taking one of the following enumerated values:

1) **NOT_PROPAGATED (0)**: The Listener Attach registration has no associated Talker Announce declaration and is not propagated.
2) **ATTACH_READY (1)**: The Listener Attach registration is propagated with **Attach Ready** [item a) in 6.4.4.3].
3) **ATTACH_FAIL (2)**: The Listener Attach registration is propagated with **Attach Fail** [item b) in 6.4.4.3].
4) **ATTACH_PARTIAL_FAIL (3)**: This Listener Attach registration is propagated with **Attach Partial Fail** [item c) in 6.4.4.3].

6.7.5.4 laDec

There is one laDec variable per bridge port that is an array in which each element represents a Listener Attach declaration maintained by RAP MAD and consists of the following member variables:

a) **assoTaReg**: A reference to an element in taReg (6.7.5.1) that is the associated Talker Announce registration of the Listener Attach declaration.
b) **attr**: The Listener Attach attribute (6.4.4) of the Listener Attach declaration.
c) **origLaRegList**: An array of references to one or more elements in laReg (6.7.5.3) that are originating Listener Attach registrations of the Listener Attach declaration and subject to Listener Attach merge (6.3.7.3). Each element in origLaRegList has a single variable origLaRegRef, which contains a reference to an originating Listener Attach registration.

6.7.5.5 localRaClass

There is one localRaClass variable per Bridge that is an array in which each element indicates an RA class supported by the Bridge and consists of the following member variables:

a) **id**: An RA class ID (6.3.2.1).
b) **priority**: An RA class priority (6.3.2.2)
c) **rtid**: An RTID (6.3.2.3)
d) **templateDefinedData:** The value to be contained in the RaClassTemplateDefinedData field (6.4.2.2.5) of the RA Class Descriptor sub- TLV for this RA class and carried in the RA attribute declared by this Bridge.

6.7.5.6 neighborRaClass

There is one neighborRaClass variable per Bridge Port that is an array in which each element indicates an RA class declared by a neighboring station on a Port of this Bridge and consists of a set of parameters copied from an RA Class Descriptor sub-TLV (6.4.2.2) contained in an RA registration on that Port, as follows:

a) **id**: The value of RaClassId (6.4.2.2.1).
b) **priority**: The value of RaClassPriority (6.4.2.2.2).
c) **rtid**: The value of RTID (6.4.2.2.3).
d) **trafficClass**: The value of TrafficClass (6.4.2.2.4).
e) **templateDefinedData:** The value in the RaClassTemplateDefinedData (6.4.2.2.5).

6.7.5.7 port

There is one port variable per Bridge Port that consists of the following member variables:

a) **streamDaPruningEnabled:** A Boolean indicating whether Stream DA Pruning (6.7.2.2) is administratively enabled (TRUE) or disabled (FALSE) on the Port.
b) **maxInterferingFrameSize:** The value of the MaxInterferingFrameSize parameter (6.4.2.1) associated with the Port.

c) **maxPropagationDelay**: An unsigned integer, indicating the maximum latency, in nanoseconds, a frame can experience when transmitted from the underlying physical medium on the Port to a reception port connected via a LAN to the Port.

d) **minPropagationDelay**: An unsigned integer, indicating the minimum latency, in nanoseconds, a frame can experience when transmitted from the underlying physical medium on the Port to a reception port connected via a LAN to the Port.

e) **portTransmitRate**: The transmission rate, in bits per second, that the underlying MAC Service that supports transmission through the Port provides. The value of this parameter is determined by the operation of the MAC.

6.7.5.8 portRaClass

There is one portRaClass variable per Bridge Port that is an array in which each element is associated with an RA class supported by the Bridge and a Bridge Port, and consists of the following member elements:

a) **raClassId**: The RA class ID (6.3.2.1) of the associated local RA class.

b) **domainBoundaryPort:** A Boolean indicating whether the Port is a domain boundary port for the RA class (TRUE) or not (FALSE).

c) **maxBandwidth**: An unsigned integer, indicating the maximum amount of bandwidth that can be allocated to the streams reserved in the RA class on the Port. The bandwidth value is represented as a percentage of the portTransmitRate [item e) in 6.7.5.7] value on that Port and expressed as a fixed-point number scaled by a factor of 1,000,000; i.e., 100,000,000 (the maximum value) represents 100%.

d) **allocatedBandwidth**: An unsigned integer, indicating the amount of bandwidth that has been allocated to the streams reserved in the RA class on the Port. The bandwidth value is represented as a percentage of the portTransmitRate [item d) in 6.7.5.7] value on that Port and expressed as a fixed-point number scaled by a factor of 1,000,000; i.e., 100,000,000 (the maximum value) represents 100%.

6.7.5.9 portPairRaClass

There is one portPairRaClass variable per Bridge that is an array in which each element is associated with an RA class supported by the Bridge and a reception Port transmission Port pair, and consists of the following member variables:

a) **receptionBridgePort**: The if-index of the associated reception Bridge Port ([Q]:48.1.1).

b) **transmissionBridgePort**: The if-index of the associated transmission Bridge Port ([Q]:48.1.1).

c) **raClassId**: The RA class ID (6.3.2.1) of the associated local RA class.

d) **maxHopLatency**: An unsigned integer, indicating a latency value, in nanoseconds, that specifies a latency constraint as described in 6.3.8.3. The latency is measured from a point located in an upstream station connected via a LAN to the reception Port, to a point located in the transmission Port, while the exact reference points are specific to and defined by the RA class template being used by the RA class.

6.7.5.10 redundancyContext

There is one redundancyContext variable per bridge that is an array in which each element represents a Redundancy Context supported by the Bridge and consists of the following member variables:

a) **id**: The Redundancy Context ID of the Redundancy Context, as defined in 6.3.11.2.

b) **vlanContextList**: The set of VLAN Context IDs associated with the Redundancy Context.

6.7.5.11 frerCapable

A Boolean value, indicating whether the Bridge is a FRER-capable RAP Bridge (TRUE) or not (FALSE).

6.7.5.12 maxProcessingDelay

An unsigned integer, indicating the maximum delay, in nanoseconds, that a frame can experience during the forwarding process of the Bridge until it is placed into an outbound queue.

6.7.5.13 minProcessingDelay

An unsigned integer, indicating the minimum delay, in nanoseconds, that a frame can experience during the forwarding process of the Bridge until it is placed into an outbound queue.

6.7.6 RAP MAP Procedures

6.7.6.1 propagateTa()

This procedure propagates a Talker Announce registration, by associating it with a new or existing Talker Announce declaration. The egress Ports for the propagation are determined by the VLAN context of the TA in single context case or the VLAN context list in the Redundancy Context otherwise.

For each egress Port:

a) If in a multi context case the stream cannot be forwarded to this port, set the *StreamSplitFailed* errorcode in the TA.
b) If the stream destination address is subject to MAC address pruning:
    1) if the TA was propagated at this egress port before: generate a MAD_Leave.request(TA_DEC) ([Q]:10.2) at the corresponding MAD instance.
    2) continue with the next egress port.
c) Call processTa() (6.8.2.5)
d) Generate a MAD_Join.request(TA_DEC) ([Q]:10.2) at the corresponding MAD instance.

6.7.6.2 depropagateTa()

This procedure stops propagation of a Talker Announce registration, as follows:

For each TA declaration that is associated with the corresponding TA registration: If the TA declaration is based on a TA merge operation for redundancy:

a) if there is no longer a TA declaration left in the redundancyContext (6.7.5.10):
    1) Generate a MAD_Leave.request(TaWithdraw) at the corresponding MAD Instance
b) otherwise
    2) call processTa() (6.8.2.5).

If isSingleContext() == TRUE:

a) Call processTa() (6.8.2.5) for each egress port and generate a MAD_Leave.request(TaWithdraw) ([Q]:10.2) at the corresponding MAD Instance.

6.7.6.3 propagateLa()

This procedures is called, when a MAD_Join.indication(LA_REG) ([Q]:10.2) is received.

If there is a corresponding TA for this LA:

a) for each egress port:
    1) if processLa() returns TRUE: generate a MAD_Join.request(LA_DEC) ([Q]:10.2) at the corresponding MAD Instance

6.7.6.4 depropagateLa()

This procedures is called, when a MAD_Leave.indication(LA_REG) ([Q]:10.2) is received.

If there is a corresponding TA for this LA:

a) for each egress port:
    1) Generate a MAD_Leave.request(LA_DEC) ([Q]:10.2) at the corresponding MAD Instance

6.8 RAP Generic Functions

6.8.1 RAP instance event state machine

The events that are generated by the RAP instance are specified in the state machine diagram in Figure 6-25. The state changes are triggered by the Bridge or end station component.

If the RAP instance detects a change in the VLAN configuration, it sends a VlanConfigChange event to the RAP instance event state machine to trigger the corresponding action. Such a change can result from a spanning tree ([Q]:7.3) or VLAN topology ([Q]:7.4) reconfiguration.

If the RAP instance detects a registration of a MAC address on a Port, it sends a MacAddrReg event to the RAP instance event state machine to trigger the corresponding action.

If the RAP instance detects a deregistration of a MAC address on a Port, it sends a MacAddrDereg event to the RAP instance event state machine to trigger the corresponding action.

If the RAP instance detects a a change of available resources, it sends a ResourceChanged event to the RAP instance event state machine to trigger the corresponding action.

If the RAP instance detects a change of the LRP Portal's operational status, it sends a LrpOperStatusChanged event to the RAP instance event state machine to trigger the corresponding action.

Not each particular bridge event has to be handled immediately, but can be handled in a cumulative processing.

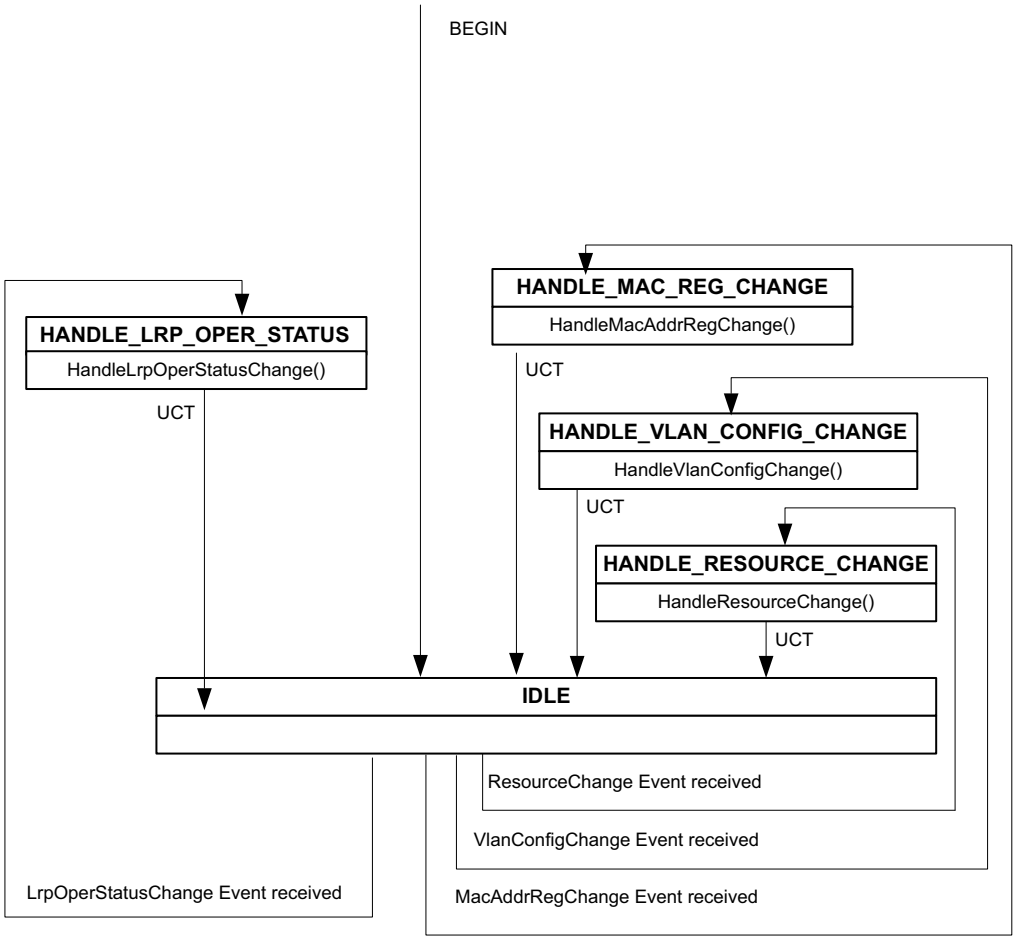

Figure 6-25—RAP instance event state machine

6.8.2 RAP generic procedures

6.8.2.1 HandleMacAddrRegChange()

This procedure is triggered by the RAP instance event state machine if it receives a MacAddrRegChanged event.

For each TA registration:

a) If the stream destination address was subject to MAC address pruning: Generate a MAD_Join.Indication(TA_REG).

6.8.2.2 HandleVlanConfigChange()

This procedure is triggered by the RAP instance event state machine if it receives a VlanConfigChange event.

For each TA registration:

a) If any of the stream's VLAN contexts is effected by the VLAN change: Generate a MAD_Join.Indication(TA_REG).

6.8.2.3 HandleResourceChange()

This procedure is triggered by the RAP instance event state machine if it receives a ResourceChange event.

For each TA declaration:

a) Generate a MAD_Join.Request(TA_DEC) at the corresponding MAD instance.

6.8.2.4 HandleLrpOperStatusChange()

This procedure is triggered by the RAP instance event state machine if it receives a LrpOperStatusChange event indicating the change of a LRP Portal's operational state.

If the port gets operational:

a) For each RA-class on this port call HandleRaRegIngress() (6.6.5.1).
b) For each existing associated LA declaration
    1) Call processLaDec() (6.8.2.8)
    2) Call updateLaReservations() (6.6.9.5).
    3) Generate MAD_Join.indication(LA_REG) ([Q]:10.2).

otherwise:

c) For each TA declaration with an associated LA registration:
    1) If there is a reservation for the LA, call updateLaReservations() (6.6.9.5).
    2) For each existing associated LA declarations generate MAD_Leave.indication(LA_REG) ([Q]:10.2).
d) Generate MAD_Leave.indication(TA_REG) ([Q]:10.2).

6.8.2.5 processTa()

This procedure validates a Talker Announce registration as follows for each egress Port:

If the TA status is already a failure state, do nothing and return from the procedure.

If the ingress-status of the Talker indicates an error, set the TA-status accordingly and return from the procedure.

Check the egress port for following conditions:

a) If the max frame size cannot be sent on this Port set the TA-status to *MaxFrameSizeTooLarge*.
b) If VLAN tagging is disabled for the VLAN ID the stream is to be sent, set the TA-status to *VlanTaggingDisabled*.

Set the accumulated latency and the network traffic specification by applying the setAccuLatencies() (6.8.2.9) and setNetworkTSpec() (6.8.2.10) functions to the TA.

For multi context talkers do the following:

a) check if the redundancy context of the TA is valid and set error code according to result.
b) check if rtag is required, set the RedundancyControl.RTagStatus accordingly.
c) If the stream is not to be merged at this bridge: set the VLAN Identifier (VID) to the numerically lowest VID among all known VLAN contexts associated with this stream.
d) Otherwise:
    1) call processTaMerged().

For multi-context talkers, the VLANContextInformation field of the TA declaration shall be restricted to include only those VLAN contexts that are to be propagated at the current bridge.

6.8.2.6 processTaMerged()

This procedure defines the processing of a Talker Announce declaration.

a) Use the StreamId (6.4.3.1), StreamRank (6.4.3.2), DestinationMacAddress (6.4.3.5.1) and Priority (6.4.3.5.2) from the originating TA Registration for the TA Declaration,
b) Set the Redundancy Control sub-TLV (6.4.3.9) as follows:
    1) Determine the list of VLANs from the Redundancy Context that are configured at the egress port,
    2) set the VID to the numerically lowest VID from this list,
    3) if the this list is the same as the list of the Redundancy Context, set the rUntagging of the TA to TRUE and RedundancyControl.RTagStatus to FALSE, otherwise set the rUntagging of the TA to FALSE and keep the RedundancyControl.RTagStatus as is,
    4) For each originating TA copy each VLAN Context Information sub-TLV (6.4.3.9.2) whose VlanContextId value is listed in determined list of VLANs to the RedundancyControl sub TLV.
    5) If a copied VLAN Context Information sub-TLV does not contain a Failure Information sub-TLV (6.4.3.10) and the originating TA indicates in ingressStatus TA_INGRESS_FAIL, construct a Failure Information sub-TLV using the RAP Failure Code value contained in tTaReg.ingressFailureCode and the SystemId value of this Bridge, and then append it to the copied sub-TLV.
    6) For each VLAN in the determined list that is not yet present in the RedundancyCotrol sub-TLV, add a VLAN Context Information sub-TLV for that VLAN with a Failure Information sub-TLV that contains the value of the RAP Failure Code *MissingTalkerAnnounce* defined in Table 6-7 and the SystemId value of this Bridge.
c) If none of the originating TA's ingressStatus indicates TA_INGRESS_SUCCESS, perform the following:
    1) Add a Failure Information sub-TLV () to the TA. Use the SystemId value of this Bridge and the numerically smallest RAP Failure Code contained in the VLAN Context Information sub-TLV(s) in the RedundancyControl sub-TLV. Add the same Failure Information sub-TLV to each VLAN Context Information sub-TLV in the TA that currently contains no Failure Information sub-TLV.
d) Otherwise:

1) Call **setAccuLatencies**() and then **setNetworkTSpec**().
2) If checkReservationConstraints() returns an error, add a Failure Information sub-TLV () to the TA. Use the SystemId value of this Bridge and the numerically smallest RAP Failure Code contained in the VLAN Context Information sub-TLV(s) in the RedundancyControl sub-TLV. Add the same Failure Information sub-TLV to each VLAN Context Information sub-TLV in the TA that currently contains no Failure Information sub-TLV.

6.8.2.7 processLa()

This procedure processes a LA according to the egress port, it is called for, as follows:

If the stream is classified as multi-context, and no suitable TA registration as specified in 6.3.7.1 for the stream is known at this egress port, return FALSE.

Otherwise if no corresponding Talker Announcement exists on the egress port, return FALSE.

If a LA from a different source port already exists on the egress port, and a new LA is being propagated to the same port, the two LAs shall be merged in accordance with the merging policy defined for LA propagation. the defined listener attachment rules.

Otherwise return TRUE.

Note: Propagation is constrained to occur only within a defined MAP context ([Q]:10.3) or redundancy context (6.3.11.2). This constraint may be generalized as a prerequisite for MAP-based operations.

6.8.2.8 processLaDec()

This procedure processes a Listener Attach declaration, as follows:

If the ListenerAttachStatus of the LA declaration is already failed: do nothing and return.

If the ingressStatus of the corresponding TA Registration [6.7.5.1 d)] is failed: set the according ListenerAttachStatus error in the LA declaration.

6.8.2.9 setAccuLatencies()

This procedure determines the value of the maximum and minimum accumulated latency values for a Talker Announce declaration element (6.7.5.2), as follows:

Calculate the min and max latency according to the parameters in the corresponding RA-class and set the according values in the TA.

6.8.2.10 setNetworkTspec()

This procedures determines the value of the NetworkTSpec (6.3.9.2) for a Talker Announce declaration element (6.7.5.2), as follows:

Calculate and set the values in the NetworkTSpec (6.3.9.2) according to the parameters in the corresponding RA-class.

6.8.2.11 allocateResourcesNonMerged()

This procedure is invoked to create a reservation for a Listener Attach registration element for a stream that is not subject to stream merging (6.3.11.5).

The procedure takes all necessary actions, such as configuration of shaper, buffer size and FRER functions, to ensure the QoS required by the stream. The information about the characteristics of the stream is contained in both the Listener Attach registration and its associated Talker Announce declaration.

If all the required actions have been successfully carried out, the time measurement for reservationAge [item d) in 6.7.5.3] is started and the procedure returns a value of zero. If some of the required actions have failed, the procedure cancels the actions already taken, if any, and returns the value of the RAP Failure Code *ResourceAllocationFailed* defined in Table 6-7.

6.8.2.12 allocateResourcesMerged()

This procedure is invoked to create a new reservation or update an existing reservation for a Listener Attach registration element for a stream subject to stream merging (6.3.11.5) on a FRER-capable RAP Bridge's Port.

NOTE—The need for updating an existing reservation arises from dynamic changes in the presence or status of the propagated Talker Announce registration of each Member Stream at a stream merging port. For example, assuming a total of three Member Streams are expected to be merged, a reservation was initially made only for two of them propagated with the Announce Success status. The reservation needs to be adjusted later on when the third one is also propagated as Announce Success, or when one previously with Announce Success is changed to Announce Fail or not any more propagated to the stream merging port.

The procedure takes all necessary actions, such as configuration of shaper, buffer size and FRER functions, to ensure the QoS required by the stream. The information about the stream characteristics is contained in both the Listener Attach registration and its associated Talker Announce declaration. Note that a reservation created or updated by this procedure only allows forwarding and merging of each Member Stream forwarded to this Bridge Port according to the rules defined in propagateTa() (6.7.6.1) and whose ingressStatus indicates TA_INGRESS_SUCCESS.

If all the required actions have been successfully carried out, the time measurement for reservationAge [item d) in 6.7.5.3] is started (only when creating a new reservation) and the procedure returns a value of zero. If some of the required actions have failed, the procedure cancels the actions already taken, if any, and returns the value of the RAP Failure Code *ResourceAllocationFailed* defined in Table 6-7.

7. Resource Allocation Protocol (RAP) management

This subclause defines the following managed objects that allows administrative configuration of RAP:

a) RAP MAD Table (7.1)
b) RAP MAP Bridge Table (7.2)
c) RAP MAP Port Table (7.3)
d) RA Class Bridge Table (7.4)
e) RA Class Port Table (7.5)
f) RA Class Port Pair Table (7.6)
g) Priority Regeneration Override Table (7.7)
h) Redundancy Context Table (7.8)
i) Talker Announce Registration Port Table (7.9)
j) Talker Announce Declaration Port Table (7.10)
k) Listener Attach Registration Port Table (7.11)
l) Listener Attach Declaration Port Table (7.12)

7.1 RAP MAD Table

There is one RAP MAD Table per RAP MAD instance (6.6). The table contains a set of parameters that supports the operation of RAP MAD for an end station or Bridge Port, as detailed in Table 7-1.

Table 7-1—RAP MAD Table

Name	Data type	Operations supported[a]	References
rapPortEnabled	Boolean	RW	6.6.4.1

[a] R = read only access; RW = Read/Write access.

7.2 RAP MAP Bridge Table

There is one RAP MAP Bridge Table. The table contains a set of parameters that supports the operation of RAP for a Bridge as a whole, as detailed in Table 7-2.

Table 7-2—RAP MAP Bridge Table

Name	Data type	Operations supported[a]	References
frerCapable	Boolean	R	6.7.5.11
maxProcessingDelay	unsigned integer	R	6.7.5.12
minProcessingDelay	unsigned integer	R	6.7.5.13

[a] R = read only access; RW = Read/Write access.

7.3 RAP MAP Port Table

There is one RAP MAP Port Table per Bridge Port. The table contains a set of parameters that supports the operation of the RAP Propagator for a Bridge Port, as detailed in Table 7-3

Table 7-3—RAP MAP Port Table

Name	Data type	Operations supported[a]	References
streamDaPruningEnabled	Boolean	RW	6.7.5.7 a)
maxInterferingFrameSize	unsigned integer	R	6.7.5.7 b)
maxPropagationDelay	unsigned integer	R	6.7.5.7 c)
minPropagationDelay	unsigned integer	R	6.7.5.7 d)

[a] R = read only access; RW = Read/Write access.

7.4 RA Class Bridge Table

There is one RA Class Bridge Table per RAP MAD instance. Each table row contains a set of parameters that defines an RA class (6.3.2) for a Bridge as a whole, as detailed in Table 7-4.

Table 7-4—RA Class Bridge Table row elements

Name	Data type	Operations supported[a]	References
raClassId	unsigned integer [0..255]	RW	6.3.2.1
raClassPriority	unsigned integer [0..7]	RW	6.3.2.2
rtid	integer	RW	6.3.2.3
templatedDefinedData	octet string	RW	6.4.2.2.5

[a] R = read only access; RW = Read/Write access.

7.5 RA Class Port Table

There is one RA Class Port Table per Bridge Port for a RAP MAD instance. Each table row contains a set of parameters associated with an RA class configured in the RA Class Bridge Table (7.4) and a Bridge Port, as detailed in Table 7-5.

Table 7-5—RA Class Port Table row elements

Name	Data type	Operations supported[a]	References
raClassId	unsigned integer [0..255]	RW	6.7.5.8 a)
domainBoundaryStatus	Boolean	R	6.7.5.8 b)
maxBandwidth	unsigned integer	RW	6.7.5.8 c)
allocatedBandwidth	unsigned integer	R	6.7.5.8 d)

[a] R = read only access; RW = Read/Write access.

7.6 RA Class Port Pair Table

There is one RA Class Port Pair Table per reception transmission Port pair of a Bridge for a RAP Propagator (6.7). Each table row is associated with an RA class present in the RA Class Bridge Table (7.4) and contains a set of parameters for a pair of a reception Port and a transmission Port, as detailed in Table 7-6.

Table 7-6—RA Class Port Pair Table row elements

Name	Data type	Operations supported[a]	References
receptionPort	integer32	RW	6.7.5.9 a)
transmissionPort	integer32	RW	6.7.5.9 b)
raClassId	unsigned integer [0..255]	RW	6.7.5.9 c)
maxHopLatency	unsigned integer	RW	6.7.5.9 d)

[a] R = read only access; RW = Read/Write access.

7.7 RA Class Domain Boundary Priority Regeneration Table

There is one RA Class Domain Boundary Priority Regeneration Table per Bridge Port for a RAP MAD instance. The values contained in this table are used in the detection of RA class domains to control the Bridge's priority regeneration function ([Q]:6.9.4) on an RA class domain boundary port, as described in 6.3.2.4. Each table row contains a set of parameters, as detailed in Table 7-7.

Table 7-7—RA Class Domain Boundary Priority Regeneration Table row elements

Name	Data type	Operations supported[a]	References
receivedPriority	unsigned integer [0..7]	RW	7.7.1
regeneratedPriority	unsigned integer [0..7]	RW	7.7.2

[a] R = read only access; RW = Read/Write access.

7.7.1 receivedPriority

This parameter contains a received priority that is associated with an RA class configured in the RA Class Bridge Table (7.4).

7.7.2 regeneratedPriority

This parameter contains the regenerated priority for the given received priority contained in 7.7.1.

7.8 Redundancy Context Table

There is one RAP Redundancy Context Table per RAP MAD instance. Each table row contains a set of parameters that defines a Redundancy Context (6.3.11.3) supported by the Bridge, as detailed in Table 7-8.

Table 7-8—Redundancy Context Table row elements

Name	Data type	Operations supported[a]	References
redundancyContextId	unsigned integer [1..4094]	RW	6.3.11.2, 6.7.5.10
vlanContextList	a list of unsigned integer [1..4094]	RW	6.3.11.2, 6.7.5.10

[a] R = read only access; RW = Read/Write access.

7.8.1 vlanContextStatus

This parameter contains a Boolean value that indicates whether a VLAN Context Status sub-TLV (6.4.4.4) is contained in the Listener Attach attribute (TRUE) or not (FALSE).

7.8.2 vlanContextStatusValue

When vlanContextStatus (7.8.1) is TRUE, this parameter contains an octet string copied from the whole of the Value field of a VLAN Context Status sub-TLV (6.4.4.4) contained in the Listener Attach attribute.

7.9 Talker Announce Registration Port Table

There is one Talker Announce Registration Port Table per Bridge Port. Each table row in the table associated with a Port represents a Talker Announce registration [6.3.6 item b)] on that Port, and contains a set of parameters as detailed in Table 7-9. Rows in the table can be created, updated and deleted dynamically as Talker Announce registrations are created, updated and deleted on the Port.

Table 7-9—Talker Announce Registration Port Table row elements

Name	Data type	Operations supported[a]	References
streamId	octet string(size(8))	R	6.4.3.1
streamRank	unsigned integer [0..1]	R	6.4.3.2
accumulatedMaxLatency	unsigned integer	R	6.4.3.3
accumulatedMinLatency	unsigned integer	R	6.4.3.4
destinationMacAddress	MAC address	R	6.4.3.5.1
priority	unsigned integer [0..7]	R	6.4.3.5.2
vid	unsigned integer [1..4094]	R	6.4.3.5.3
talkerTokenBucketTSpec	Boolean	R	7.9.1
talkerTokenBucketTSpecValue	octet string	R	7.9.2
talkerMsrpTSpec	Boolean	R	7.9.3
talkerMsrpTSpecValue	octet string	R	7.9.4
networkTSpec	Boolean	R	7.9.5

Table 7-9—Talker Announce Registration Port Table row elements

Name	Data type	Operations supported[a]	References
networkTSpecValue	octet string	R	7.9.6
redundancyControl	Boolean	R	7.9.7
redundancyControlValue	octet string	R	7.9.8
failureInfo	Boolean	R	7.9.9
failureInfoValue	octet string	R	7.9.10
orgDefinedInfo	Boolean	R	7.9.11
orgDefinedInfoValue	octet string	R	7.9.12
invalid	Boolean	R	7.9.13
ingressBlocked	Boolean	R	7.9.14
ingressStatus	unsigned integer	R	6.7.5.1 d)

[a] R = read only access; RW = Read/Write access.

7.9.1 talkerTokenBucketTSpec

This parameter contains a Boolean value that indicates whether a Token Bucket TSpec sub-TLV (6.4.3.6) is contained in the TalkerTSpec field (6.4.3.6) of the Talker Announce attribute (TRUE) or not (FALSE).

7.9.2 talkerTokenBucketTSpecValue

When talkerTokenBucketTSpec (7.9.1) is TRUE, this parameter contains an octet string copied from the whole of the Value field of a Token Bucket TSpec sub-TLV (6.4.3.6) contained in the TalkerTSpec field (6.4.3.6) of the Talker Announce attribute.

When talkerTokenBucketTSpec is FALSE, this parameter contains the empty octet string.

7.9.3 talkerMsrpTSpec

This parameter contains a Boolean value that indicates whether an Interval-based TSpec sub-TLV (6.4.3.7) is contained in the TalkerTSpec field (6.4.3.6) of the Talker Announce attribute.

7.9.4 talkerMsrpTSpecValue

When talkerMsrpTSpec (7.9.3) is TRUE, this parameter contains an octet string copied from the whole of the Value field of an Interval-based TSpec sub-TLV (6.4.3.7) contained in the TalkerTSpec field (6.4.3.6) of the Talker Announce attribute.

When talkerMsrpTSpec is FALSE, this parameter contains the empty octet string.

7.9.5 networkTSpec

This parameter contains a Boolean value that indicates whether a Token Bucket TSpec sub-TLV (6.4.3.6) is contained in the NetworkTSpec field (6.4.3.9) of the Talker Announce attribute (TRUE) or not (FALSE).

7.9.6 networkTSpecValue

When networkTSpec (7.9.5) is TRUE, this parameter returns an octet string copied from the whole of the Value field of a Token Bucket TSpec sub-TLV (6.4.3.6) contained in the NetworkTSpec field (6.4.3.9) of the Talker Announce attribute.

When networkTSpec is FALSE, this parameter contains the empty octet string.

7.9.7 redundancyControl

This parameter contains a Boolean value that indicates whether a Redundancy Control sub-TLV (6.4.3.9) is contained in the Talker Announce attribute (TRUE) or not (FALSE).

7.9.8 redundancyControlValue

When redundancyControl (7.9.7) is TRUE, this parameter contains an octet string copied from the whole of the Value field of a Redundancy Control sub-TLV (6.4.3.9) contained in the Talker Announce attribute.

When redundancyControl is FALSE, this parameter contains the empty octet string.

7.9.9 failureInfo

This parameter contains a Boolean value that indicates whether a Failure Information sub-TLV (6.4.3.9.2) is contained in the Talker Announce attribute (TRUE) or not (FALSE).

7.9.10 failureInfoValue

When failureInfo (7.9.9) is TRUE, this parameter contains an octet string copied from the whole of the Value field of a Failure Information sub-TLV (6.4.3.9.2) contained in the Talker Announce attribute.

When failureInfo is FALSE, this parameter contains the empty octet string.

7.9.11 orgDefinedInfo

This parameter contains a Boolean value that indicates whether an Organizationally Defined sub-TLV (6.4.3.10) is contained in the Talker Announce attribute (TRUE) or not (FALSE).

7.9.12 orgDefinedInfoValue

When orgDefinedInfo (7.9.11) is TRUE, this parameter contains an octet string copied from the whole of the Value field of an Organizationally Defined sub-TLV (6.4.3.10) contained in the Talker Announce attribute.

When orgDefinedInfo is FALSE, this parameter contains the empty octet string.

7.9.13 invalid

A Boolean indicating whether the Talker Announce registration is invalid (TRUE) or not (FALSE), as determined by the isTaInvalid() procedure (6.6.9.2).

7.9.14 ingressBlocked

A Boolean indicating whether the Talker Announce registration is ingress blocked (TRUE) or not (FALSE), as determined by the isTaIngressBlocked() procedure (6.7.2.3).

7.10 Talker Announce Declaration Port Table

There is one Talker Announce Declaration Port Table per Bridge Port for a RAP MAD instance. Each table row in the Table associated with a Port represents a Talker Announce declaration [item a) in 6.7.5.2] on that Port, and contains each of the parameters as specified in Table 7-9 except the invalid and ingressBlocked parameters. Rows in the table can be created, updated and deleted dynamically as Talker Announce declarations are created, updated and deleted on the Port.

7.11 Listener Attach Registration Port Table

There is one Listener Attach Registration Port Table per Bridge Port for a RAP MAD instance. Each table row in the Table associated with a Port represents a Listener Attach registration [item b) in 6.7.5.3] on that Port and contains a set of parameters as detailed in Table 7-10. Rows in the table can be created, updated and deleted dynamically as the Listener Attach registrations are created, updated and deleted on the Port.

Table 7-10—Listener Attach Registration Port Table row elements

Name	Data type	Operations supported[a]	References
streamId	octet string(size(8))	R	6.4.4.1
vid	unsigned integer [1..4094]	R	6.4.4.2
listenerAttachStatus	Enumerated	R	6.4.4.3
vlanContextStatus	Boolean	R	7.8.1
vlanContexStatusValue	octet string	R	7.8.2
reservationAge	unsigned integer	R	6.7.5.3 d)
ingressStatus	unsigned integer	R	6.7.5.3 e)

[a] R = read only access; RW = Read/Write access.

7.12 Listener Attach Declaration Port Table

There is one Listener Attach Declaration Port Table per Bridge Port for a RAP MAD instance. Each table row in the Table associated with a Port represents a Listener Attach declaration [item a) in 6.7.5.4] on that Port and contains each of the parameters as specified in Table 7-10 except the reservationAge and ingressStatus parameters. Rows in the table can be created, updated and deleted dynamically as Listener Attach declarations are created, updated and deleted on the Port.

8. YANG data model for RAP

YANG (IETF RFC 7950) is a data modeling language used to model configuration data and state data for remote network management protocols. YANG-based remote network management protocols include the Network Configuration Protocol (NETCONF) (IETF RFC 6242) and RESTCONF (IETF RFC 8040). Each remote network management protocol uses a specific encoding on the wire, such as XML or JSON. A YANG module specifies the organization and rules for the management data, and a mapping from YANG to the specific encoding enables the data to be understood correctly by both client and server.

This clause specifies the YANG data model for RAP.

This clause:

a) Introduces the organization of the data models (8.1)
b) Shows the relationship with other standards (8.2)
c) Provides an overview of the hierarchy of the data models using a representation similar to the Unified Modeling Language (UML) (8.3)
d) Summarizes the structure of the YANG data model (8.4)
e) Reviews security considerations (8.5)
f) Provides a schema tree as an overview of the YANG module (8.6)
g) Specifies the YANG module (8.7)

8.1 The YANG framework

The YANG framework applies hierarchy in the following areas:

a) The Uniform Resource Name (URN), as specified in IEEE Std 802d-2017.
b) The YANG objects form a hierarchy of configuration and operational data structures that define the YANG model.

Clause 7 specifies the information model for management of this standard. The data model for a specific management mechanism is derived from the information model. Since YANG-based protocols are an example of a management mechanism, the YANG data model of this clause is derived from Clause 7.

8.2 Relationship to other YANG modules

The RAP YANG model augments the VLAN Bridge component model as defined in [Q].

8.3 RAP YANG model

This clause uses a UML-like representation to provide an overview of the hierarchy of the RAP YANG data model.

For all figures, data that is imported from outside this standard is shown in white, and data in augments of those YANG modules is shown in gray.

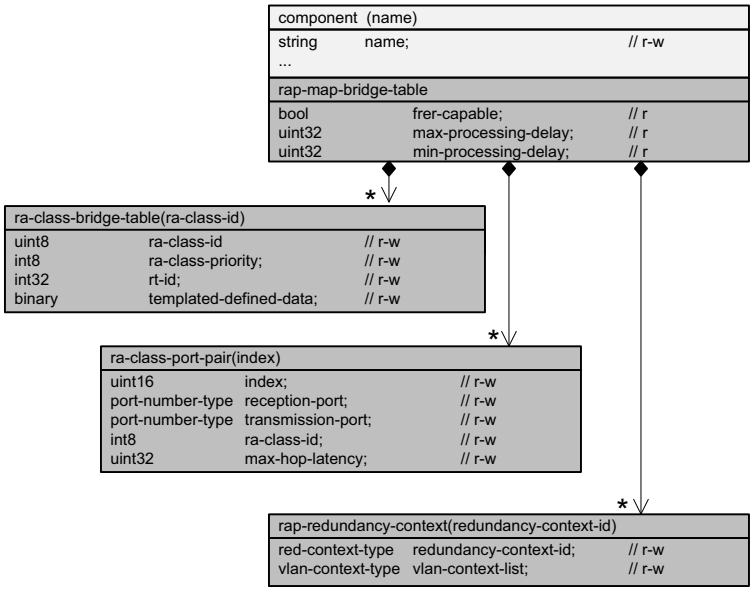

Figure 8-1—Bridge component augmentation

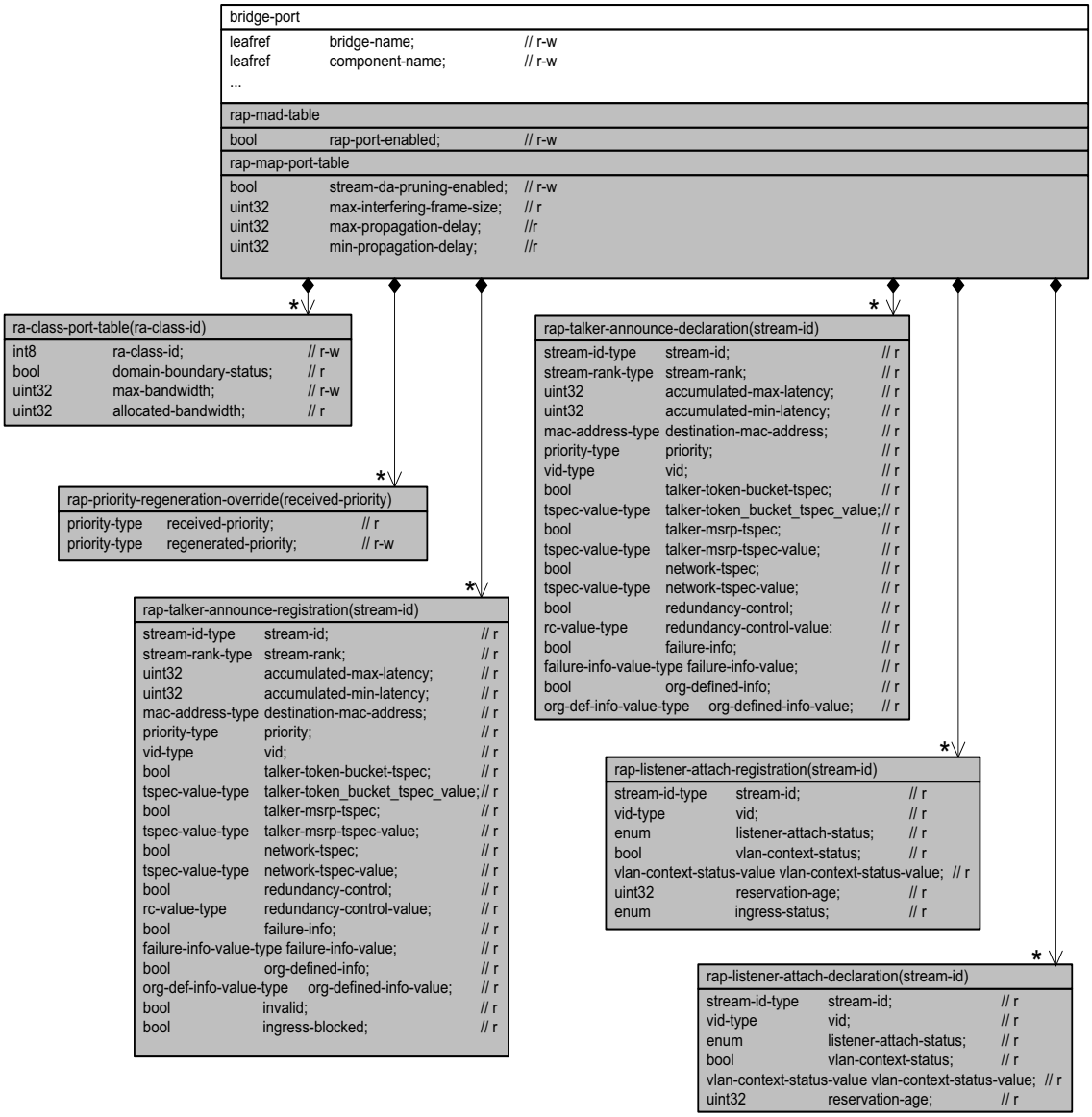

Figure 8-2—Bridge port augmentation

8.4 Structure of the YANG model

A bridge implementing RAP implements the YANG modules in Table 8-1.

In the YANG module definitions, if any discrepancy between the “description” text and the corresponding definition in any other part of this standard occurs, the definitions outside this clause take precedence.

Table 8-1—RAP YANG modules

YANG module
ieee802-dot1dd-rap
ieee802-dot1dd-rap-bridge

8.5 Security Considerations

The YANG module specified in this clause is designed to be accessed via a network configuration protocol, such as NETCONF (IETF RFC 6242 [B13a]) and RESTCONF (IETF RFC 8040). NETCONF and RESTCONF protocols provide the means to secure communication between client and server, using secure transport layers such as Secure Shell (SSH) (IETF RFC 6242) and Transport Layer Security (TLS) (IETF RFC 7589).

It is the responsibility of a system's implementor and administrator to ensure that the protocol entities in the system that support NETCONF, and any other remote configuration protocols that make use of these YANG modules, are properly configured to allow access only to those principals (users) that have legitimate rights to read or write data nodes. This standard does not specify how the credentials of those users are to be stored or validated.

8.6 YANG data scheme tree definitions

The schema tree in this clause is provided as an overview of the RAP YANG module. The symbols and their meaning are specified in YANG Tree Diagrams (IETF RFC 8340).

8.6.1 Tree diagram for ieee802-dot1dd-rap-bridge.yang

```
module: ieee802-dot1dd-rap-bridge

  augment /dot1q:bridges/dot1q:bridge/dot1q:component:
    +--rw rap-map-bridge-table
    |  +--ro frer-capable?           boolean
    |  +--ro max-processing-delay?   uint32
    |  +--ro min-processing-delay?   uint32
    +--rw ra-class-bridge-table* [ra-class-id]
    |  +--rw ra-class-id               uint8
    |  +--rw ra-class-priority?        int8
    |  +--rw rt-id?                    int32
    |  +--rw templated-defined-data?   binary
    +--rw ra-class-port-pair* [index]
    |  +--rw index                 uint16
    |  +--rw reception-port?       dot1qtypes:port-number-type
    |  +--rw transmission-port?    dot1qtypes:port-number-type
    |  +--rw ra-class-id?          int8
    |  +--rw max-hop-latency?      uint32
    +--rw rap-redundancy-context* [redundancy-context-id]
       +--rw redundancy-context-id    uint16
       +--rw vlan-context-list* [vlan-context]
          +--rw vlan-context    uint16
  augment /if:interfaces/if:interface/dot1q:bridge-port:
    +--rw rap-mad-table
    |  +--rw rap-port-enabled?   boolean
    +--rw rap-map-port-table
    |  +--rw stream-da-pruning-enabled?    boolean
    |  +--ro max-interfering-frame-size?   uint32
    |  +--ro max-propagation-delay?        uint32
    |  +--ro min-propagation-delay?        uint32
    +--rw ra-class-port-table* [ra-class-id]
    |  +--rw ra-class-id               uint8
    |  +--ro domain-boundary-status?   boolean
```

```
   |  +--rw max-bandwidth?            uint32
   |  +--ro allocated-bandwidth?      uint32
   +--rw rap-priority-regeneration-override* [received-priority]
   |  +--rw received-priority       dot1qtypes:priority-type
   |  +--rw regenerated-priority?   dot1qtypes:priority-type
   +--ro rap-talker-announce-registration* [stream-id vid]
   |  +--ro stream-id                          tsn:stream-id-type
   |  +--ro stream-rank?                       uint8
   |  +--ro accumulated-max-latency?           uint32
   |  +--ro accumulated-min-latency?           uint32
   |  +--ro destination-mac-address?           ieeetypes:mac-address
   |  +--ro priority?                          dot1qtypes:priority-type
   |  +--ro vid                                uint16
   |  +--ro talker-token-bucket-tspec?         boolean
   |  +--ro talker-token_bucket_tspec_value?   binary
   |  +--ro talker-msrp-tspec?                 boolean
   |  +--ro talker-msrp-tspec-value?           binary
   |  +--ro network-tspec?                     boolean
   |  +--ro network-tspec-value?               binary
   |  +--ro redundancy-control?                boolean
   |  +--ro redundancy-control-value?          binary
   |  +--ro failure-info?                      boolean
   |  +--ro failure-info-value?                binary
   |  +--ro org-defined-info?                  boolean
   |  +--ro org-defined-info-value?            binary
   |  +--ro invalid?                           boolean
   |  +--ro ingress-blocked?                   boolean
   +--ro rap-talker-announce-declaration* [stream-id vid]
   |  +--ro stream-id                          tsn:stream-id-type
   |  +--ro stream-rank?                       uint8
   |  +--ro accumulated-max-latency?           uint32
   |  +--ro accumulated-min-latency?           uint32
   |  +--ro destination-mac-address?           ieeetypes:mac-address
   |  +--ro priority?                          dot1qtypes:priority-type
   |  +--ro vid                                uint16
   |  +--ro talker-token-bucket-tspec?         boolean
   |  +--ro talker-token_bucket_tspec_value?   binary
   |  +--ro talker-msrp-tspec?                 boolean
   |  +--ro talker-msrp-tspec-value?           binary
   |  +--ro network-tspec?                     boolean
   |  +--ro network-tspec-value?               binary
   |  +--ro redundancy-control?                boolean
   |  +--ro redundancy-control-value?          binary
   |  +--ro failure-info?                      boolean
   |  +--ro failure-info-value?                binary
   |  +--ro org-defined-info?                  boolean
   |  +--ro org-defined-info-value?            binary
   |  +--ro invalid?                           boolean
   |  +--ro ingress-blocked?                   boolean
   +--ro rap-listener-attach-registration* [stream-id vid]
   |  +--ro stream-id                    tsn:stream-id-type
   |  +--ro vid                          uint16
   |  +--ro listener-attach-status?      enumeration
   |  +--ro vlan-context-status?         boolean
   |  +--ro vlan-context-status-value?   binary
   |  +--ro reservation-age?             uint32
   |  +--ro ingress-status?              enumeration
   +--ro rap-listener-attach-declaration* [stream-id vid]
      +--ro stream-id                    tsn:stream-id-type
      +--ro vid                          uint16
      +--ro listener-attach-status?      enumeration
      +--ro vlan-context-status?         boolean
      +--ro vlan-context-status-value?   binary
```

8.7 YANG module ieee802-dot1dd-rap

```
module ieee802-dot1dd-rap {
  yang-version 1.1;
  namespace "urn:ieee:std:802.1DD:yang:ieee802-dot1dd-rap";
```

```
prefix rap;

import ieee802-dot1q-types {
  prefix dot1qtypes;
}
import ieee802-dot1q-tsn-types {
  prefix tsn;
}
import ieee802-types {
  prefix ieeetypes;
}

organization
  "IEEE 802.1 Working Group";
contact
  "WG-URL: http://www.ieee802.org/1/
   WG-EMail: stds-802-1-l@ieee.org

   Contact: IEEE 802.1 Working Group Chair
   Postal: C/O IEEE 802.1 Working Group
           IEEE Standards Association
           445 Hoes Lane
           Piscataway, NJ 08854
           USA

   E-mail: stds-802-1-chairs@ieee.org";
description
  "This module provides management of 802.1Q Bridge components that
   support the Resource Allocation Protocol (RAP).

   Copyright (C) IEEE (202x).

   This version of this YANG module is part of IEEE Std 802.1DD;
   see the standard itself for full legal notices.";

revision 2026-04-20 {
  description
    "Published as part of IEEE Std 802.1DD-202x.

     The following reference statement identifies each referenced
     IEEE Standard as updated by applicable amendments.";
  reference
    "IEEE Std 802.1DD Resource Allocation Protocol:
     IEEE Std 802.1Q Bridges and Bridged Networks:
     IEEE Std 802.1Q-2022, IEEE Std 802.1Qcz-2023,
     IEEE Std 802.1Qcw-2023, IEEE Std 802.1Qcj-2023,
     IEEE Std 802.1Qdj-2024, IEEE Std 802.1Qdx-2024,
     IEEE Std 802.1Qdy-2024.";
}

grouping rap-talker-announce {
  description
    "Grouping for RAP Talker Announce";
  leaf stream-id {
    type tsn:stream-id-type;
    description
      "An 8-octet field encoding the StreamID element as specified in
       46.2.3.1 of IEEE Std 802.1Q.";
    reference
      "6.4.3.1 of IEEE Std 802.1DD";
  }
  leaf stream-rank {
    type uint8;
    description
      "A 1-octet field encoding a Rank value as specified in
       46.2.3.2.1 of IEEE Std 802.1Q.";
    reference
      "6.4.3.2 of IEEE Std 802.1DD";
  }
  leaf accumulated-max-latency {
    type uint32;
    description
```

```
        "A 4-octet field encoding the AccumulatedLatency element as
         specified in 46.2.5.2 of IEEE Std 802.1Q.";
      reference
        "6.4.3.3 of IEEE Std 802.1DD";
    }
    leaf accumulated-min-latency {
      type uint32;
      description
        "A 4-octet unsigned integer, indicating the minimum latency, in
         nanoseconds, that a single frame of the stream can encounter when
         transmitted from the Talker along a given path to the Port
         declaring this attribute.";
      reference
        "6.4.3.4 of IEEE Std 802.1DD";
    }
    leaf destination-mac-address {
      type ieeetypes:mac-address;
      description
        "A 6-octet destination MAC address of the data frames of the
         stream.";
      reference
        "6.4.3.5.1 of IEEE Std 802.1DD";
    }
    leaf priority {
      type dot1qtypes:priority-type;
      description
        "A 3-bit unsigned integer, indicating the priority to be encoded in
         the PCP field of the VLAN tag, with which the data frames of the
         stream are tagged. This priority value is used by each receiving
         Bridge to associate the stream to a local RA class of the same
         priority.";
      reference
        "6.4.3.5.2 of IEEE Std 802.1DD";
    }
    leaf vid {
      type uint16 {
        range "1..4094";
      }
      description
        "A 12-bit VID. The semantics of this field is dependent on the type
         of a Talker Announcement in which the Talker Announce attribute is
         used, as follows:
         a) In the case of a Single-Context Talker
         Announcement, this field indicates a VID to be encoded in the VLAN
         tag with which the data frames of the stream are tagged, and also
         a single VLAN Context used by the Talker Announce attribute.
         b) In the case of a Multi-Context Talker Announcement, this field
         is set to the numerically smallest VlanContextId value contained
         in a VLAN Context Information sub-TLV.";
      reference
        "6.4.3.5.3 of IEEE Std 802.1DD";
    }
    leaf talker-token-bucket-tspec {
      type boolean;
      description
        "This parameter returns a Boolean value that indicates whether a
         Token Bucket TSpec sub-TLV is contained in the TalkerTSpec field
         of the Talker Announce attribute (TRUE) or not (FALSE).";
      reference
        "7.9.1 of IEEE Std 802.1DD";
    }
    leaf talker-token_bucket_tspec_value {
      type binary;
      description
        "When talkerTokenBucketTSpec is TRUE, this parameter returns an
         octet string that is copied from the whole of the Value field of a
         Token Bucket TSpec sub-TLV contained in the TalkerTSpec field of
         the Talker Announce attribute.";
      reference
        "7.9.2 of IEEE Std 802.1DD";
    }
    leaf talker-msrp-tspec {
```

```
      type boolean;
      description
        "This parameter returns a Boolean value that indicates whether a
         MSRP TSpec sub-TLV is contained in the TalkerTSpec field of the
         Talker Announce attribute.";
      reference
        "7.9.3 of IEEE Std 802.1DD";
    }
    leaf talker-msrp-tspec-value {
      type binary;
      description
        "When talkerMsrpTSpec is TRUE, this parameter returns an octet
         string that is copied from the whole of the Value field of a MSRP
         TSpec sub-TLV contained in the TalkerTSpec field of the Talker
         Announce attribute.";
      reference
        "7.9.4 of IEEE Std 802.1DD";
    }
    leaf network-tspec {
      type boolean;
      description
        "This parameter returns a Boolean value that indicates whether a
         Token Bucket TSpec sub-TLV is contained in the NetworkTSpec field
         of the Talker Announce attribute (TRUE) or not (FALSE).";
      reference
        "7.9.5 of IEEE Std 802.1DD";
    }
    leaf network-tspec-value {
      type binary;
      description
        "When networkTSpec is TRUE, this parameter returns an octet string
         that is copied from the whole of the Value field of a Token Bucket
         TSpec sub-TLV contained in the NetworkTSpec field of the Talker
         Announce attribute.";
      reference
        "7.9.6 of IEEE Std 802.1DD";
    }
    leaf redundancy-control {
      type boolean;
      description
        "This parameter returns a Boolean value that indicates whether a
         Redundancy Control sub-TLV is contained in the Talker Announce
         attribute (TRUE) or not (FALSE).";
      reference
        "7.9.7 of IEEE Std 802.1DD";
    }
    leaf redundancy-control-value {
      type binary;
      description
        "When redundancyControl is TRUE, this parameter returns an octet
         string that is copied from the whole of the Value field of a
         Redundancy Control sub-TLV contained in the Talker Announce
         attribute.";
      reference
        "7.9.8 of IEEE Std 802.1DD";
    }
    leaf failure-info {
      type boolean;
      description
        "This parameter returns a Boolean value that indicates whether a
         Failure Information sub-TLV is contained in the Talker Announce
         attribute (TRUE) or not (FALSE).";
      reference
        "7.9.9 of IEEE Std 802.1DD";
    }
    leaf failure-info-value {
      type binary;
      description
        "When failureInfo is TRUE, this parameter returns an octet string
         that is copied from the whole of the Value field of a Failure
         Information sub-TLV contained in the Talker Announce attribute.";
      reference
```

```
        "7.9.10 of IEEE Std 802.1DD";
    }
    leaf org-defined-info {
      type boolean;
      description
        "This parameter returns a Boolean value that indicates whether an
         Organizationally Defined sub-TLV is contained in the Talker
         Announce attribute (TRUE) or not (FALSE).";
      reference
        "7.9.11 of IEEE Std 802.1DD";
    }
    leaf org-defined-info-value {
      type binary;
      description
        "When orgDefinedInfo is TRUE, this parameter returns an octet
         string that is copied from the whole of the Value field of an
         Organizationally Defined sub-TLV contained in the Talker Announce
         attribute.";
      reference
        "7.9.12 of IEEE Std 802.1DD";
    }
    leaf invalid {
      type boolean;
      description
        "A Boolean indicating whether the Talker Announce registration is
         invalid (TRUE) or not (FALSE), as determined by the isTaInvalid()
         procedure (6.8.5.3)";
      reference
        "item c) in 6.8.4.1 of IEEE Std 802.1DD";
    }
    leaf ingress-blocked {
      type boolean;
      description
        "A Boolean indicating whether the Talker Announce registration is
         ingress blocked (TRUE) or not (FALSE), as determined by the
         isTaIngressBlocked() procedure (6.8.5.4)";
      reference
        "item d) in 6.8.4.1 of IEEE Std 802.1DD";
    }
  }

  grouping rap-listener-attach {
    description
      "Grouping for RAP Listener Attachment";
    leaf stream-id {
      type tsn:stream-id-type;
      description
        "An 8-octet field encoding the StreamID element as specified in
         46.2.3.1 of IEEE Std 802.1Q.";
      reference
        "6.4.4.1 of IEEE Std 802.1D";
    }
    leaf vid {
      type uint16 {
        range "1..4094";
      }
      description
        "A 12-bit integer, indicating a VID to be encoded in the VLAN tag
         with which the data frames of the stream are tagged.";
      reference
        "6.4.4.2 of IEEE Std 802.1DD";
    }
    leaf listener-attach-status {
      type enumeration {
        enum attach-ready {
          value 1;
          description
            "listener-attach-status is ATTACH_READY";
        }
        enum attach-fail {
          value 2;
          description
```

```
            "listener-attach-status is ATTACH_FAIL";
        }
        enum attach-partial-fail {
          value 3;
          description
            "listener-attach-status is ATTACH_PARTIAL_FAIL";
        }
      }
      description
        "An enumeration indicating the listener attach status";
      reference
        "6.4.4.3 of IEEE Std 802.1DD";
    }
    leaf vlan-context-status {
      type boolean;
      description
        "This parameter returns a Boolean value that indicates whether a
         VLAN Context Status sub-TLV is contained in the Listener Attach
         attribute (TRUE) or not (FALSE).";
      reference
        "7.11.1 of IEEE Std 802.1DD";
    }
    leaf vlan-context-status-value {
      type binary;
      description
        "When vlanContextStatus is TRUE, this parameter returns an octet
         string that is copied from the whole of the Value field of a VLAN
         Context Status sub-TLV contained in the Listener Attach
         attribute.";
      reference
        "6.4.4.4 of IEEE Std 802.1DD";
    }
  }

  grouping rap-component-parameters {
    description
      "Augment Bridge with RAP configuration.";
    reference
      "6 of IEEE Std 802.1DD.";
    container rap-map-bridge-table {
      description
        "Container RAP MAP Bridge Table";
      leaf frer-capable {
        type boolean;
        config false;
        description
          "A Boolean value, indicating whether the Bridge is a
           FRER-capable Bridge (TRUE) or not (FALSE).";
        reference
          "6.7.6.11 of IEEE Std 802.1DD.";
      }
      leaf max-processing-delay {
        type uint32;
        config false;
        description
          "An unsigned integer, indicating the maximum delay, in
           nanoseconds, that a frame can experience during the
           forwarding process of the Bridge until it is placed into an
           outbound queue.";
        reference
          "6.7.6.12 of IEEE Std 802.1DD.";
      }
      leaf min-processing-delay {
        type uint32;
        config false;
        description
          "An unsigned integer, indicating the minimum delay, in
           nanoseconds, that a frame can experience during the
           forwarding process of the Bridge until it is placed into an
           outbound queue.";
        reference
          "6.7.6.13 of IEEE Std 802.1DD.";
```

```
    }
  }
  list ra-class-bridge-table {
    key "ra-class-id";
    description
      "Each table row contains a set of parameters that defines a
       single RA class";
    leaf ra-class-id {
      type uint8 {
        range "0 .. 255";
      }
      description
        "The RA class ID is an integer in the range of 0 through 255
         that identifies an RA class supported by an end station or
         Bridge.";
      reference
        "6.3.2.1 of IEEE Std 802.1DD.";
    }
    leaf ra-class-priority {
      type int8 {
        range "0 .. 7";
      }
      description
        "Each RA class supported by an end station or Bridge is
         associated with a unique priority value in the range 0
         through 7, termed RA class priority. The RA class priority
         indicates the received priority of the frames which are to
         be mapped to the traffic class(es) with which that RA class
         is associated.";
      reference
        "6.3.2.2 of IEEE Std 802.1DD.";
    }
    leaf rt-id {
      type int32;
      description
        "An RA class template is identified by an RA Class Template
         Identifier (RTID), which encodes a 3-octet OUI or CID value
         identifying the organization that defines that template,
         followed by a 1-octet index allocated by that
         organization.";
      reference
        "6.3.2.3 of IEEE Std 802.1DD.";
    }
    leaf templated-defined-data {
      type binary;
      description
        "The encoding of this field and the semantics associated
         with its values if any, is specific to the RA class
         template identified by the value contained in rt-id";
      reference
        "6.4.2.2.5 of IEEE Std 802.1DD.";
    }
  }
  list ra-class-port-pair {
    key "index";
    description
      "Each table row corresponds to an RA class configured in the
       RA Class Bridge Table and contains a set of parameters
       associated with a reception Port and a transmission Port";
    leaf index {
      type uint16;
      description
        "The index for the list";
    }
    leaf reception-port {
      type dot1qtypes:port-number-type;
      description
        "The port number of the associated reception Port.";
      reference
        "Item a) in 6.7.6.9 of IEEE Std 802.1DD.";
    }
    leaf transmission-port {
```

```
        type dot1qtypes:port-number-type;
        description
          "The port number of the associated transmission Port.";
        reference
          "Item b) in 6.7.6.9 of IEEE Std 802.1DD.";
      }
      leaf ra-class-id {
        type int8;
        description
          "The RA class ID of the associated local RA class. ";
        reference
          "Item c) in 6.7.6.9 of IEEE Std 802.1DD.";
      }
      leaf max-hop-latency {
        type uint32;
        description
          "An unsigned integer, containing the administratively
           configured maximum latency value, in nanoseconds, that is
           used as an upper bound latency in the latency constraint
           imposed on each stream reserved in the RA class, received
           on the reception Port, and transmitted on the transmission
           Port. The latency is measured from a point located in an
           upstream station connected via a LAN to the reception Port,
           to a point located in the transmission Port, where the
           exact measurement points are specific to and defined by the
           RA class template being used by the RA class.";
        reference
          "Item d) in 6.7.6.9 of IEEE Std 802.1DD.";
      }
    }
    list rap-redundancy-context {
      key "redundancy-context-id";
      description
        "Each table row contains a set of parameters associated with a
         Redundancy Context supported by the Bridge.";
      leaf redundancy-context-id {
        type uint16 {
          range "1..4096";
        }
        description
          "The Redundancy Context ID of the Redundancy Context";
        reference
          "Item a) in 6.7.6.10 of IEEE Std 802.1DD.";
      }
      list vlan-context-list {
        key "vlan-context";
        description
          "The list of VLAN Context IDs associated with the Redundancy
                   Context.";
        reference
          "Item b) in 6.7.6.10 of IEEE Std 802.1DD.";
        leaf vlan-context {
          type uint16 {
            range "1..4096";
          }
          description
            "VLAN Context ID";
        }
      }
    }
  }

  grouping rap-port-parameters {
    description
      "Augment Bridge Port with RAP configuration";
    reference
      "6 of IEEE Std 802.1DD.";
    container rap-mad-table {
      description
        "Container RAP MAD table";
      leaf rap-port-enabled {
        type boolean;
```

```
      description
        "A Boolean variable indicating whether RAP MAD is enabled for this Port.";
      reference
        "6.6.4.1 of IEEE Std 802.1DD.";
    }
  }
  container rap-map-port-table {
    description
      "Container RAP MAP Port Table";
    leaf stream-da-pruning-enabled {
      type boolean;
      description
        "A Boolean indicating whether Stream DA Pruning (51.3.4.1.2 of IEEE 802.1Q)
         is administratively enabled (TRUE) or disabled (FALSE) on the
         Port.";
      reference
        "Item a) in 6.7.6.7 of IEEE Std 802.1DD.";
    }
    leaf max-interfering-frame-size {
      type uint32;
      config false;
      description
        "An unsigned integer, indicating the maximum frame size, in
         bytes, including media-dependent overhead (12.4.2.2 of IEEE 802.1Q), that is
         allowed to be transmitted through the Port. The value of this
         parameter is determined by the operation of the underlying
         MAC Service.";
      reference
        "Item b) in 6.7.6.7 of IEEE Std 802.1DD.";
    }
    leaf max-propagation-delay {
      type uint32;
      config false;
      description
        "An unsigned integer, indicating the maximum latency, in
         nanoseconds, a frame can experience when transmitted from the
         underlying physical medium on the Port to a reception port
         connected via a LAN to the Port.";
      reference
        "Item c) in 6.7.6.7 of IEEE Std 802.1DD.";
    }
    leaf min-propagation-delay {
      type uint32;
      config false;
      description
        "An unsigned integer, indicating the minimum latency, in
         nanoseconds, a frame can experience when transmitted from the
         underlying physical medium on the Port to a reception port
         connected via a LAN to the Port.";
      reference
        "Item d) in 6.7.6.7 of IEEE Std 802.1DD.";
    }
  }
  list ra-class-port-table {
    key "ra-class-id";
    description
      "Each table row contains a set of parameters that defines a
       single RA class";
    leaf ra-class-id {
      type uint8 {
        range "0 .. 255";
      }
      description
        "The RA class ID of the associated local RA class.";
      reference
        "Item a) in 6.7.6.8 of IEEE Std 802.1DD.";
    }
    leaf domain-boundary-status {
      type boolean;
      config false;
      description
        "A Boolean indicating whether the Port is a domain boundary
```

```
          port for the RA class (TRUE) or not (FALSE).";
        reference
          "Item b) in 6.7.6.8 of IEEE Std 802.1DD.";
      }
      leaf max-bandwidth {
        type uint32;
        description
          "An unsigned integer, indicating the maximum amount of
           bandwidth that can be allocated to the streams reserved in
           the RA class on the Port. The bandwidth value is represented
           as a percentage of the portTransmitRate value on that Port
           and expressed as a fixed-point number scaled by a factor of
           1,000,000; i.e., 100,000,000 (the maximum value)represents
           100%.";
        reference
          "Item c) in 6.7.6.8 of IEEE Std 802.1DD.";
      }
      leaf allocated-bandwidth {
        type uint32;
        config false;
        description
          "An unsigned integer, indicating the amount of bandwidth that has
           been allocated to the streams reserved in the RA class on the
           Port. The bandwidth value is represented as a percentage of the
           portTransmitRate value on that Port and expressed as a fixed-
           point number scaled by a factor of 1,000,000; i.e., 100,000,000
           (the maximum value) represents 100%.";
        reference
          "Item d) in 6.7.6.8 of IEEE Std 802.1DD.";
      }
    }
    list rap-priority-regeneration-override {
      key "received-priority";
      description
        "Each table row contains a set of parameters for a received
         priority value that is associated with an RA class ";
      leaf received-priority {
        type dot1qtypes:priority-type;
        description
          "Received priority value.";
        reference
          "7.7.1 of IEEE Std 802.1DD";
      }
      leaf regenerated-priority {
        type dot1qtypes:priority-type;
        description
          "Priority regeneration value.";
        reference
          "7.7.2 of IEEE Std 802.1DD";
      }
    }
    list rap-talker-announce-registration {
      key "stream-id vid";
      config false;
      description
        "Each table row in the table associated with a Port corresponds
         to a registration of the Talker Announce attribute on that Port,
         and contains a set of parameters";
      uses rap-talker-announce;
    }
    list rap-talker-announce-declaration {
      key "stream-id vid";
      config false;
      description
        "Each table row in the Table associated with a Port corresponds
         to a declaration of the Talker Announce attribute on that Port,
         and contains the same set of parameters as in the Talker Announce
         Registration";
      uses rap-talker-announce;
    }
    list rap-listener-attach-registration {
      key "stream-id vid";
```

```
      config false;
      description
        "Each table row in the Table associated with a Port corresponds to
         a registration of the Listener Attach attribute on that Port and
         contains a set of parameters";
      uses rap-listener-attach;
      leaf reservation-age {
        type uint32;
        description
          "A 32-bit unsigned integer, indicating the time, in seconds,
           since a reservation associated with the Listener Attach
           registration was successfully made, and set to zero when the
           reservation is removed.";
        reference
          "Item e) in 6.8.4.3 of IEEE Std 802.1DD";
      }
      leaf ingress-status {
        type enumeration {
          enum not-propagated {
            value 0;
            description
              "ingress-status is NOT_PROPAGATED";
          }
          enum attach-ready {
            value 1;
            description
              "ingress-status is ATTACH_READY";
          }
          enum attach-fail {
            value 2;
            description
              "ingress-status is ATTACH_FAIL";
          }
          enum attach-partial-fail {
            value 3;
            description
              "ingress-status is ATTACH_PARTIAL_FAIL";
          }
        }
        description
          "The ingress status of the Listener Attach registration.";
        reference
          "item f) in 6.8.4.3 of IEEE Std 802.1DD";
      }
    }
    list rap-listener-attach-declaration {
      key "stream-id vid";
      config false;
      description
        "Each table row in the Table associated with a Port corresponds to
         a declaration of the Listener Attach attribute on that Port and
         contains the same set of parameters except the reservationAge
         parameter as in the Listener Attach Registration";
      uses rap-listener-attach;
    }
  }
}
```

8.8 YANG module ieee802-dot1dd-rap.bridge

```
module ieee802-dot1dd-rap-bridge {
  yang-version 1.1;
  namespace “urn:ieee:std:802.1Q:yang:ieee802-dot1dd-rap-bridge”;
  prefix rap-bridge;

  import ietf-interfaces {
    prefix if;
  }
  import ieee802-dot1q-bridge {
    prefix dot1q;
```

```
}
import ieee802-dot1dd-rap {
  prefix rap;
}

organization
  “IEEE 802.1 Working Group”;
contact
  “WG-URL: http://www.ieee802.org/1/
   WG-EMail: stds-802-1-l@ieee.org

   Contact: IEEE 802.1 Working Group Chair
   Postal: C/O IEEE 802.1 Working Group
          IEEE Standards Association
          445 Hoes Lane
          Piscataway, NJ 08854
          USA

   E-mail: stds-802-1-chairs@ieee.org”;
description
  “This module provides management of 802.1Q Bridge components that
   support the Resource Allocation Protocol (RAP).

   Copyright (C) IEEE (2025).

   This version of this YANG module is part of IEEE Std 802.1DD; see the
   standard itself for full legal notices.”;

revision 2026-04-20 {
  description
    “Published as part of IEEE Std 802.1DD-20xx.

     The following reference statement identifies each referenced IEEE
     Standard as updated by applicable amendments.”;
  reference
    “IEEE Std 802.1DD Resource Allocation Protocol:
     IEEE Std 802.1Q Bridges and Bridged Networks:
     IEEE Std 802.1Q-2022, IEEE Std 802.1Qcz-2023,
     IEEE Std 802.1Qcw-2023, IEEE Std 802.1Qcj-2023,
     IEEE Std 802.1Qdj-2024, IEEE Std 802.1Qdx-2024,
     IEEE Std 802.1Qdy-2024.”;
}

 augment “/dot1q:bridges/dot1q:bridge/dot1q:component” {
  description
    “Augment Bridge Port with RAP configuration”;
  reference
    “???.”;
  uses rap:rap-component-parameters;
}
augment “/if:interfaces/if:interface/dot1q:bridge-port” {
  description
    “Augment Bridge Port with RAP configuration”;
  reference
    “6 of IEEE Std 802.1DD.”;
  uses rap:rap-port-parameters;
}
}
```

Annex A

(normative)

PICS proforma—<RAP implementations>[11]

A.1 Introduction

The supplier of a protocol implementation that is claimed to conform to this standard shall complete the following Protocol Implementation Conformance Statement (PICS) proforma.

A completed PICS proforma is the PICS for the implementation in question. The PICS is a statement of which capabilities and options of the protocol have been implemented. The PICS can have a number of uses, including use

a) By the protocol implementer, as a checklist to reduce the risk of failure to conform to the standard through oversight.
b) By the supplier and acquirer—or potential acquirer—of the implementation, as a detailed indication of the capabilities of the implementation, stated relative to the common basis for understanding provided by the standard PICS proforma.
c) By the user—or potential user—of the implementation, as a basis for initially checking the possibility of interworking with another implementation (note that, while interworking can never be guaranteed, failure to interwork can often be predicted from incompatible PICSs).
d) By a protocol tester, as the basis for selecting appropriate tests against which to assess the claim for conformance of the implementation.

A.2 Abbreviations and special symbols

A.2.1 Status symbols

M	mandatory
O	optional
O.n	optional, but support of at least one of the group of options labeled by the same numeral n is required
X	prohibited
pred:	conditional-item symbol, including predicate identification: see A.3.4
¬	logical negation, applied to a conditional item’s predicate

A.2.2 General abbreviations

N/A	not applicable
PICS	Protocol Implementation Conformance Statement

[11] *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this annex so that it can be used for its intended purpose and may further publish the completed PICS.

A.3 Instructions for completing the PICS proforma

A.3.1 General structure of the PICS proforma

The first part of the PICS proforma, implementation identification and protocol summary, is to be completed as indicated with the information necessary to identify fully both the supplier and the implementation.

The main part of the PICS proforma is a fixed-format questionnaire, divided into several subclauses, each containing a number of individual items. Answers to the questionnaire items are to be provided in the rightmost column, either by simply marking an answer to indicate a restricted choice (usually Yes or No) or by entering a value or a set or range of values. (Note that there are some items where two or more choices from a set of possible answers can apply; all relevant choices are to be marked.)

Each item is identified by an item reference in the first column. The second column contains the question to be answered; the third column records the status of the item—whether support is mandatory, optional, or conditional: see also A.3.4. The fourth column contains the reference or references to the material that specifies the item in the main body of this standard, and the fifth column provides the space for the answers.

A supplier may also provide (or be required to provide) further information, categorized as either Additional Information or Exception Information. When present, each kind of further information is to be provided in a further subclause of items labeled Ai or Xi, respectively, for cross-referencing purposes, where i is any unambiguous identification for the item (e.g., simply a numeral). There are no other restrictions on its format and presentation.

A completed PICS proforma, including any Additional Information and Exception Information, is the Protocol Implementation Conformation Statement for the implementation in question.

NOTE—Where an implementation is capable of being configured in more than one way, a single PICS may be able to describe all such configurations. However, the supplier has the choice of providing more than one PICS, each covering some subset of the implementation's configuration capabilities, in case that makes for easier and clearer presentation of the information.

A.3.2 Additional information

Items of Additional Information allow a supplier to provide further information intended to assist the interpretation of the PICS. It is not intended or expected that a large quantity will be supplied, and a PICS can be considered complete without any such information. Examples might be an outline of the ways in which a (single) implementation can be set up to operate in a variety of environments and configurations, or information about aspects of the implementation that are outside the scope of this standard but that have a bearing on the answers to some items.

References to items of Additional Information may be entered next to any answer in the questionnaire and may be included in items of Exception Information.

A.3.3 Exception information

It may occasionally happen that a supplier will wish to answer an item with mandatory status (after any conditions have been applied) in a way that conflicts with the indicated requirement. No preprinted answer will be found in the Support column for this item. Instead, the supplier shall write the missing answer into

the Support column, together with an *Xi* reference to an item of Exception Information, and shall provide the appropriate rationale in the Exception item itself.

An implementation for which an Exception item is required in this way does not conform to this standard.

NOTE—A possible reason for the situation described previously is that a defect in this standard has been reported, a correction for which is expected to change the requirement not met by the implementation.

A.3.4 Conditional status

A.3.4.1 Conditional items

The PICS proforma contains a number of conditional items. These are items for which both the applicability of the item itself, and its status if it does apply—mandatory or optional—are dependent on whether certain other items are supported.

Where a group of items is subject to the same condition for applicability, a separate preliminary question about the condition appears at the head of the group, with an instruction to skip to a later point in the questionnaire if the "Not Applicable" answer is selected. Otherwise, individual conditional items are indicated by a conditional symbol in the Status column.

A conditional symbol is of the form "**pred:** S" where **pred** is a predicate as described in A.3.4.2 below, and S is a status symbol, M or O.

If the value of the predicate is true (see A.3.4.2), the conditional item is applicable, and its status is indicated by the status symbol following the predicate: The answer column is to be marked in the usual way. If the value of the predicate is false, the "Not Applicable" (N/A) answer is to be marked.

A.3.4.2 Predicates

A predicate is one of the following:

a) An item-reference for an item in the PICS proforma: The value of the predicate is true if the item is marked as supported and is false otherwise.

b) A predicate-name, for a predicate defined as a boolean expression constructed by combining item-references using the boolean operator OR: The value of the predicate is true if one or more of the items is marked as supported.

c) The logical negation symbol "¬" prefixed to an item-reference or predicate-name: The value of the predicate is true if the value of the predicate formed by omitting the "¬" symbol is false, and vice versa.

Each item whose reference is used in a predicate or predicate definition, or in a preliminary question for grouped conditional items, is indicated by an asterisk in the Item column.

A.4 PICS proforma—<RAP implementations>

A.4.1 Implementation identification

Supplier	
Contact point for queries about the PICS	
Implementation Name(s) and Version(s)	
Other information necessary for full identification, e.g., name(s) and version(s) of machines and/or operating system names	
NOTE 1—Only the first three items are required for all implementations; other information may be completed as appropriate in meeting the requirement for full identification. NOTE 2—The terms “Name” and “Version” should be interpreted appropriately to correspond with a supplier’s terminology (e.g., Type, Series, Model).	

A.4.2 Protocol summary

Identification of protocol specification	IEEE Std 802.1DD, Resource Allocation Protocol			
Identification of amendments and corrigenda to the PICS proforma that have been completed as part of the PICS	Amd.	:	Corr.	: :
	Amd.	:	Corr.	:
Have any Exception items been required? (See A.3.3: the answer “Yes” means that the implementation is not conformant).	No [ ]		Yes [ ]	

Date of Statement	

A.5 Major capabilities

Item	Feature	Status	References	Support
RI	Is a RAP instance implemented?	O	5.4	Yes [ ] No [ ]
RB	Is a RAP Bridge implemented?	O	5.5	Yes [ ] No [ ]
FRB	FRER-capable RAP Bridge implemented	O	5.6	Yes [ ] No [ ]
RE	Is a RAP end station implemented?	O	5.7	Yes [ ] No [ ]
FRT	FRER-capable RAP Talker implemented?	O	5.8	Yes [ ] No [ ]
FRL	FRER-capable RAP Listener implemented?	O	5.9	Yes [ ] No [ ]

A.6 RAP instance capabilities

Item	Feature	Status	References	Support
RI-1	Does the implementation support the RAP MAD ingress state machine?	RRI:M	5.4 item a), 6.6.3	Yes [ ]
RI-2	Does the implementation support the RAP MAD egress state machine?	RRI:M	5.4 item b), 6.6.6	Yes [ ]
RI-3	Does the implementation support the RAP Endpoint state machine?	RRI:M	5.7 item b), 6.6.10	Yes [ ]
RI-4	Does the implementation support the RAP MAP state machine?	RRI:M	5.5 item d), 6.7.4	Yes [ ]
RI-5	Does the implementation support the RAP instance event state machine?	RRI:M	5.4 item c), 6.8.1	Yes [ ]
RI-6	Does the implementation support the Application Information TLV?	RRI:M	5.4 item d), 6.6.8.3	Yes [ ]
RI-7	Does the implementation support the RAP attributes and their TLV encodings?	RRI:M	5.4 item e), 6.4	Yes [ ]
RI-8	Does the implementation support the RAP management?	RRI:M	5.4 item f), Clause 7	Yes [ ]

A.7 RAP Bridge capabilities

Item	Feature	Status	References	Support
RB-1	Does the implementation include a single conformant VLAN Bridge component?	RNB:M	5.5 item a), [Q]:5.4	Yes [ ]
RB-2	Does the implementation conform to the required and optional behaviors for a relay system?	RNB:M	5.5 item b), [CS]:5.6, [CS]:5.7	Yes [ ]
RB-3	Does the implementation include a single conformant RAP instance?	RNB:M	5.5 item c), 5.4	Yes [ ]

A.8 FRER-capable RAP Bridge capabilities

Item	Feature	Status	References	Support
FRB-1	Does the implementation conform to the requirements for a RAP Bridge?	FRB:M	5.5	Yes [ ]
FRB-2	Does the implementation conform to the required and optional behaviors for a FRER C-component?	FRB:M	5.6 item a), [CB]:5.15	Yes [ ]
FRB-3	Does the implementation support Active Destination MAC and VLAN Stream identification function?	FRB:M	5.6 item b), [CB]:6.6	Yes [ ]

A.9 RAP end station capabilities

Item	Feature	Status	References	Support
RE-1	Does the implementation conform to the required behaviors for a end system?	RNE:M	5.7 item a), [CS]:5.4	Yes [ ]
RE-2	Does the implementation support the operation of the Applicant state machine and the Registrar state machine?	RNE:M	5.7 item c), [CS]:8.3, [CS]:8.4	Yes [ ]
RE-3	Does the implementation include a single conformant RAP instance?	RNE:M	5.7 item d), 5.4	Yes [ ]

A.10 FRER-capable RAP Talker capabilities

Item	Feature	Status	References	Support
FRT-1	Does the implementation conform to the requirements for a RAP end station ?	FRT:M	5.7	Yes [ ]
FRT-2	Does the implementation conform to the required, recommended and optional behaviors for a Talker end system?	FRT:M	5.8 item a), [CB]:5.6, [CB]:5.7, [CB]:5.8	Yes [ ]

A.11 FRER-capable RAP Listener capabilities

Item	Feature	Status	References	Support
FRL-1	Does the implementation conform to the requirements for a RAP end station ?	FRL:M	5.7	Yes [ ]
FRL-2	Does the implementation conform to the required, recommended and optional behaviors for a Talker end system?	FRL:M	5.9 item a), [CB]:5.9, [CB]:5.10, [CB]:5.11	Yes [ ]

Annex B

(informative)

Resource allocation examples

This Annex provides some examples of resource allocation using the Resource Allocation Protocol (RAP) specified in this standard. These examples are chosen only for the purpose of illustrating the operation of resource allocation under different scenarios, and are not intended to constraint the range of applicability of RAP in terms of all of the possible usage scenarios.

Note that, unless otherwise stated, the examples in this Annex are based on the following assumptions:

a) In Talker Announce propagation (6.7.2), no further conditions, other than those related to the VLAN Context rule as defined in 6.7.2.1, that would prevent a Talker Announce registration from being propagated to other Port(s), are met.
b) Resource allocation is successful without encountering any problems that would cause Talker Announcement or Listener Attachment to fail (except the example in B.5)

B.1 Single-path stream example

This subclause gives an example of resource allocation for transmission of a stream along a single path between the Talker and each Listener. Figure B-1 shows the VLAN topology configured in the network for use as a VLAN Context in the resource allocation for the stream.

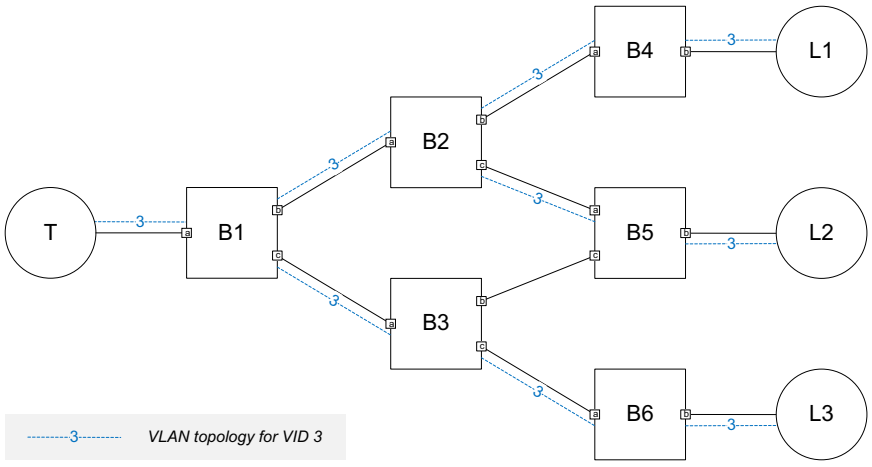

Figure B-1—Single-path stream example: topology and VLAN configuration

Figure B-2 illustrates the propagation of a Single-Context Talker Announcement [item a) in 6.7.2.1] initiated by the Talker within the VLAN Context. Note that the Talker Announcement does not go through the link between B3 and B5, as this link is not part of the VLAN Context in use.

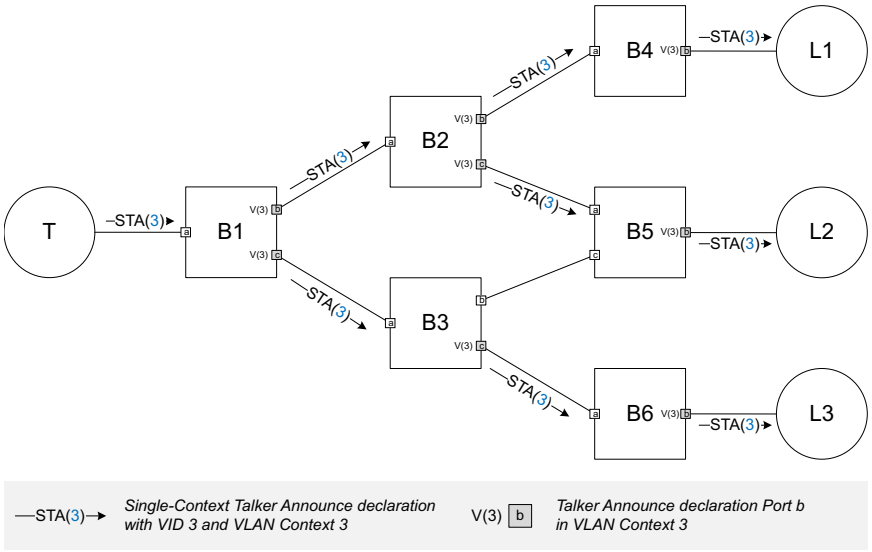

Figure B-2—Single-path stream example: Talker Announcement

As illustrated in Figure B-3, three Listeners participate in the Single-Context Listener Attachment [item a) in 6.3.7.1], which is propagated along the path previously traversed by the associated Talker Announcement in reversed direction and is subject to Listener Attach merge (6.3.7.3) on both Port B2.a and Port B1.a.

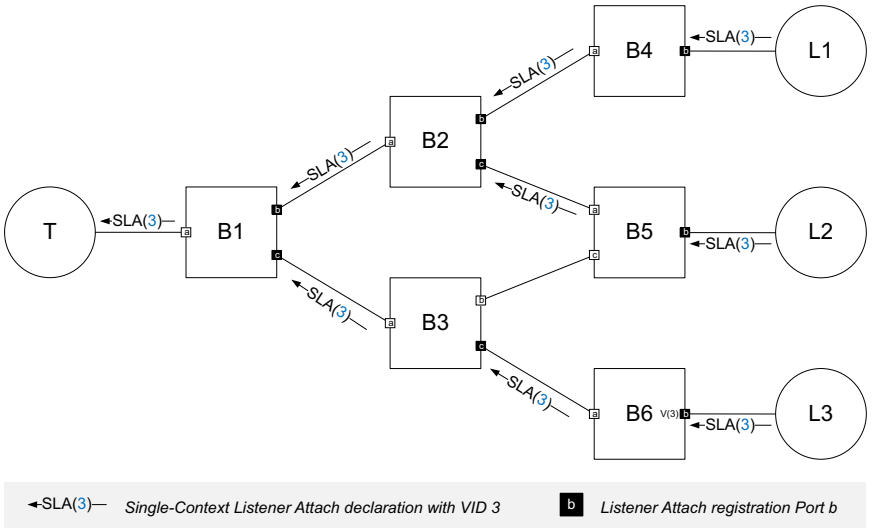

Figure B-3—Single-path stream example: Listener Attachment

Figure B-4 shows the single path established for transmission of the stream from the Talker to each Listener after successful resource allocation.

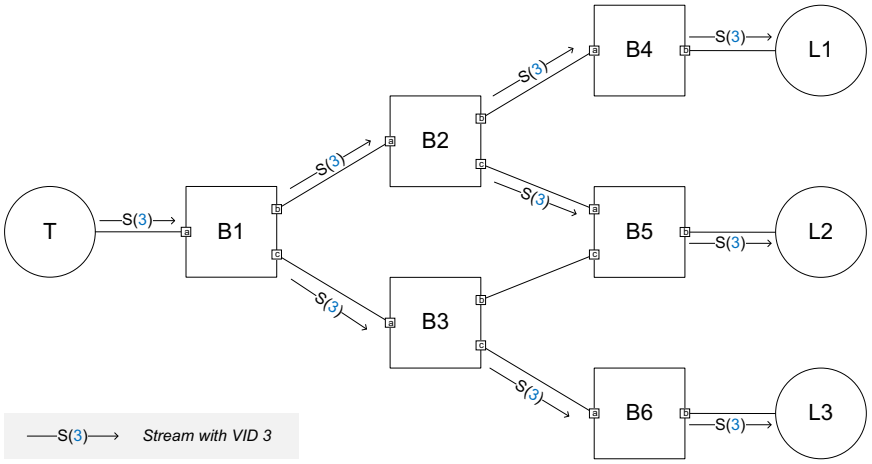

Figure B-4—Single-path stream example: established stream

B.2 Multi-path stream example 1

This subclause gives an example of resource allocation for transmission of a Compound Stream in a network analogous to the one described in [CB]:C.1. As shown in Figure B-5, three VLAN topologies representing an instance of redundant trees constitute a Redundancy Context used in the resource allocation. In this example, FRER functionality is provided only in three FRER-capable end stations.

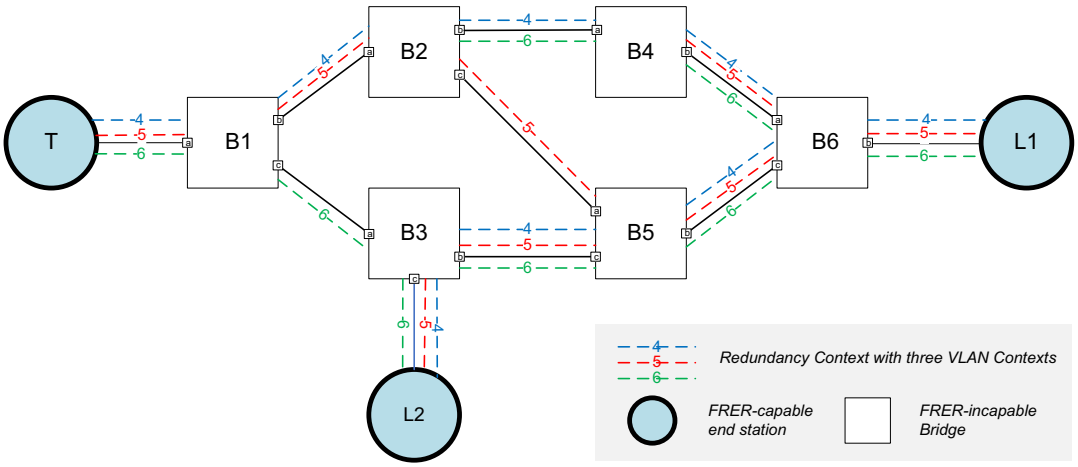

Figure B-5—Multi-path stream example 1: topology and VLAN configuration

As illustrated in Figure B-6, the FRER-capable Talker initiates a Multi-Context Talker Announcement [item b) in 6.7.2.1] with three separate Talker Announce declarations, each using a different VID and VLAN Context in the Redundancy Context to represent a Member Stream split from the Compound Stream. Since there are no FRER-capable Bridges in the network, each of these Talker Announce declarations is propagated and processed by the Bridges in the network independently.

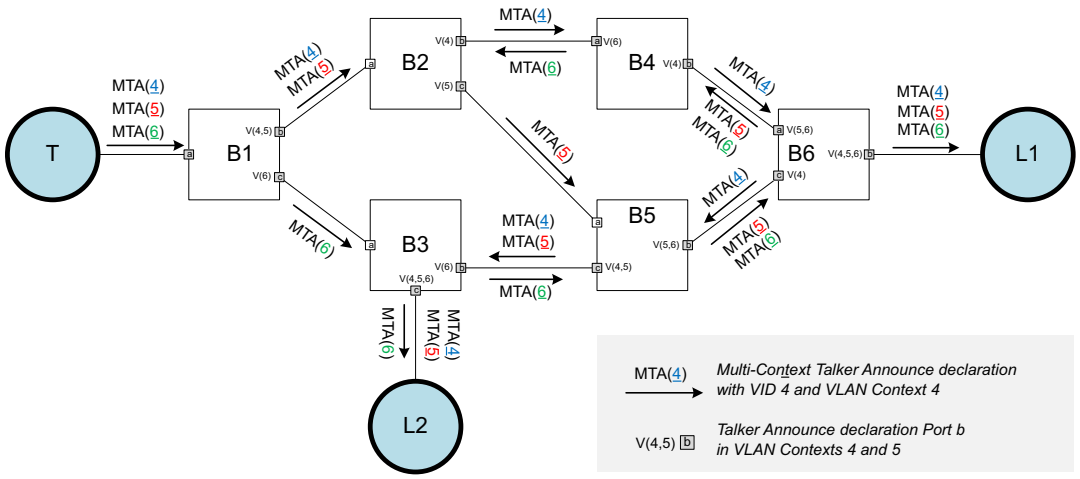

Figure B-6—Multi-path stream example 1: Talker Announcement

Figure B-7 illustrates the Multi-Context Listener Attachment associated with the Multi-Context Talker Announcement shown in Figure B-6. Each Listener makes three Listener Attach declarations, each with the same VID and VLAN Context as received in one of the three Talker Announce registrations. Each Listener Attach declaration made by a Listener is propagated along the path formed by the set of Bridge Ports that contain the associated Taker Announce registration. Listener Attach merge (6.3.7.3) occurs on the Bridge Ports B5.a and B6.a where two Listener Attach registrations propagated from the different Listeners in the same VLAN Context are merged into a joint Listener Attach declaration, indicating the need for the Bridge to multicast the Member Stream received on that Port.

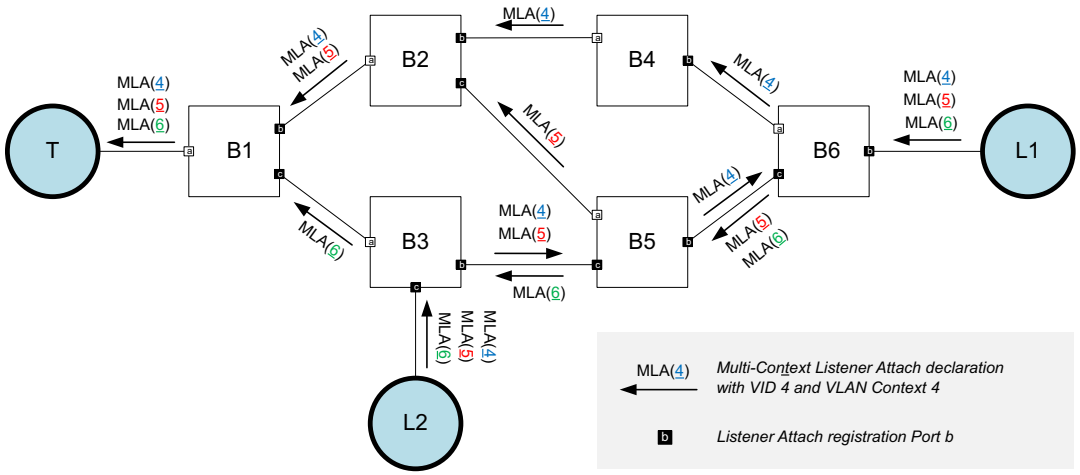

Figure B-7—Multi-path stream example 1: Listener Attachment

Figure B-8 shows the paths established and the FRER functions configured for redundant transmission of the Compound Stream with three Member Streams after successful resource allocation.

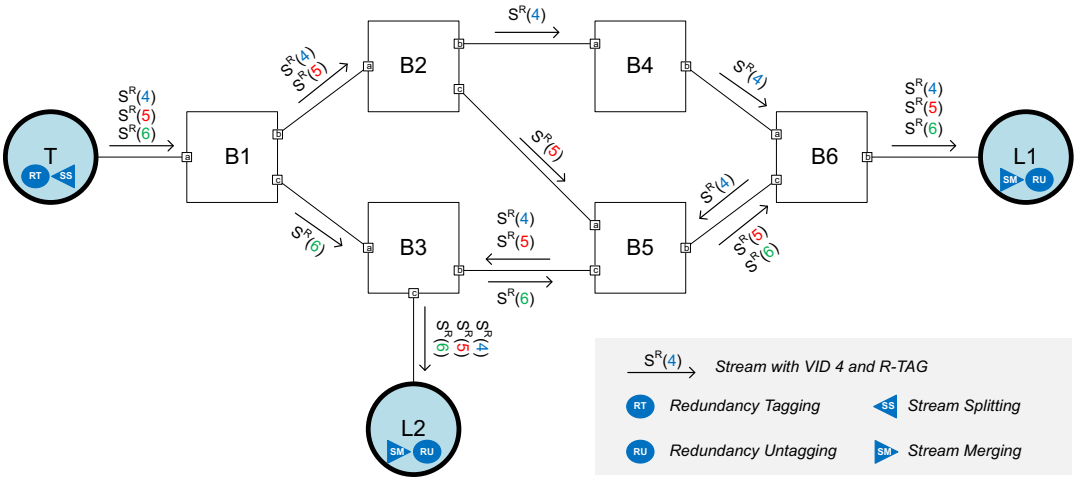

Figure B-8—Multi-path stream example 1: established stream

B.3 Multi-path stream example 2

This subclause gives an example of resource allocation for transmission of a Compound Stream in a network analogous to the one described in [CB]:C.2. As shown in Figure B-9, this example has the same physical topology as that in Figure B-5, but different placement of FRER-capable stations and VLAN configuration. As both the Talker and Listener are not FRER-capable, three FRER-capable Bridges B1, B5 and B6 are placed at the edge of the network as proxies to offer FRER functionality to the end stations. Additionally, another FRER-capable Bridge B2 is placed within the network to split the VLAN topologies, in conjunction with B1, into three disjoint paths. Note that the reason for using a different VLAN configuration in this example is to ensure the VLAN membership configuration in each FRER-capable Bridge where stream splitting is expected, e.g., B1 and B2, meet the condition described in b).) of 6.3.11.4.

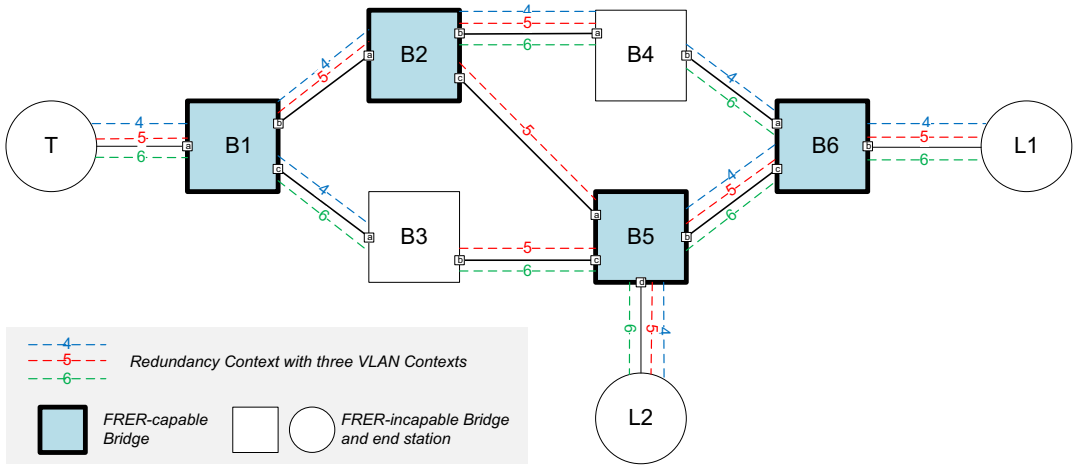

Figure B-9—Multi-path stream example 2: topology and VLAN configuration

As illustrated in Figure B-10, the Talker initiates a Multi-Context Talker Announcement by making a Talker Announce declaration for all of the three VLAN Contexts and with RTagStatus (6.4.3.9.1) set FALSE. The RTagStatus flag of the Talker Announcement is set TRUE by B1, which is responsible for redundancy tagging in accordance with the rule described in a) of 6.3.11.3. Both B1 and B2 are responsible for applying stream splitting to the stream, as all the conditions described in b) of 6.3.11.4 are met. The Talker Announcement is subject to Talker Announce merge (6.3.6.1) on the Bridge Ports B5.b, B5.d and B6.b, each corresponding to a potential stream merging port for the stream, in accordance with the rule described in a) of 6.3.11.5. In addition, the RTagStatus flag of the Talker Announcement is set FALSE on B5.d and B6.b, as the conditions for redundancy untagging are met on these two Ports, in accordance with the rule described in b) of 6.3.11.3.

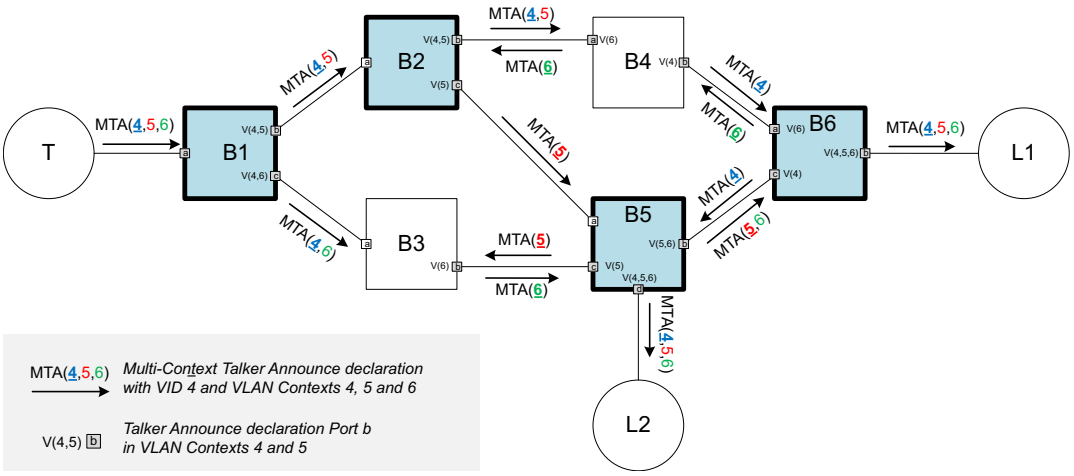

Figure B-10—Multi-path stream example 2: Talker Announcement

Figure B-11 illustrates the Multi-Context Listener Attachment associated with the Multi-Context Talker Announcement shown in Figure B-10. Each Listener makes a single Listener Attach declaration with the same VLAN Contexts as received in the Talker Announce registration and a desired VID chosen from among these VLAN Context IDs. Similar to that in Figure B-7, the Listener Attach declaration on both B5.a and B6.a is merged from the two Listener Attach registrations propagated from different Listeners in the same VLAN Context. Additionally in this example, Listener Attach merge (6.3.7.3) occurs on the Bridge Ports B2.a and B1.a where two Listener Attach registrations containing the split VLAN Contexts are merged into a single Listener Attach declaration with joint VLAN Contexts, as a reserve operation of splitting the Talk Announcement previously performed on the same Port. Note that the Listener Attach declaration on B2.a and B1.a takes a VID that is contained in the VLAN Context(s) of both of the Listener Attach registrations merging into that Listener Attach declaration, because the multicast forwarding mechanism used in stream splitting by FRER-capable Bridges requires stream transmission Ports to be in the member set for a common VID.

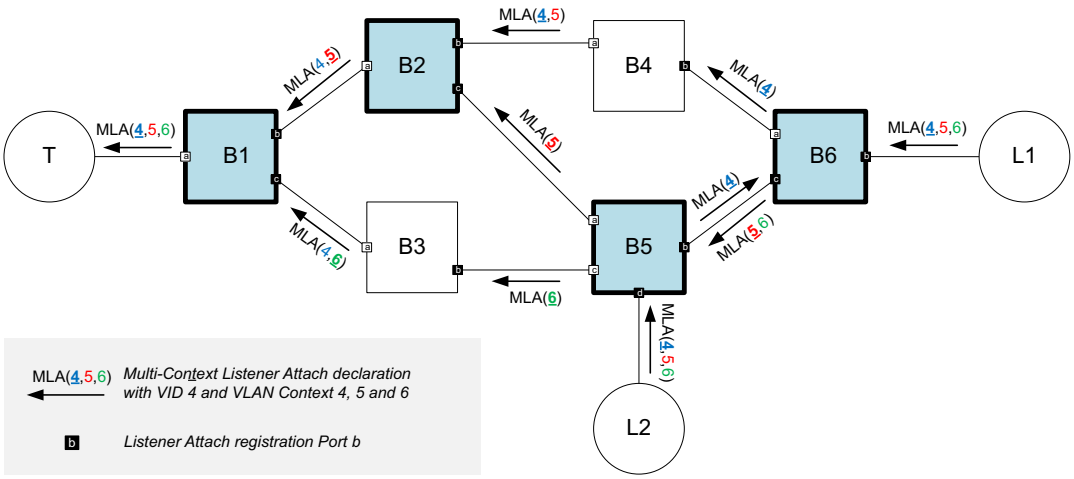

Figure B-11—Multi-path stream example 2: Listener Attachment

Figure B-12 shows the paths established and the FRER functions configured for redundant transmission of the Compound Stream with three Member Streams after successful resource allocation.

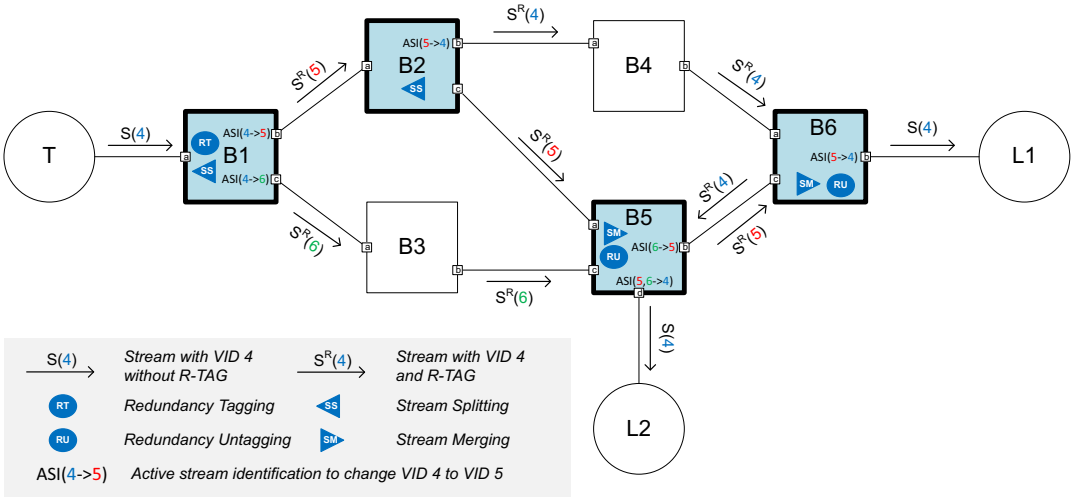

Figure B-12—Multi-path stream example 2: established stream

B.4 Multi-path stream example 3

This subclause gives an example of resource allocation for transmission of a Compound Stream in a network analogous to the one described in [CB]:C.3. As shown in Figure B-13, FRER functionality is provided in each end station and some of the Bridges. Four FRER-capable Bridges are connected as a ring, each acting as both a stream splitting point and a stream merging point, to provide protection against multiple simultaneous failures. A Redundancy Context with two VLAN topologies is configured in the network. In order to meet the stream splitting condition described in b).) of 6.3.11.4, each FRER-capable Bridge has one VLAN whose member set includes all the Bridge Ports. As a consequence, each of those Bridge Ports marked as VLAN topology termination point needs be excluded from the member set for the indicated

VLAN and to have ingress filtering ([Q]:8.6.2) enabled for that VLAN, in order to ensure loop-free Talker Announce propagation.

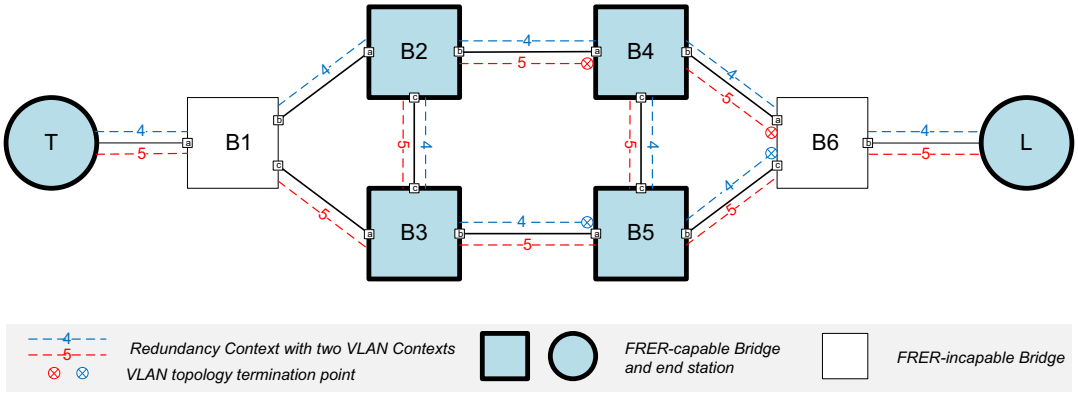

Figure B-13—Multi-path stream example 3: topology and VLAN configuration

As illustrated in Figure B-14, two Talker Announce declarations initiated by the FRER-capable Talker, each with a distinct VLAN Context, are propagated by B1 separately. According to the VLAN configuration, each FRER-capable Bridge propagates the Talker Announce registration on Port a with a single VLAN Context to the other two Ports, and meanwhile merges on Port b two Talker Announce registrations propagated from the other two Ports into a joint Talker Announce declaration with both VLAN contexts. Note that on each Bridge Port that is excluded from the member set for one VLAN Context, ingress blocking (6.7.2.3) ensures that the Talker Announce registration received with two VLAN Contexts is propagated only with the other VLAN Context for which the Port in the member set.

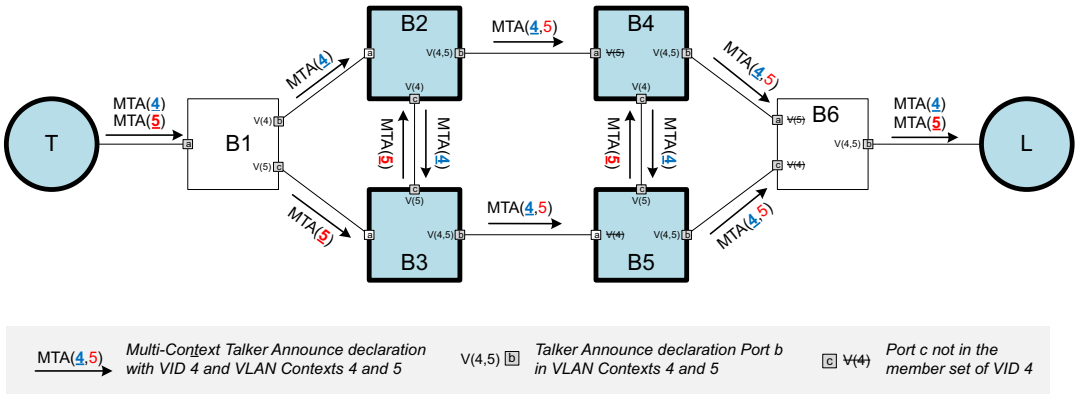

Figure B-14—Multi-path stream example 3: Talker Announcement

Figure B-15 illustrates the Multi-Context Listener Attachment associated with the Multi-Context Talker Announcement shown in Figure B-14. The FRER-capable Listener makes two Listener Attach declarations, each with a different VLAN Context. Listener Attach merge occurs on Port a of each FRER-capable Bridge, resulting in a joint Listener Attach declaration with the same VLAN Context as that contained in the

associated Talker Announce registration on the same Port. Finally, B1 passes two Listener Attach declarations, each with a different VLAN Context, to the Talker.

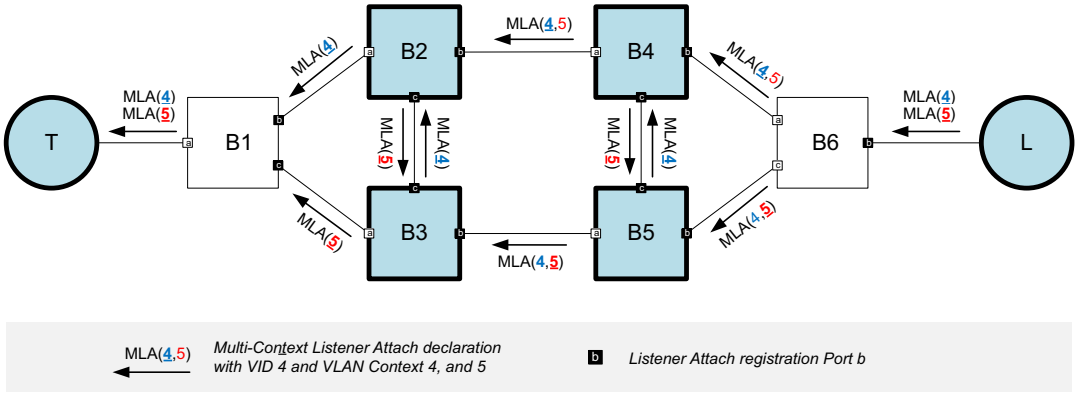

Figure B-15—Multi-path stream example 3: Listener Attachment

Figure B-16 shows the paths established and the required FRER functions configured for redundant transmission of the Compound Stream with two Member Streams after successful resource allocation

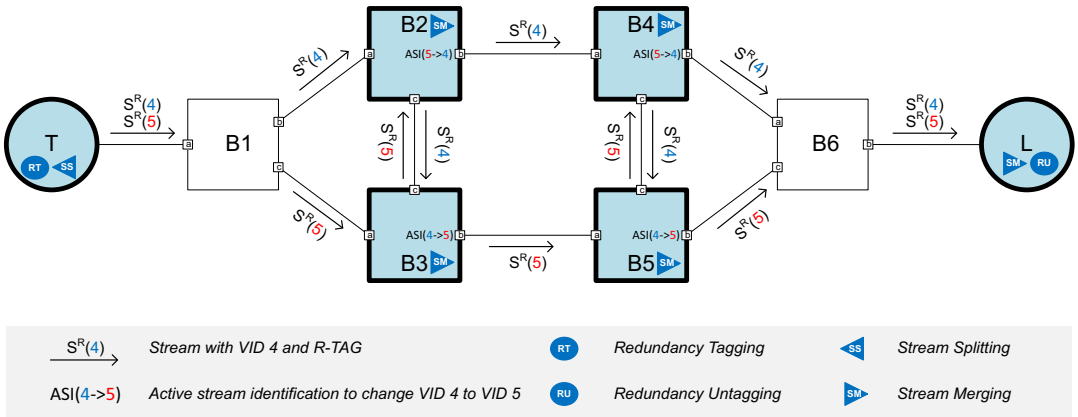

Figure B-16—Multi-path stream example 3: established stream

B.5 Example of status notification and failure diagnosis

This subclause uses the example network of B.4 to illustrate the status notification and failure diagnosis mechanisms in resource allocation. As shown in Figure B-17, it is assumed that the Multi-Context Talker Announcement encounters resource problems on three Bridge Ports.

The problem on Port B1.b causes the Talker Announce declaration on that Port to change to the Announce Fail status [item b) in 6.3.6.2] and to attach the failure information about the problem on that Port to VLAN Context 4. The failure information generated on B1.b remains attached to VLAN Context 4 in the subsequent propagation of the Talker Announcement across the network toward the Listener, regardless of the situations encountered on any downstream Port. Similarly, the problems on B3.b and B4.b lead to Announce Fail in the Talker Announce declarations on those Ports. The failure information of VLAN Context 5 is generated on B3.b, and is later merged along with that of VLAN Context 4 to the Talker Announce declarations on B4.b and B5.b. The reason for the Announce Success status on both B2.b and B5.b is that the Talker Announce declaration merges one Talker Announce registration propagated as

Announce Success, meaning that a reservation can be made for a stream on a merged path as long as at least one of the paths merging into that merged path is available for reservation.

In the end, the Listener receives two Talker Announce declarations, which indicate that at least one path in the network can be reserved for transmission of the Compound Stream as indicated by the Announce Success status of one Talker Announce registration, even though there are problems detected at the locations associated with both VLAN topologies as indicated in the supplied failure information.

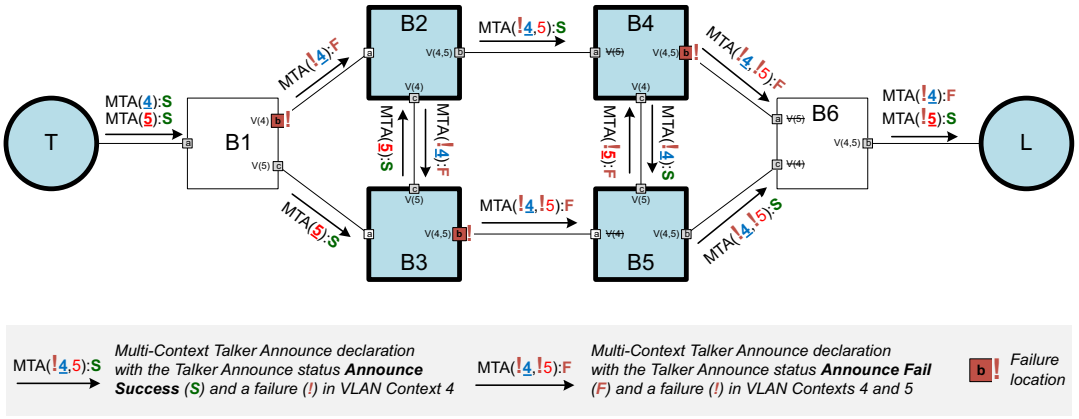

Figure B-17—Example Talker Announce status and failure information

Figure B-18 illustrates the Multi-Context Listener Attachment associated with the Talker Announcement shown in Figure B-17. The Listener makes two Listener Attach declarations with Attach Ready contained in the Listener Attach status (6.3.7.4), indicating it is ready to receive both Member Streams. As the Listener Attachment is propagated along each stream path back to the Talker, a corresponding reservation is made on each Port for a Listener Attach registration with the Attach Ready status and its associated Talker Announce declaration with the Announce Success status. A Listener Attach registration for which a reservation cannot be made is propagated as Attach Fail. On each Bridge Port where Listener Attach merge occurs in this example, i.e., Port a or each FRER-capable Bridge, the Listener Attach declaration is made with Attach Ready because one of the two Listener Attach registrations being merged on that Port is propagated as Attach Ready. Note that, on those Ports with a terminated VLAN, such as B6.a and B6.c, the Listener Attach declaration still contains the terminated VLAN (e.g., VLAN Context 5 on B6.a) in order to match the VLAN Contexts contained in the associated Talker Announce registration on the same Port, but needs to set the corresponding Path status to Unresponsive Path to indicate that Listener Attach declaration does not originate from that VLAN Context.

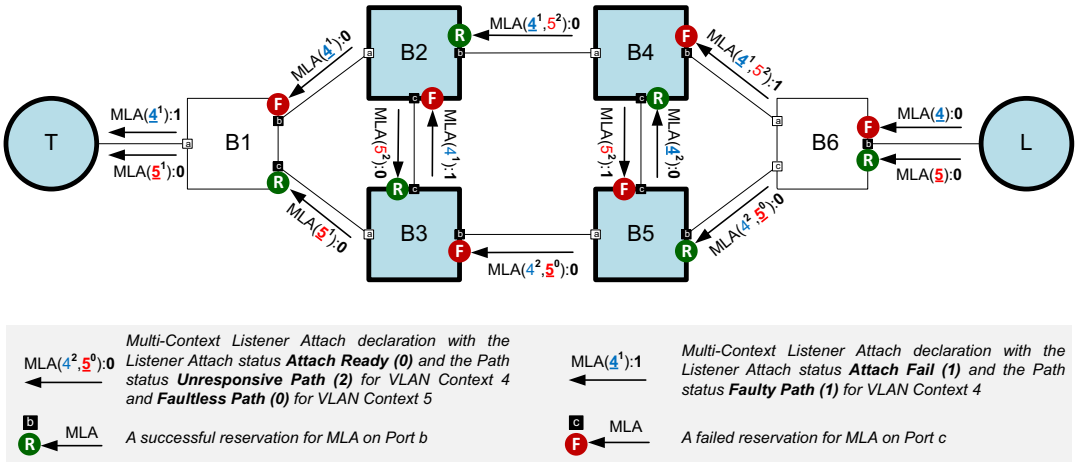

Figure B-18—Example Listener Attach status and Path status

Figure B-19 shows the single path established and the FRER functions configured for the Compound Stream in the case of three problems in the network.

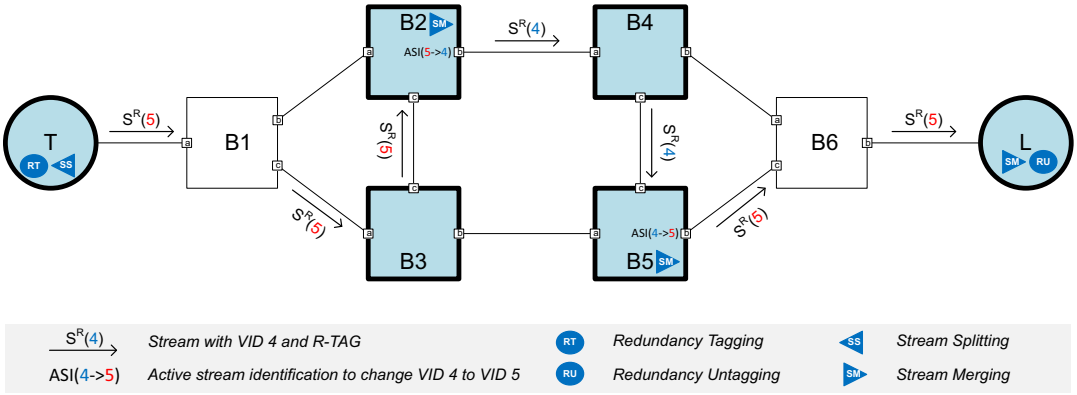

Figure B-19—Example partially established Compound Stream

Annex C

(informative)

Bibliography

Bibliographical references are resources that provide additional or helpful material but do not need to be understood or used to implement this standard. Reference to these resources is made for informational use only.

[B1] Bridge-Local Guaranteed Latency with Strict Priority Schedulin, https://www.ieee802.org/1/files/public/docs2020/dd-grigorjew-strict-priority-latency-0320-v02.pdf.

[B2] Calculating the Delay Added by Qav Stream Queue, http://www.ieee802.org/1/files/public/docs2009/av-fuller-queue-delay-calculation-0809-v02.pdf.

[B3] IEEE Std 802™-2014, IEEE Standard for Local and metropolitan area networks—Overview and Architecture.[12,13]

[B4] IEEE Std 802.1AB™-2016, IEEE Standard for Local and metropolitan area networks—Station and Media Access Control Connectivity Discovery.

[B5] IEEE Std 802.1AC™-2016, IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Service Definition.

[B6] IEEE Std 802.1BA™-2021, IEEE Standard for Local and metropolitan area networks—Audio Video Bridging (AVB) Systems.

[B7] IEEE Std 802.1CB™-2017, IEEE Standard for Local and metropolitan area networks—Frame Replication and Elimination for Reliability.

[B8] IEEE Std 802.1CS™-2020, IEEE Standard for Local and metropolitan area networks—Link-local Registration Protocol.

[B9] IEEE Std 802.1Q™-2022, IEEE Standard for Local and metropolitan area networks—Bridges and Bridged Networks.

[B10] IEEE Std 802.3™-2022, IEEE Standard for Ethernet.

[B11] IETF RFC 1633, Integrated Services in the Internet Architecture: An Overview, June 1994, https://tools.ietf.org/html/rfc1633.

[B12] IETF RFC 2210, The Use of RSVP with IETF Integrated Services, Sept. 1997, https://tools.ietf.org/html/rfc2210.

[B13] IETF RFC 2212, Specification of Guaranteed Quality of Service, Sept. 1997, https://tools.ietf.org/html/rfc2212.

[B14] IETF RFC 2215, General Characterization Parameters for Integrated Service Network Elements, Sept. 1997, https://tools.ietf.org/html/rfc2215.

[B15] IETF RFC 2475, An Architecture for Differentiated Services, Dec. 1998, https://tools.ietf.org/html/rfc2475.

[12] IEEE publications are available from The Institute of Electrical and Electronics Engineers (https://standards.ieee.org/).

[13] The IEEE standards or products referred to in this annex are trademarks of The Institute of Electrical and Electronics Engineers, Inc.

[B16] IETF RFC 2814, SBM (Subnet Bandwidth Manager): A Protocol for Admission Control over IEEE 802-style Networks, May 2000, https://tools.ietf.org/html/rfc2814.

[B17] IETF RFC 2815, Integrated Service Mappings on IEEE 802 Networks, May 2000, https://tools.ietf.org/html/rfc2815.

[B18] IETF RFC 4960, Stream Control Transmission Protocol, Sept. 2007, https://tools.ietf.org/html/rfc4960.

[B19] IETF RFC 6087, Guidelines for Authors and Reviewers of YANG Data Model Documents, January 2011.

[B20] IETF RFC 6241, Network Configuration Protocol (NETCONF), June 2011.

[B21] IETF RFC 6328, IANA Considerations for Network Layer Protocol Identifiers, July 2011, https://tools.ietf.org/html/rfc6328.

[B22] IETF RFC 6329, IS-IS Extensions Supporting IEEE 802.1aq Shortest Path Bridging, Apr. 2012, https://tools.ietf.org/html/rfc6329.

[B23] IETF RFC 6536, Network Configuration Protocol (NETCONF) Access Control Model, March 2012.

[B24] IETF RFC 6991, Common YANG Data Types, July 2013.

[B25] IETF RFC 7223, A YANG Data Model for Interface Management, May 2014.

[B26] IETF RFC 7224, IANA Interface Type YANG Module, May 2014.

[B27] IETF RFC 7317, A YANG Data Model for System Management, August 2014.

[B28] IETF RFC 7813, IS-IS Path Control and Reservation, 2016, https://tools.ietf.org/html/rfc7813.

[B29] IETF RFC 7950, The YANG 1.1 Data Modeling Language, August 2016.

[B30] IETF RFC 8069, URN Namespace for IEEE, February 2017.

[B31] IETF RFC 8200, Internet Protocol, Version 6 (IPv6) Specification, July 2017, https://tools.ietf.org/html/rfc8200.

[B32] MEF Technical Specification 41 (MEF 41), Generic Token Bucket Algorithm.

[B33] OMG Unified Modeling Language (OMG UML), Version 2.5, March 2015.

[B34] Specht, J., and S. Samii, “Urgency-Based Scheduler for Time-Sensitive Switched Ethernet Networks,” *28th Euromicro Conference on Real-Time Systems (ECRTS)*, pp. 75–85, 2016.

[B35] Latency Model and Example Reservation Flow in RAP, https://www.ieee802.org/1/files/public/docs2023/dd-grigorjew-latency-model-0123-v03.pdf.

[B36] Alexej Grigorjew, and Tobias Hossfeld, “Decentralized Latency Models and Information Flow for TSN Shapers”, <link>

Annex D

(normative)

RA class template definitions

This Annex provides template specific parameter definitions for RA attributes (6.4.2) and for TA attributes (6.4.3).

D.1 RA class template IDs

Table D-1—IEEE 802.1 RA class templates

RTID	Transmission Selection Algorithm	Latency Reference Points	Reference
00-80-C2-00	Strict Priority ([Q]:8.6.8.1)	Fully received to fully received	D.2
00-80-C2-01	Strict Priority ([Q]:8.6.8.1)	Shaper to shaper	D.3
00-80-C2-02	ATS Transmission Selection ([Q]:8.6.8.5)	Shaper to shaper	D.4
00-80-C2-03	Credit-Based Shaper Algorithm ([Q]:8.6.8.2)	Fully received to fully received	D.5
00-80-C2-04	Strict Priority with fan-in / fan-out optimization	Shaper to shaper	D.6

D.2 RA class template definitions for RTID 00-80-C2-00

D.2.1 Introduction

This RA class template uses the Strict Priority Transmission Selection Algorithm with fully received to fully received latency reference points (Figure 6-4).

D.2.2 Parameters

This class does not use template specific data for the RA attribute and TA attribute.

D.2.3 Calculation Examples

This subclause provides a possible method to calculate the effective burstiness, AccuMinLatency and AccuMaxLatency for the SP transmission selection algorithm as an example.

The maximum effective burstiness b_i^{eff} of a stream i at any point in the network is calculated as follows:

$$b_i^{eff}(\mathrm{Down}) = b_i^{eff}(\mathrm{Up}) + r_i \cdot \left(\delta_{Pi}^{seg} - \frac{l_i^{min}}{R^{seg}} \right)$$

The downstream accumulated latencies are calculated as follows:

$$L_i^{max}\ (\text{Downstream}) = L_i^{max}\ (\text{Upstream}) + \delta_{Pi}^{seg}$$
$$L_i^{min}\ (\text{Downstream}) = L_i^{min}\ (\text{Upstream}) + l_i^{min} / R^{seg}$$

with

b_i^{eff} = Network Tspec CommittedBurstSize [item 4) of 6.3.9.2],
r_i = Network TSpec CommittedInformationRate [item 3) of 6.3.9.2],
δ_{Pi}^{seg} = Max latency threshold of the delay segment for priority Pi [item d) of 6.7.5.9]
l_i^{min} = MinFrameSize of stream i [item 2) of 6.3.9.2]
R^{seg} = Link speed of egress Port

D.3 RA class template definitions for RTID 00-80-C2-01

D.3.1 Introduction

This RA class template uses the Strict Priority Transmission Selection Algorithm with shaper to shaper latency (Figure 6-4).

D.3.2 Parameters

This class does not use template specific data for the RA attribute and TA attribute.

D.3.3 Calculation Examples

This subclause provides a possible method to calculate the effective burstiness, AccuMinLatency and AccuMaxLatency for the SP transmission selection algorithm as an example.

The maximum effective burstiness b_i^{eff} of a stream i at any point in the network is calculated as follows:

$$b_i^{eff}(\text{Down}) = b_i^{eff}(\text{Up}) + r_i \cdot \left(\delta_{Pi}^{seg} - \frac{l_i^{min}}{R^{seg}} \right)$$

The downstream accumulated latencies are calculated as follows:

$$L_i^{max}\ (\text{Downstream}) = L_i^{max}\ (\text{Upstream}) + \delta_{Pi}^{seg}$$
$$L_i^{min}\ (\text{Downstream}) = L_i^{min}\ (\text{Upstream}) + l_i^{min} / R^{seg}$$

with

b_i^{eff} = Network Tspec CommittedBurstSize [item 4) of 6.3.9.2],
r_i = Network TSpec CommittedInformationRate [item 3) of 6.3.9.2],
δ_{Pi}^{seg} = Max latency threshold of the delay segment for priority Pi [item d) of 6.7.5.9]
l_i^{min} = MinFrameSize of stream i [item 2) of 6.3.9.2]
R^{seg} = Link speed of egress Port

D.4 RA class template definitions for RTID 00-80-C2-02

D.4.1 Introduction

This RA class template uses simple (not optimized) ATS Transimssion Selection with shaper to shaper latency (Figure 6-4).

D.4.2 Parameters

This class does not use template specific data for the RA attribute and TA attribute.

D.4.3 Calculation Examples

This subclause provides a possible method to calculate the effective burstiness, AccuMinLatency and AccuMaxLatency for the SP transmission selection algorithm as an example.

The maximum effective burstiness b_i^{eff} of a stream i at any point in the network is calculated as follows:

$$b_i^{eff}(\text{Down}) = b_i^{eff}(\text{Up})$$

The downstream accumulated latencies are calculated as follows:

$$L_i^{max}\ (\text{Downstream}) = L_i^{max}\ (\text{Upstream}) + \delta_{Pi}^{seg}$$
$$L_i^{min}\ (\text{Downstream}) = L_i^{min}\ (\text{Upstream}) + l_i^{min}/R^{seg}$$

with

b_i^{eff} = Network Tspec CommittedBurstSize [item 4) of 6.3.9.2],
δ_{Pi}^{seg} = Max latency threshold of the delay segment for priority Pi [item d) of 6.7.5.9]
l_i^{min} = MinFrameSize of stream i [item 2) of 6.3.9.2]
R^{seg} = Link speed of egress Port

D.5 RA class template definitions for RTID 00-80-C2-03

D.5.1 Introduction

This RA class template uses the Credit Based Shaper together with the Stream Reservation Protocol as defined in IEEE Std 802.1BA-2021.

D.5.2 Parameters

The content of the OrganizationallyDefinedData for the RA class specific data sub-TLV is the SRclassVID.

	Length	Offset
SRclassVID	12 bits	0

Figure D-1—SR class VID

D.5.3 Calculation Examples

AccuMaxLatency is calculated following the equations provided in IEEE Std 802.1BA-2021, Clause 6.6.

D.6 RA class template definitions for RTID 00-80-C2-04

D.6.1 Introduction

This RA class template uses the the fan-in / fan-out information to optimize the reservation calculation for Strict Priority Transmission Selection Algorithm with shaper to shaper latency (Figure 6-4).

D.6.2 Parameters

This class does not use template specific data for the RA attribute.

The content of the OrganizationallyDefinedData for the RA class specific data sub-TLV (6.4.3.11) is a list of Port entries as shown in Figure D-2.

	Length	Offset
1 or more Port list entries	variable	0

Figure D-2—List of Port entries

	Length	Offset
Ingress Port	2	0
Bandwidth	2	2

Figure D-3—Port list entry

D.6.3 Calculation Examples

For calculation of effective burstiness, AccuMinLatency and AccuMaxLatency for fan-in / fan-out optimized SP Transmission Selection Algorithm cf. [B36].